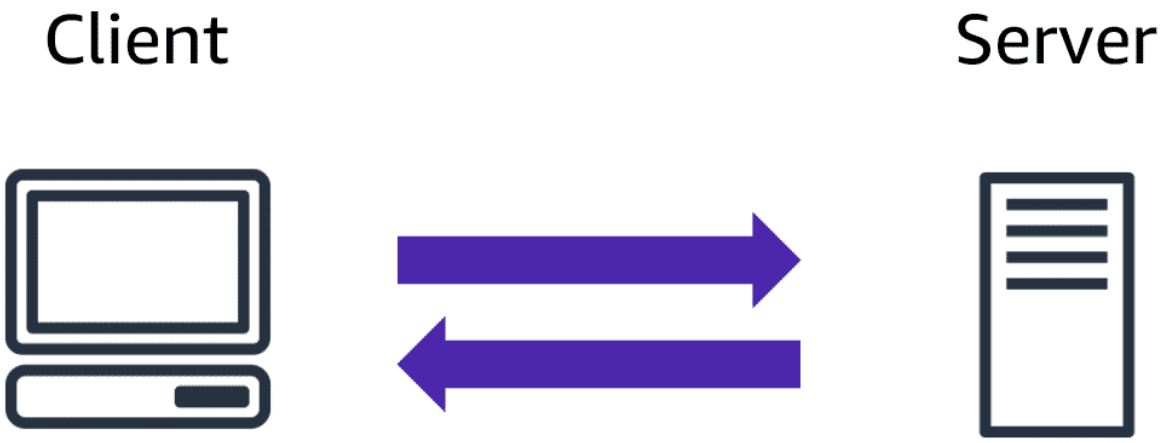
AWS CLOUD PRACTITIONER STUDY GUIDE

What is a client-server model?

You just learned more about AWS and how almost all of modern computing uses a basic client-server model. Let’s recap what a client-server model is.



In computing, a **client** can be a web browser or desktop application that a person interacts with to make requests to computer servers. A **server** can be services, such as Amazon Elastic Compute Cloud (Amazon EC2) – a type of virtual server.

For example, suppose that a client makes a request for a news article, the score in an online game, or a funny video. The server evaluates the details of this request and fulfills it by returning the information to the client.

Cloud Computing:

Cloud computing is the on-demand delivery of IT resources over the internet with pay-as-you-go pricing. Let's break this down. On-demand delivery indicates that AWS has the resources you need, when you need them. You don't need to tell us in advance that you're going to need them. Suddenly you find yourself needing 300 virtual servers. Well, just a few clicks and launch them. Or you need 2000 terabytes of storage. You don't have to tell us in advance, just start using the storage you need,

when you need it. Don't need them anymore, just as quickly, you can return them and stop paying immediately. That kind of flexibility is just not possible when you're managing your own data centers.

The idea of IT resources is actually a big part of the AWS philosophy. We often get asked why AWS has so many products and the answer is really simple: Because businesses need them. If there are IT elements that are common across a number of businesses, then this is not a differentiator.

Take a MySQL database as an example. If your business runs a MySQL database, does your ability to install the MySQL engine make you a better company than your competitors? Well, probably not that. Do you keep backups in a way that makes you superior to other players in your vertical? Again, doubtful. The data inside your database, now that's critically different. The way you build your tables and manage the structures, absolutely separates you from the competition. But the engine is just the engine.

At AWS, we call that the undifferentiated heavy lifting of IT. Tasks that are common, often repetitive and ultimately time-consuming; these are the tasks AWS wants to help you with. So you can focus on what makes you unique. Over the internet, seems simple enough, but it implies that you can access those resources using a secure webpage console or programmatically.

No additional contracts or sales calls are needed. With pay-as-you-go pricing, we re-emphasize what we pointed out here in the coffee shop. You don't staff a shop with employees 24 hours a day at the same levels you do during peak hours. In fact, some hours, you might not even staff them at all. So why pay for developer environments, for example, on weekends, if your developers aren't working on the weekends?

Deployment models for cloud computing

When selecting a cloud strategy, a company must consider factors such as required cloud application components, preferred resource management tools, and any legacy IT infrastructure requirements.

The three cloud computing deployment models are cloud-based, on-premises, and hybrid.

1.Cloud Based Deployment

- Run all parts of the application in the cloud.
- Migrate existing applications to the cloud.
- Design and build new applications in the cloud.

In a cloud-based deployment model, you can migrate existing applications to the cloud, or you can design and build new applications in the cloud. You can build those applications on low-level infrastructure that requires your IT staff to manage them. Alternatively, you can build them using higher-level services that reduce the management, architecting, and scaling requirements of the core infrastructure.

For example, a company might create an application consisting of virtual servers, databases, and networking components that are fully based in the cloud.

2.On premise Deployment

- Deploy resources by using virtualization and resource management tools.
- Increase resource utilization by using application management and virtualization technologies.

On-premises deployment is also known as a *private cloud* deployment. In this model, resources are deployed on premises by using virtualization and resource management tools.

For example, you might have applications that run on technology that is fully kept in your on-premises data center. Though this model is much like legacy IT infrastructure, its incorporation of application management and virtualization technologies helps to increase resource utilization.

3.hybrid Deployment

- Connect cloud-based resources to on-premises infrastructure.
- Integrate cloud-based resources with legacy IT applications.

In a **hybrid deployment**, cloud-based resources are connected to on-premises infrastructure. You might want to use this approach in a number of situations. For example, you have legacy applications that are better maintained on premises, or government regulations require your business to keep certain records on premises.

For example, suppose that a company wants to use cloud services that can automate batch data processing and analytics. However, the company has several legacy applications that are more suitable on premises and will not be migrated to the cloud. With a hybrid deployment, the company would be able to keep the legacy applications on premises while benefiting from the data and analytics services that run in the cloud.

Benefits of cloud computing

Consider why a company might choose to take a particular cloud computing approach when addressing business needs.

To learn more about the benefits, expand each for the following six categories.

1. **Trade upfront expense for variable expense**

Upfront expense refers to data centers, physical servers, and other resources that you would need to invest in before using them. Variable expense means you only pay for computing resources you consume instead of investing heavily in data centers and servers before you know how you're going to use them.

By taking a cloud computing approach that offers the benefit of variable expense, companies can implement innovative solutions while saving on costs.

2. **Stop spending money to run and maintain data centers**

Computing in data centers often requires you to spend more money and time managing infrastructure and servers.

A benefit of cloud computing is the ability to focus less on these tasks and more on your applications and customers.

3. **Stop guessing capacity**

With cloud computing, you don't have to predict how much infrastructure capacity you will need before deploying an application.

For example, you can launch Amazon EC2 instances when needed, and pay only for the compute time you use. Instead of paying for unused resources or having to deal with limited capacity, you can access only the capacity that you need. You can also scale in or scale out in response to demand.

4. **Benefit from massive economies of scale**

By using cloud computing, you can achieve a lower variable cost than you can get on your own.

Because usage from hundreds of thousands of customers can aggregate in the cloud, providers, such as AWS, can achieve higher economies of scale. The economy of scale translates into lower pay-as-you-go prices.

5. **Increase speed and agility**

The flexibility of cloud computing makes it easier for you to develop and deploy applications.

This flexibility provides you with more time to experiment and innovate. When computing in data centers, it may take weeks to obtain new resources that you need. By comparison, cloud computing enables you to access new resources within minutes.

6. **Go global in minutes**

The global footprint of the AWS Cloud enables you to deploy applications to customers around the world quickly, while providing them with low latency. This means that even if you are located in a different part of the world than your customers, customers are able to access your applications with minimal delays.

Later in this course, you will explore the AWS global infrastructure in greater detail. You will examine some of the services that you can use to deliver content to customers around the world.

Amazon Elastic Compute Cloud (Amazon EC2)

Amazon Elastic Compute Cloud (Amazon EC2)(opens in a new tab) provides secure, resizable compute capacity in the cloud as Amazon EC2 instances.

Imagine you are responsible for the architecture of your company's resources and need to support new websites. With traditional on-premises resources, you have to do the following:

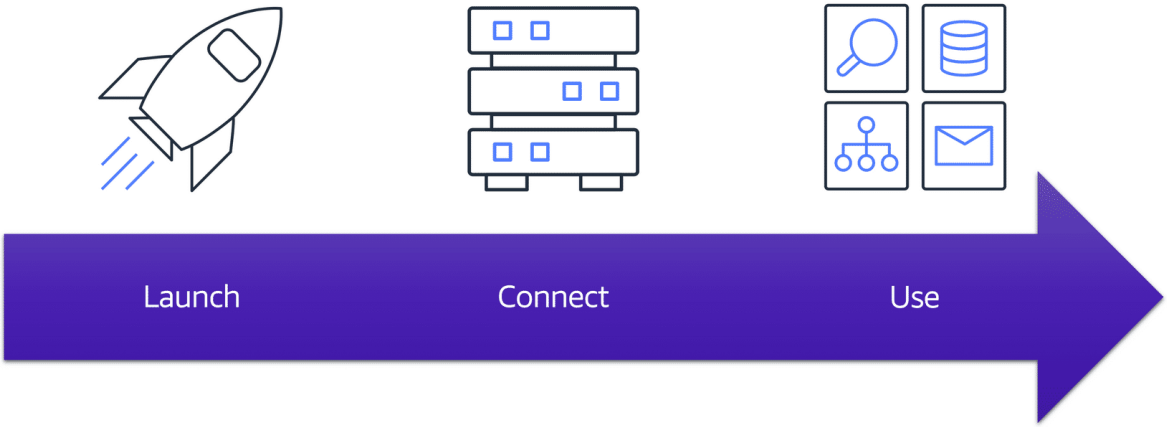
- Spend money upfront to purchase hardware.
- Wait for the servers to be delivered to you.
- Install the servers in your physical data center.
- Make all the necessary configurations.

By comparison, with an Amazon EC2 instance you can use a virtual server to run applications in the AWS Cloud.

- You can provision and launch an Amazon EC2 instance within minutes.
- You can stop using it when you have finished running a workload.
- You pay only for the compute time you use when an instance is running, not when it is stopped or terminated.
- You can save costs by paying only for server capacity that you need or want.

How Amazon EC2 works

To learn more, choose each numbered marker.



Launch

First, you launch an instance. Begin by selecting a template with basic configurations for your instance. These configurations include the operating system, application server, or applications. You also select the instance type, which is the specific hardware configuration of your instance.

As you are preparing to launch an instance, you specify security settings to control the network traffic that can flow into and out of your instance. Later in this course, we will explore Amazon EC2 security features in greater detail.

Connect

Next, connect to the instance. You can connect to the instance in several ways. Your programs and applications have multiple different methods to connect directly to the instance and exchange data. Users can also connect to the instance by logging in and accessing the computer desktop.

Use

After you have connected to the instance, you can begin using it. You can run commands to install software, add storage, copy and organize files, and more.

Amazon EC2 instance types

Amazon EC2 instance types(opens in a new tab) are optimized for different tasks. When selecting an instance type, consider the specific needs of your workloads and applications. This might include requirements for compute, memory, or storage capabilities.

To learn more about Amazon EC2 instance types, expand each of the following five categories.

General purpose instances

General purpose instances provide a balance of compute, memory, and networking resources. You can use them for a variety of workloads, such as:

- application servers
- gaming servers
- backend servers for enterprise applications
- small and medium databases

Suppose that you have an application in which the resource needs for compute, memory, and networking are roughly equivalent. You might consider running it on a general purpose instance because the application does not require optimization in any single resource area.

Compute optimized instances

Compute optimized instances are ideal for compute-bound applications that benefit from high-performance processors. Like general purpose instances, you can use compute optimized instances for workloads such as web, application, and gaming servers.

However, the difference is compute optimized applications are ideal for high-performance web servers, compute-intensive applications servers, and dedicated gaming servers. You can also use compute optimized instances for batch processing workloads that require processing many transactions in a single group.

Memory optimized instances

Memory optimized instances are designed to deliver fast performance for workloads that process large datasets in memory. In computing, memory is a temporary storage area. It holds all the data and instructions that a central processing unit (CPU) needs to be able to complete actions. Before a computer program or application is able to run, it is loaded from storage into memory. This preloading process gives the CPU direct access to the computer program.

Suppose that you have a workload that requires large amounts of data to be preloaded before running an application. This scenario might be a high-performance database or a workload that involves performing real-time processing of a large amount of unstructured data. In these types of use cases, consider using a memory optimized instance. Memory optimized instances enable you to run workloads with high memory needs and receive great performance.

Accelerated computing instances

Accelerated computing instances use hardware accelerators, or coprocessors, to perform some functions more efficiently than is possible in software running on CPUs. Examples of these functions include floating-point number calculations, graphics processing, and data pattern matching.

In computing, a hardware accelerator is a component that can expedite data processing. Accelerated computing instances are ideal for workloads such as graphics applications, game streaming, and application streaming.

Storage optimized instances

Storage optimized instances are designed for workloads that require high, sequential read and write access to large datasets on local storage. Examples of workloads suitable for storage optimized instances include distributed file systems, data warehousing applications, and high-frequency online transaction processing (OLTP) systems.

In computing, the term input/output operations per second (IOPS) is a metric that measures the performance of a storage device. It indicates how many different input or output operations a device can perform in one second. Storage optimized instances are designed to deliver tens of thousands of low-latency, random IOPS to applications.

You can think of input operations as data put into a system, such as records entered into a database. An output operation is data generated by a server. An example of output might be the analytics performed on the records in a database. If you have an application that has a high IOPS requirement, a storage optimized instance can provide better performance over other instance types not optimized for this kind of use case.

Amazon EC2 pricing

With Amazon EC2, you pay only for the compute time that you use. Amazon EC2 offers a variety of pricing options for different use cases. For example, if your use case can withstand interruptions, you can save with Spot Instances. You can also save by committing early and locking in a minimum level of use with Reserved Instances.

To learn more Amazon EC2 pricing, choose each of the following five categories.

On-Demand

On-Demand Instances are ideal for short-term, irregular workloads that cannot be interrupted. No upfront costs or minimum contracts apply. The instances run continuously until you stop them, and you pay for only the compute time you use.

Sample use cases for On-Demand Instances include developing and testing applications and running applications that have unpredictable usage patterns. On-Demand Instances are not recommended for workloads that last a year or longer because these workloads can experience greater cost savings using Reserved Instances.

Reserved Instances

Reserved Instances are a billing discount applied to the use of On-Demand Instances in your account. There are two available types of Reserved Instances:

- Standard Reserved Instances
- Convertible Reserved Instances

You can purchase Standard Reserved and Convertible Reserved Instances for a 1-year or 3-year term. You realize greater cost savings with the 3-year option.

Standard Reserved Instances: This option is a good fit if you know the EC2 instance type and size you need for your steady-state applications and in which AWS Region you plan to run them. Reserved Instances require you to state the following qualifications:

- **Instance type and size:** For example, m5.xlarge
- **Platform description (operating system):** For example, Microsoft Windows Server or Red Hat Enterprise Linux
- **Tenancy:** Default tenancy or dedicated tenancy

You have the option to specify an Availability Zone for your EC2 Reserved Instances. If you make this specification, you get EC2 capacity reservation. This ensures that your desired amount of EC2 instances will be available when you need them.

Convertible Reserved Instances: If you need to run your EC2 instances in different Availability Zones or different instance types, then Convertible Reserved Instances might be right for you. Note: You trade in a deeper discount when you require flexibility to run your EC2 instances.

At the end of a Reserved Instance term, you can continue using the Amazon EC2 instance without interruption. However, you are charged On-Demand rates until you do one of the following:

- Terminate the instance.

- Purchase a new Reserved Instance that matches the instance attributes (instance family and size, Region, platform, and tenancy).

EC2 Instance Savings Plans

AWS offers Savings Plans for a few compute services, including Amazon EC2. **EC2 Instance Savings Plans** reduce your EC2 instance costs when you make an hourly spend commitment to an instance family and Region for a 1-year or 3-year term. This term commitment results in savings of up to 72 percent compared to On-Demand rates. Any usage up to the commitment is charged at the discounted Savings Plans rate (for example, $10 per hour). Any usage beyond the commitment is charged at regular On-Demand rates.

The EC2 Instance Savings Plans are a good option if you need flexibility in your Amazon EC2 usage over the duration of the commitment term. You have the benefit of saving costs on running any EC2 instance within an EC2 instance family in a chosen Region (for example, M5 usage in N. Virginia) regardless of Availability Zone, instance size, OS, or tenancy. The savings with EC2 Instance Savings Plans are similar to the savings provided by Standard Reserved Instances.

Unlike Reserved Instances, however, you don't need to specify up front what EC2 instance type and size (for example, m5.xlarge), OS, and tenancy to get a discount. Further, you don't need to commit to a certain number of EC2 instances over a 1-year or 3-year term. Additionally, the EC2 Instance Savings Plans don't include an EC2 capacity reservation option.

Later in this course, you'll review AWS Cost Explorer, which you can use to visualize, understand, and manage your AWS costs and usage over time. If you're considering your options for Savings Plans, you can use AWS Cost Explorer to analyze your Amazon EC2 usage over the past 7, 30, or 60 days. AWS Cost Explorer also provides customized recommendations for Savings Plans. These recommendations estimate how much you could save on your monthly Amazon EC2 costs, based on previous Amazon EC2 usage and the hourly commitment amount in a 1-year or 3-year Savings Plan.

Spot Instances

Spot Instances are ideal for workloads with flexible start and end times, or that can withstand interruptions. Spot Instances use unused Amazon EC2 computing capacity and offer you cost savings at up to 90% off of On-Demand prices.

Suppose that you have a background processing job that can start and stop as needed (such as the data processing job for a customer survey). You want to start and stop the processing job without affecting the overall operations of your business. If you make a Spot request and Amazon EC2 capacity is available, your Spot Instance launches. However, if you make a Spot request and Amazon EC2 capacity is unavailable, the request is not successful until capacity becomes available. The unavailable capacity might delay the launch of your background processing job.

After you have launched a Spot Instance, if capacity is no longer available or demand for Spot Instances increases, your instance may be interrupted. This might not pose any issues for your background processing job. However, in the earlier example of developing and testing applications, you would most likely want to avoid unexpected interruptions. Therefore, choose a different EC2 instance type that is ideal for those tasks.

Dedicated Hosts

Dedicated Hosts are physical servers with Amazon EC2 instance capacity that is fully dedicated to your use.

You can use your existing per-socket, per-core, or per-VM software licenses to help maintain license compliance. You can purchase On-Demand Dedicated Hosts and Dedicated Hosts Reservations. Of all the Amazon EC2 options that were covered, Dedicated Hosts are the most expensive.

Scaling Amazon EC2

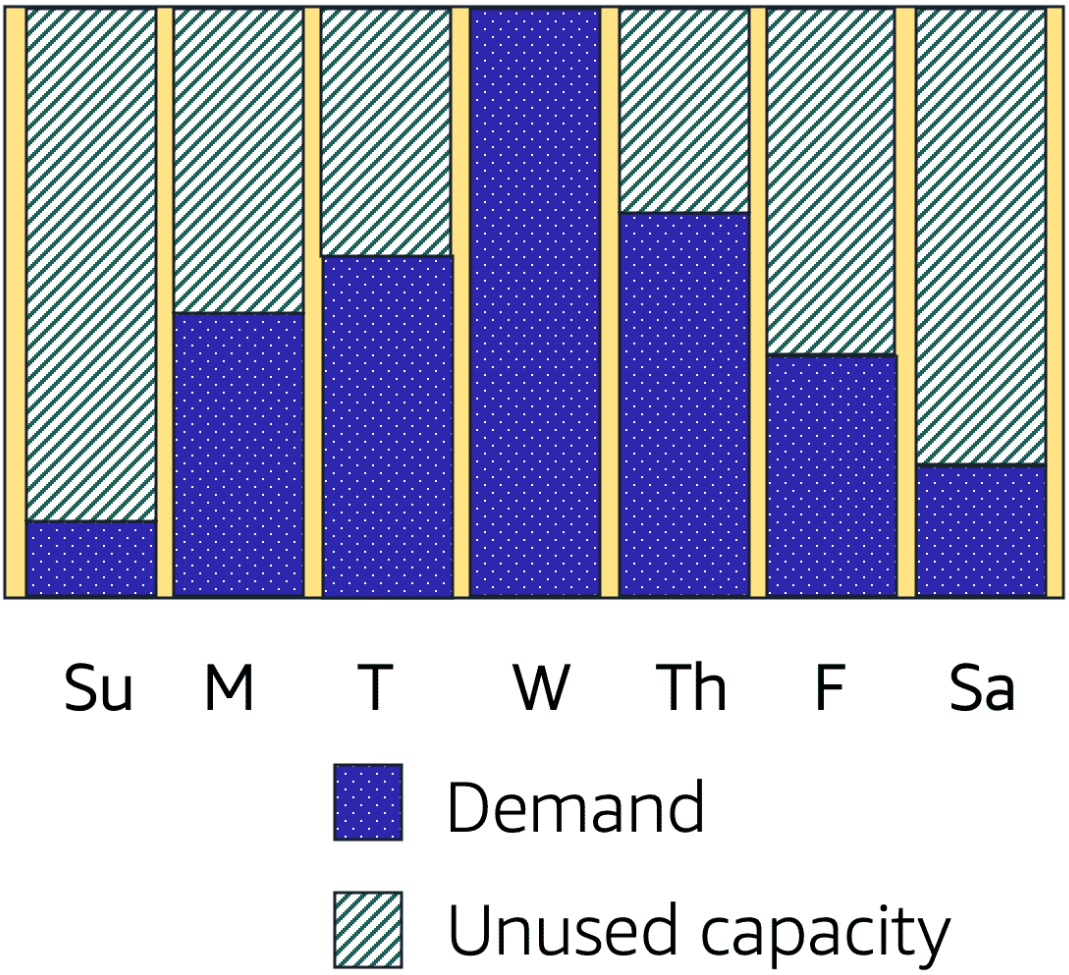
Scalability

Scalability involves beginning with only the resources you need and designing your architecture to automatically respond to changing demand by scaling out or in. As a result, you pay for only the resources you use. You don't have to worry about a lack of computing capacity to meet your customers' needs.

If you wanted the scaling process to happen automatically, which AWS service would you use? The AWS service that provides this functionality for Amazon EC2 instances is **Amazon EC2 Auto Scaling**.

Amazon EC2 Auto Scaling

If you've tried to access a website that wouldn't load and frequently timed out, the website might have received more requests than it was able to handle. This situation is similar to waiting in a long line at a coffee shop, when there is only one barista present to take orders from customers.



Amazon EC2 Auto Scaling enables you to automatically add or remove Amazon EC2 instances in response to changing application demand. By automatically scaling your instances in and out as needed, you can maintain a greater sense of application availability.

Within Amazon EC2 Auto Scaling, you can use two approaches: dynamic scaling and predictive scaling.

- *Dynamic scaling* responds to changing demand.
- *Predictive scaling* automatically schedules the right number of Amazon EC2 instances based on predicted demand.

To scale faster, you can use dynamic scaling and predictive scaling together.

Scaling Amazon EC2

To start the video Scaling Amazon EC2 (Part 1), choose the play button.

Play Video

Play

Loaded: 100.00%

Remaining Time -3:43

1x

Playback Rate 1x

Captions

Picture-in-PictureFullscreen

Mute

To access a transcript of the video, choose the following transcript.

Video transcript

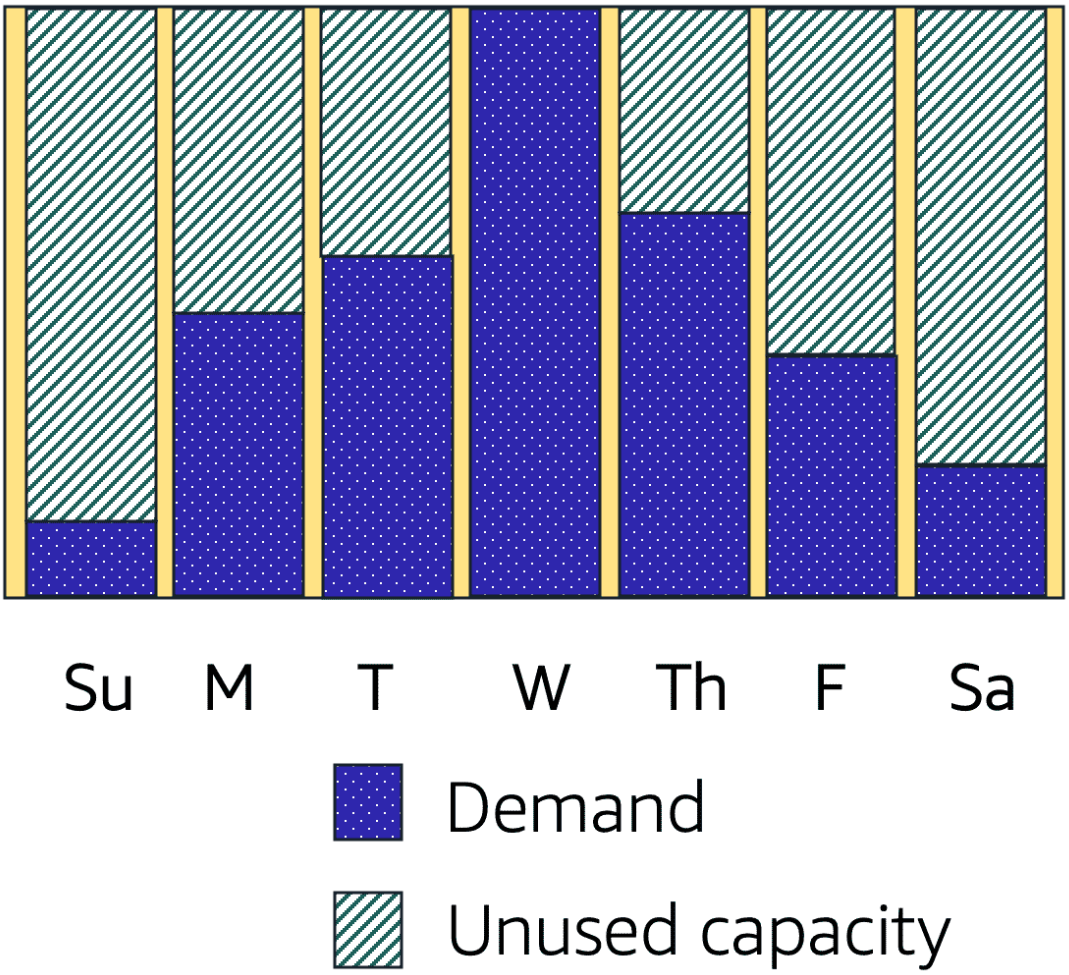
Scalability

Scalability involves beginning with only the resources you need and designing your architecture to automatically respond to changing demand by scaling out or in. As a result, you pay for only the resources you use. You don't have to worry about a lack of computing capacity to meet your customers' needs.

If you wanted the scaling process to happen automatically, which AWS service would you use? The AWS service that provides this functionality for Amazon EC2 instances is **Amazon EC2 Auto Scaling**.

Amazon EC2 Auto Scaling

If you've tried to access a website that wouldn't load and frequently timed out, the website might have received more requests than it was able to handle. This situation is similar to waiting in a long line at a coffee shop, when there is only one barista present to take orders from customers.



Amazon EC2 Auto Scaling enables you to automatically add or remove Amazon EC2 instances in response to changing application demand. By automatically scaling your instances in and out as needed, you can maintain a greater sense of application availability.

Within Amazon EC2 Auto Scaling, you can use two approaches: dynamic scaling and predictive scaling.

- *Dynamic scaling* responds to changing demand.
- *Predictive scaling* automatically schedules the right number of Amazon EC2 instances based on predicted demand.

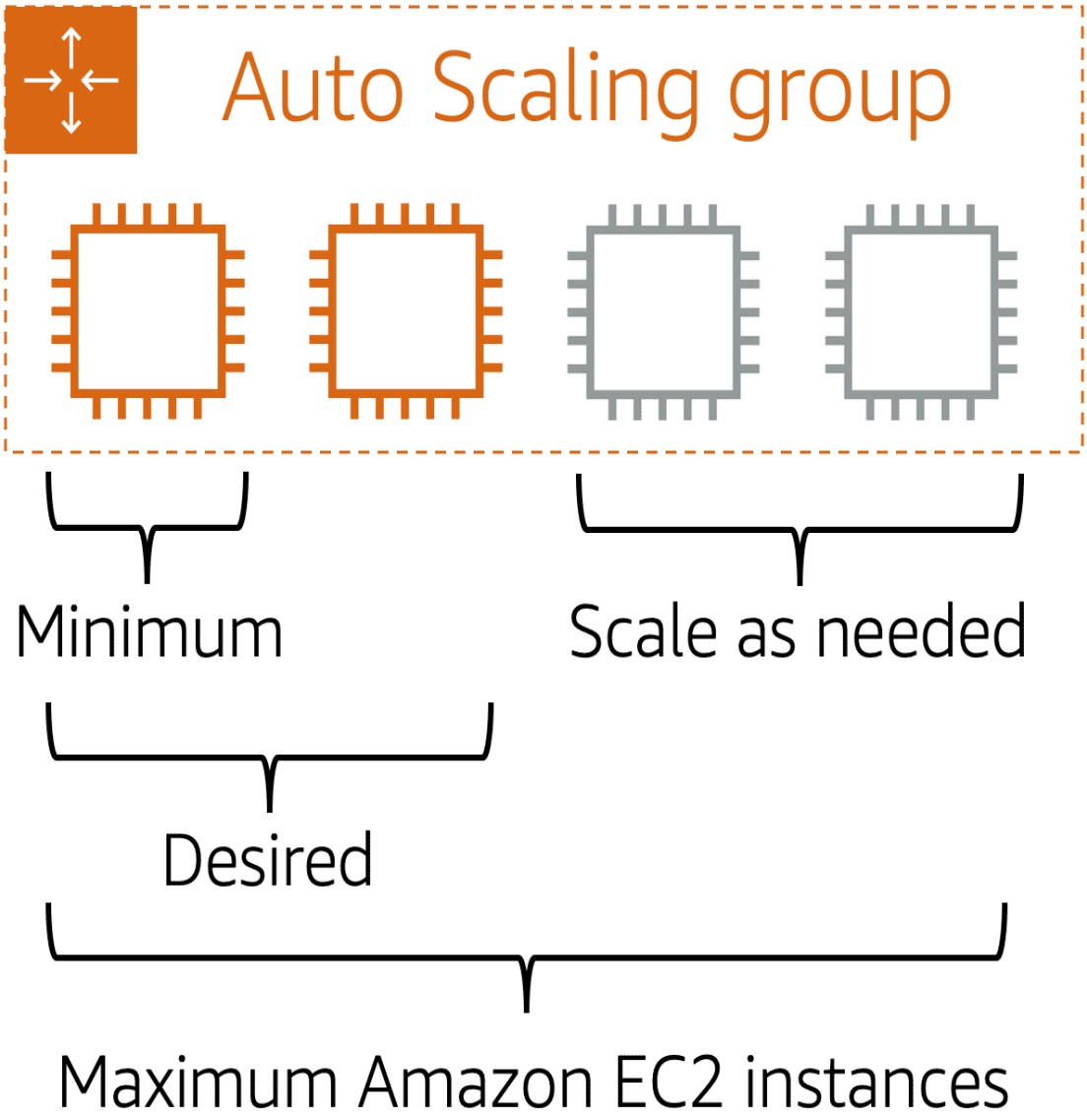
To scale faster, you can use dynamic scaling and predictive scaling together.

Example: Amazon EC2 Auto Scaling

In the cloud, computing power is a programmatic resource, so you can take a more flexible approach to the issue of scaling. By adding Amazon EC2 Auto Scaling to an application, you can add new instances to the application when necessary and terminate them when no longer needed.

Suppose that you are preparing to launch an application on Amazon EC2 instances. When configuring the size of your Auto Scaling group, you might set the minimum number of Amazon EC2

instances at one. This means that at all times, there must be at least one Amazon EC2 instance running.



When you create an Auto Scaling group, you can set the minimum number of Amazon EC2 instances. The **minimum capacity** is the number of Amazon EC2 instances that launch immediately after you have created the Auto Scaling group. In this example, the Auto Scaling group has a minimum capacity of one Amazon EC2 instance.

Next, you can set the **desired capacity** at two Amazon EC2 instances even though your application needs a minimum of a single Amazon EC2 instance to run.

If you do not specify the desired number of Amazon EC2 instances in an Auto Scaling group, the desired capacity defaults to your minimum capacity.

The third configuration that you can set in an Auto Scaling group is the **maximum capacity**. For example, you might configure the Auto Scaling group to scale out in response to increased demand, but only to a maximum of four Amazon EC2 instances.

Because Amazon EC2 Auto Scaling uses Amazon EC2 instances, you pay for only the instances you use, when you use them. You now have a cost-effective architecture that provides the best customer experience while reducing expenses.

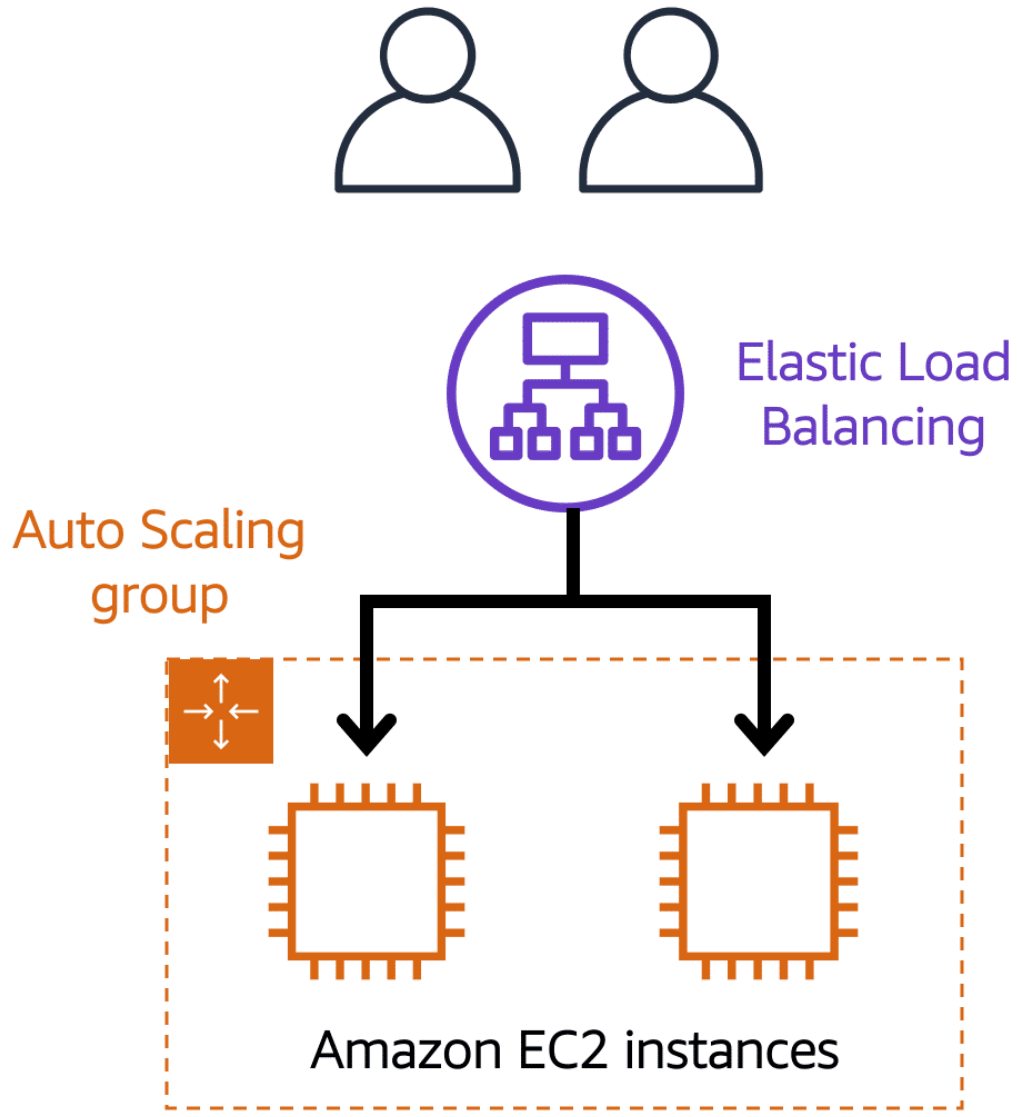
Elastic Load Balancing

Elastic Load Balancing is the AWS service that automatically distributes incoming application traffic across multiple resources, such as Amazon EC2 instances.

A load balancer acts as a single point of contact for all incoming web traffic to your Auto Scaling group. This means that as you add or remove Amazon EC2 instances in response to the amount of incoming traffic, these requests route to the load balancer first. Then, the requests spread across multiple resources that will handle them. For example, if you have multiple Amazon EC2 instances, Elastic Load Balancing distributes the workload across the multiple instances so that no single instance has to carry the bulk of it.

Although Elastic Load Balancing and Amazon EC2 Auto Scaling are separate services, they work together to help ensure that applications running in Amazon EC2 can provide high performance and availability.

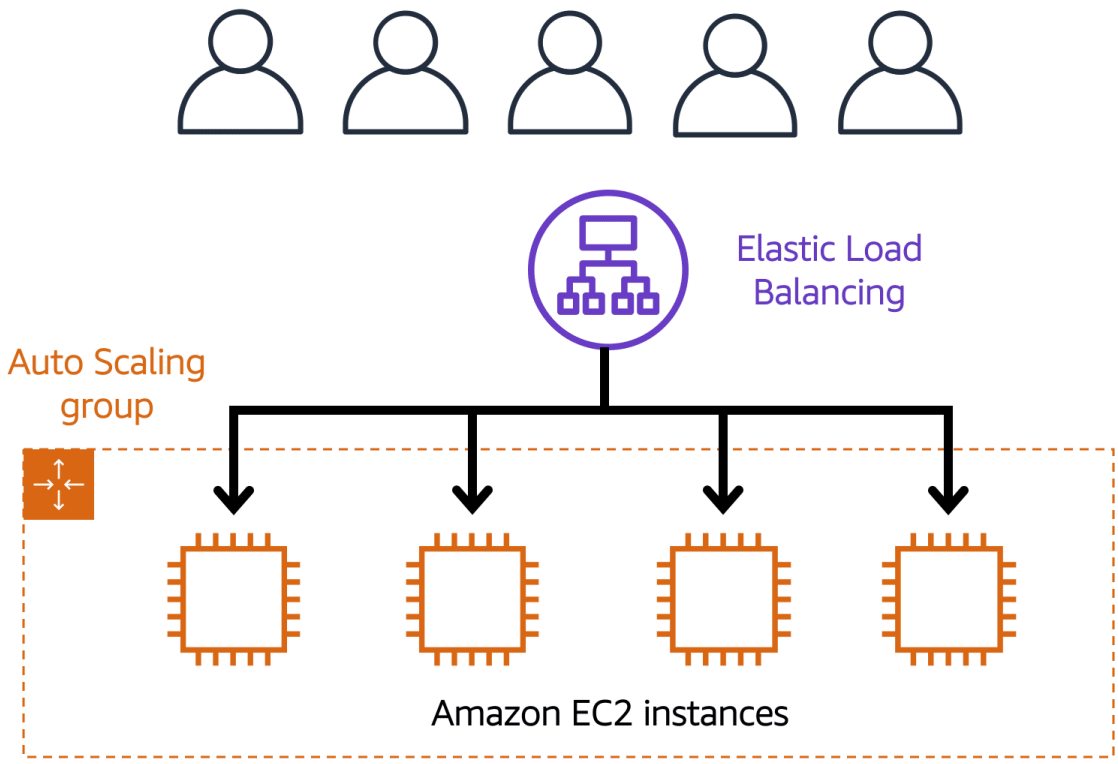
Example: Elastic Load Balancing



Low-demand period

Here's an example of how Elastic Load Balancing works. Suppose that a few customers have come to the coffee shop and are ready to place their orders.

If only a few registers are open, this matches the demand of customers who need service. The coffee shop is less likely to have open registers with no customers. In this example, you can think of the registers as Amazon EC2 instances.



High-demand period

Throughout the day, as the number of customers increases, the coffee shop opens more registers to accommodate them.

Additionally, a coffee shop employee directs customers to the most appropriate register so that the number of requests can evenly distribute across the open registers. You can think of this coffee shop employee as a load balancer.

Elastic Load Balancing

Elastic Load Balancing is the AWS service that automatically distributes incoming application traffic across multiple resources, such as Amazon EC2 instances.

A load balancer acts as a single point of contact for all incoming web traffic to your Auto Scaling group. This means that as you add or remove Amazon EC2 instances in response to the amount of incoming traffic, these requests route to the load balancer first. Then, the requests spread across multiple resources that will handle them. For example, if you have multiple Amazon EC2 instances, Elastic Load Balancing distributes the workload across the multiple instances so that no single instance has to carry the bulk of it.

Although Elastic Load Balancing and Amazon EC2 Auto Scaling are separate services, they work together to help ensure that applications running in Amazon EC2 can provide high performance and availability.

Example: Elastic Load Balancing

ELB pointing to two EC2 instances in an Auto Scaling group.

Low-demand period

Here’s an example of how Elastic Load Balancing works. Suppose that a few customers have come to the coffee shop and are ready to place their orders.

If only a few registers are open, this matches the demand of customers who need service. The coffee shop is less likely to have open registers with no customers. In this example, you can think of the registers as Amazon EC2 instances.

Elastic Load Balancing during a high-demand period.

High-demand period

Throughout the day, as the number of customers increases, the coffee shop opens more registers to accommodate them.

Additionally, a coffee shop employee directs customers to the most appropriate register so that the number of requests can evenly distribute across the open registers. You can think of this coffee shop employee as a load balancer.

Messaging and Queing

Let's talk about messaging and queuing. In the coffee shop, there are cashiers taking orders from the customers and baristas making the orders. Currently, the cashier takes the order, writes it down with a pen and paper, and delivers this order to the barista. The barista then takes the paper and makes the order. When the next order comes in, the process repeats. This works great as long as both the cashier and the barista are in sync. But what would happen if the cashier took the order and turned to pass it to the barista and the barista was out on break or busy with another order? Well, that cashier is stuck until the barista is ready to take the order. And at a certain point, the order will probably be dropped so the cashier can go serve the next customer.

You can see how this is a flawed process, because as soon as either the cashier or barista is out of sync, the process will degrade, causing slow downs in receiving orders and failures to complete orders at all. A much better process would be to introduce some sort of buffer or queue into the system. Instead of handing the order directly to the barista, the cashier would post the order to some sort of buffer, like an order board.

This idea of placing messages into a buffer is called messaging and queuing. Just as our cashier sends orders to the barista, applications send messages to each other to communicate. If applications communicate directly like our cashier and barista previously, this is called being tightly coupled.

A hallmark trait of a tightly coupled architecture is where if a single component fails or changes, it causes issues for other components or even the whole system. For example, if we have Application A and it is sending messages directly to Application B, if Application B has a failure and cannot accept those messages, Application A will begin to see errors as well. This is a tightly coupled architecture.

A more reliable architecture is loosely coupled. This is an architecture where if one component fails, it is isolated and therefore won't cause cascading failures throughout the whole system. If we coded the application to use a more loosely coupled architecture, it could look as follows.

Just like our cashier and barista, we introduced a buffer between the two. In this case, we introduced a message queue. Messages are sent into the queue by Application A and they are processed by Application B. If Application B fails, Application A doesn't experience any disruption. Messages being sent can still be sent to the queue and will remain there until they are eventually processed.

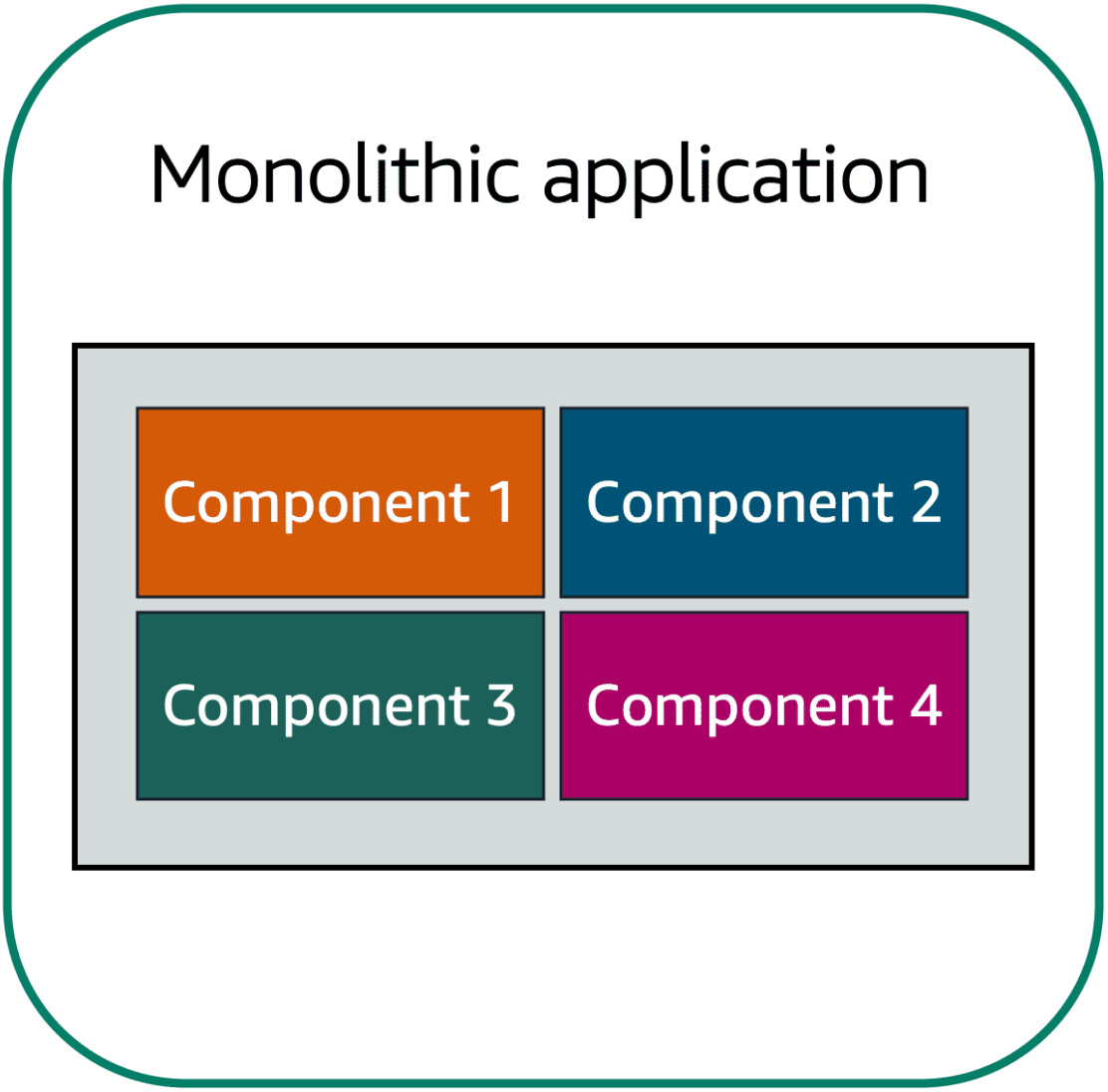
This is loosely coupled. This is what we strive to achieve with architectures on AWS. And this brings me to two AWS services that can assist in this regard. Amazon Simple Queue Service or SQS and Amazon Simple Notification Service or SNS. But before I dive into those two services, let me just order a to-go coffee on our cafe website. Done. All right, well, that's great. I should get a message when that order is ready.

First up, let's discuss Amazon SQS. SQS allows you to send, store, and receive messages between software components at any volume. This is without losing messages or requiring other services to be available. Think of messages as our coffee orders and the order board as an SQS queue. Messages have the person's name, coffee order, and time they ordered. The data contained within a message is called a payload, and it's protected until delivery. SQS queues are where messages are placed until they are processed. And AWS manages the underlying infrastructure for you to host those queues. These scale automatically, are reliable, and are easy to configure and use.

Now, Amazon SNS is similar in that it is used to send out messages to services, but it can also send out notifications to end users. It does this in a different way called a publish/subscribe or pub/sub model. This means that you can create something called an SNS topic which is just a channel for messages to be delivered. You then configure subscribers to that topic and finally publish messages for those subscribers. In practice, that means you can send one message to a topic which will then fan out to all the subscribers in a single go. These subscribers can also be endpoints such as SQS queues, AWS Lambda functions, and HTTPS or HTTP web hooks.

Additionally, SNS can be used to fan out notifications to end users using mobile push, SMS, and email. Taking this back to our coffee shop, we could send out a notification when a customer's order is ready. This could be a simple SMS to let them know to pick it up or even a mobile push.
In fact, it looks like my phone just received a message. Looks like my order is ready. See you soon.

Monolithic applications and microservices

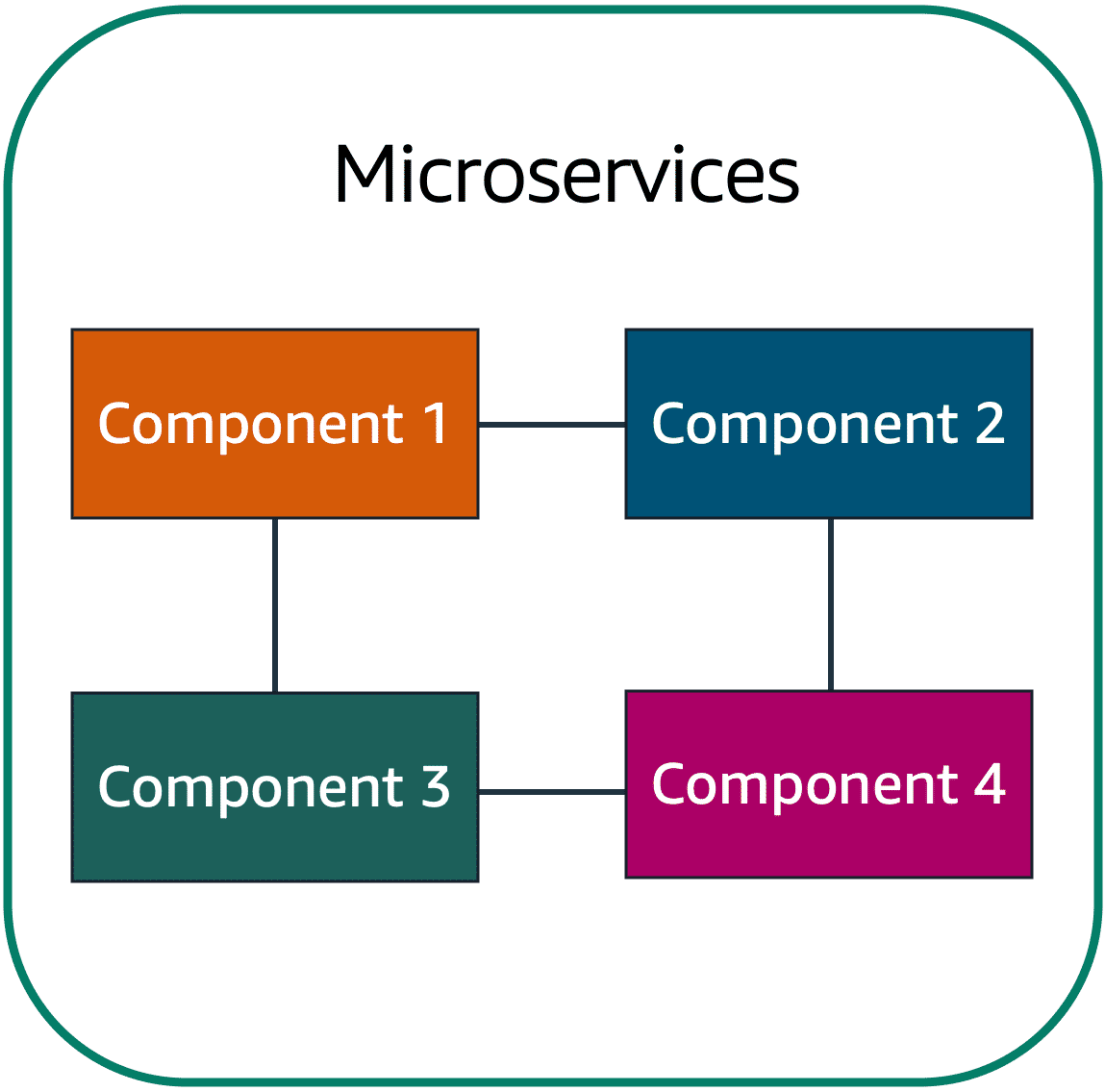


Applications are made of multiple components. The components communicate with each other to transmit data, fulfill requests, and keep the application running.

Suppose that you have an application with tightly coupled components. These components might include databases, servers, the user interface, business logic, and so on. This type of architecture can be considered a **monolithic application**.

In this approach to application architecture, if a single component fails, other components fail, and possibly the entire application fails.

To help maintain application availability when a single component fails, you can design your application through a **microservices** approach.



In a microservices approach, application components are loosely coupled. In this case, if a single component fails, the other components continue to work because they are communicating with each other. The loose coupling prevents the entire application from failing.

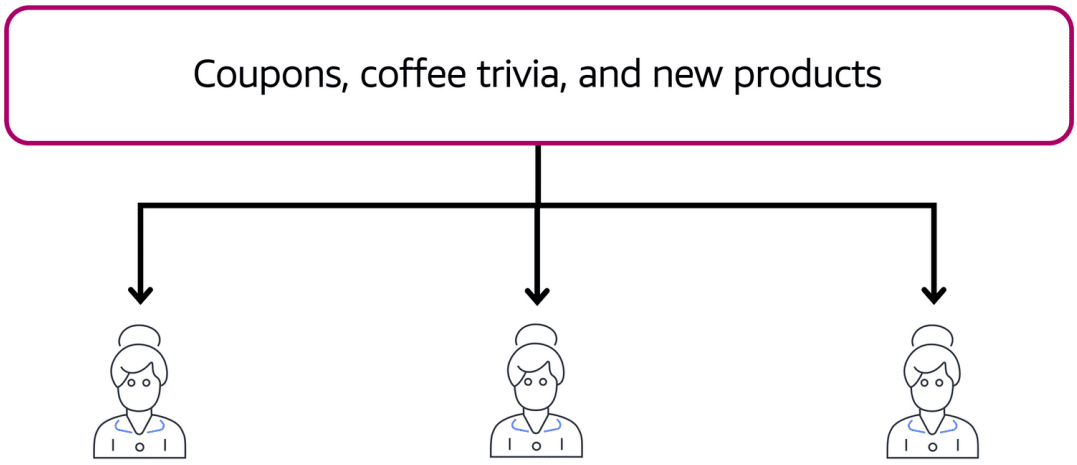
When designing applications on AWS, you can take a microservices approach with services and components that fulfill different functions. Two services facilitate application integration: Amazon Simple Notification Service (Amazon SNS) and Amazon Simple Queue Service (Amazon SQS).

Amazon Simple Notification Service (Amazon SNS)

Amazon Simple Notification Service (Amazon SNS) is a publish/subscribe service. Using Amazon SNS topics, a publisher publishes messages to subscribers. This is similar to the coffee shop; the cashier provides coffee orders to the barista who makes the drinks.

In Amazon SNS, subscribers can be web servers, email addresses, AWS Lambda functions, or several other options.

Publishing updates from a single topic

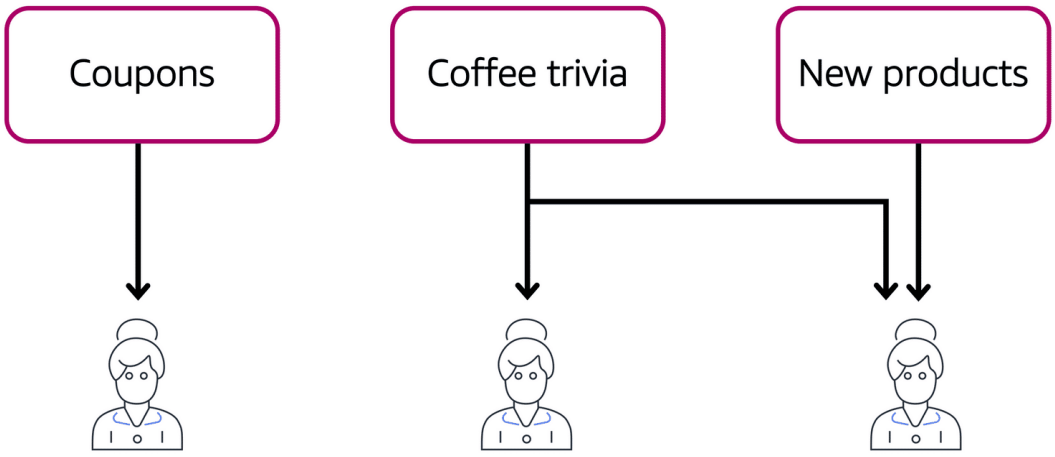


Suppose that the coffee shop has a single newsletter that includes updates from all areas of its business. It includes topics such as coupons, coffee trivia, and new products. All of these topics are grouped because this is a single newsletter. All customers who subscribe to the newsletter receive updates about coupons, coffee trivia, and new products.

After a while, some customers express that they would prefer to receive separate newsletters for only the specific topics that interest them. The coffee shop owners decide to try this approach.

Step 2

Publishing updates from multiple topics



Now, instead of having a single newsletter for all topics, the coffee shop has broken it up into three separate newsletters. Each newsletter is devoted to a specific topic: coupons, coffee trivia, and new products.

Subscribers will now receive updates immediately for only the specific topics to which they have subscribed.

It is possible for subscribers to subscribe to a single topic or to multiple topics. For example, the first customer subscribes to only the coupons topic, and the second subscriber subscribes to only the coffee trivia topic. The third customer subscribes to both the coffee trivia and new products topics.

Summary

Although these examples from the coffee shop involve subscribers who are people, in Amazon SNS, subscribers can be web servers, email addresses, AWS Lambda functions, or several other options.

START AGAIN

Amazon Simple Queue Service (Amazon SQS)

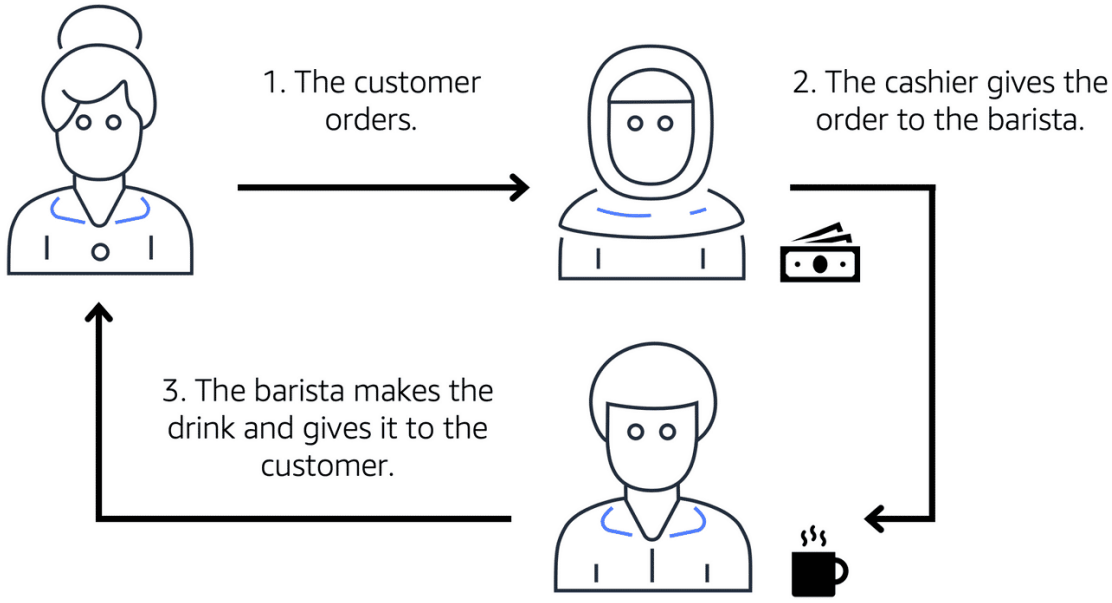
Amazon Simple Queue Service (Amazon SQS) is a message queuing service.

Using Amazon SQS, you can send, store, and receive messages between software components, without losing messages or requiring other services to be available. In Amazon SQS, an application sends messages into a queue. A user or service retrieves a message from the queue, processes it, and then deletes it from the queue.

To review two examples of how to use Amazon SQS, choose the arrow buttons to display each one.

Step 1

Example 1: Fulfilling an order



Suppose that the coffee shop has an ordering process in which a cashier takes orders, and a barista makes the orders. Think of the cashier and the barista as two separate components of an application.

First, the cashier takes an order and writes it down on a piece of paper. Next, the cashier delivers the paper to the barista. Finally, the barista makes the drink and gives it to the customer.

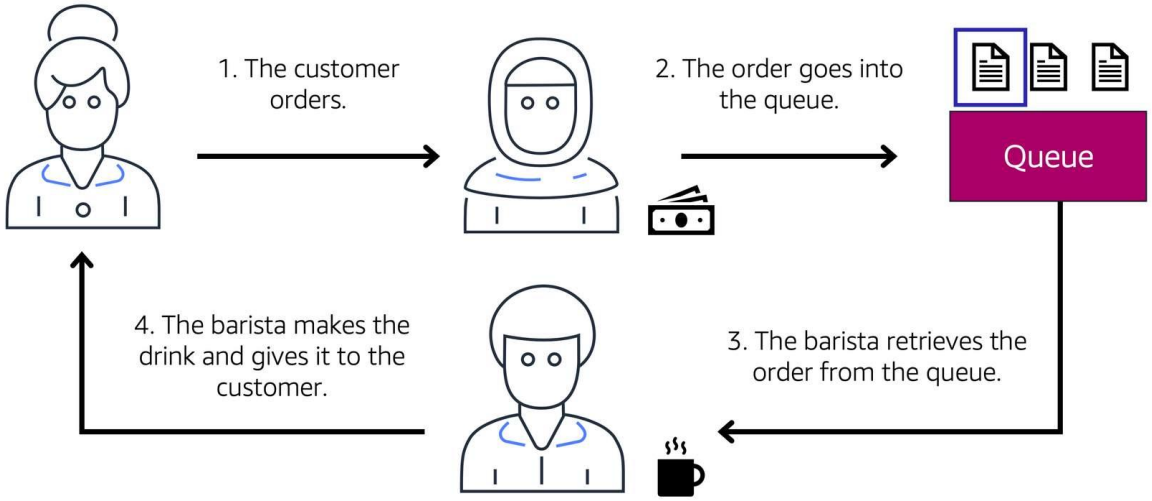
When the next order comes in, the process repeats. This process runs smoothly as long as both the cashier and the barista are coordinated.

What might happen if the cashier took an order and went to deliver it to the barista, but the barista was out on a break or busy with another order? The cashier would need to wait until the barista is ready to accept the order. This would cause delays in the ordering process and require customers to wait longer to receive their orders.

As the coffee shop has become more popular and the ordering line is moving more slowly, the owners notice that the current ordering process is time consuming and inefficient. They decide to try a different approach that uses a queue.

Step 2

Example 2: Orders in a queue



Recall that the cashier and the barista are two separate components of an application. A message queuing service, such as Amazon SQS, lets messages between decoupled application complements.

In this example, the first step in the process remains the same as before: a customer places an order with the cashier.

The cashier puts the order into a queue. You can think of this as an order board that serves as a buffer between the cashier and the barista. Even if the barista is out on a break or busy with another order, the cashier can continue placing new orders into the queue.

Next, the barista checks the queue and retrieves the order.

The barista prepares the drink and gives it to the customer.

The barista then removes the completed order from the queue.

While the barista is preparing the drink, the cashier is able to continue taking new orders and add them to the queue.

Additional Compute Services

EC2 instances are virtual machines that you can provision with minimal friction to get up and running on AWS. EC2 is great for all sorts of different use cases like running basic web servers to running high performance computing clusters and everything in between.

That being said, though EC2 is incredibly flexible, reliable, and scalable, depending on your use case, you might be looking at alternatives for your compute capacity. EC2 requires that you set up and manage your fleet of instances over time. When you're using EC2, you are responsible for patching your instances when new software packages come out, setting up the scaling of those instances as well as ensuring that you've architected your solutions to be hosted in a highly available manner. This is still not as much management as you would have if you hosted these on-premises. But management processes will still need to be in place.

You might be wondering what other services does AWS offer for compute that are more convenient from a management perspective. This is where the term serverless comes in. AWS offers multiple

serverless compute options. Serverless means that you cannot actually see or access the underlying infrastructure or instances that are hosting your application. Instead, all the management of the underlying environment from a provisioning, scaling, high availability, and maintenance perspective are taken care of for you. All you need to do is focus on your application and the rest is taken care of.

AWS Lambda is one serverless compute option. Lambda's a service that allows you to upload your code into what's called a Lambda function. Configure a trigger and from there, the service waits for the trigger. When the trigger is detected, the code is automatically run in a managed environment, an environment you do not need to worry too much about because it is automatically scalable, highly available and all of the maintenance in the environment itself is done by AWS. If you have one or 1,000 incoming triggers, Lambda will scale your function to meet demand. Lambda is designed to run code under 15 minutes so this isn't for long running processes like deep learning. It's more suited for quick processing like a web backend, handling requests or a backend expense report processing service where each invocation takes less than 15 minutes to complete.

If you weren't quite ready for serverless yet or you need access to the underlying environment, but still want efficiency and portability, you should look at AWS container services like Amazon Elastic Container Service, otherwise known as ECS. Or Amazon Elastic Kubernetes Service, otherwise known as EKS.

Both of these services are container orchestration tools, but before I get too far here, a container in this case is a Docker container. Docker is a widely used platform that uses operating system level virtualization to deliver software in containers. Now a container is a package for your code where you package up your application, its dependencies as well as any configurations that it needs to run. These containers run on top of EC2 instances and run in isolation from each other similar to how virtual machines work. But in this case, the host is an EC2 instance. When you use Docker containers on AWS, you need processes to start, stop, restart, and monitor containers running across not just one EC2 instance, but a number of them together which is called a cluster.

The process of doing these tasks is called container orchestration and it turns out it's really hard to do on your own. Orchestration tools were created to help you manage your containers. ECS is designed to help you run your containerized applications at scale without the hassle of managing your own container orchestration software. EKS does a similar thing, but uses different tooling and with different features.

Both Amazon ECS and Amazon EKS can run on top of EC2. But if you don't want to even think about using EC2s to host your containers because you either don't need access to the underlying OS or you don't want to manage those EC2 instances, you can use a compute platform called AWS Fargate. Fargate is a serverless compute platform for ECS or EKS. That's a bit high level and it might be confusing, so let's clear that up.

If you are trying to host traditional applications and want full access to the underlying operating system like Linux or Windows, you are going to want to use EC2. If you are looking to host short running functions, service-oriented or event driven applications and you don't want to manage the underlying environment at all, look into the serverless AWS Lambda. If you are looking to run Docker container-based workloads on AWS, you first need to choose your orchestration tool. Do you want to use Amazon ECS or Amazon EKS? After you choose your tool, you then need to chose your platform. Do you want to run your containers on EC2 instances that you manage or in a serverless environment like AWS Fargate that is managed for you?

Those are just some of your compute options with AWS and it's not even a complete list. Check out the notes for more information on AWS compute services as well as others that we didn't get to talk about in this video.

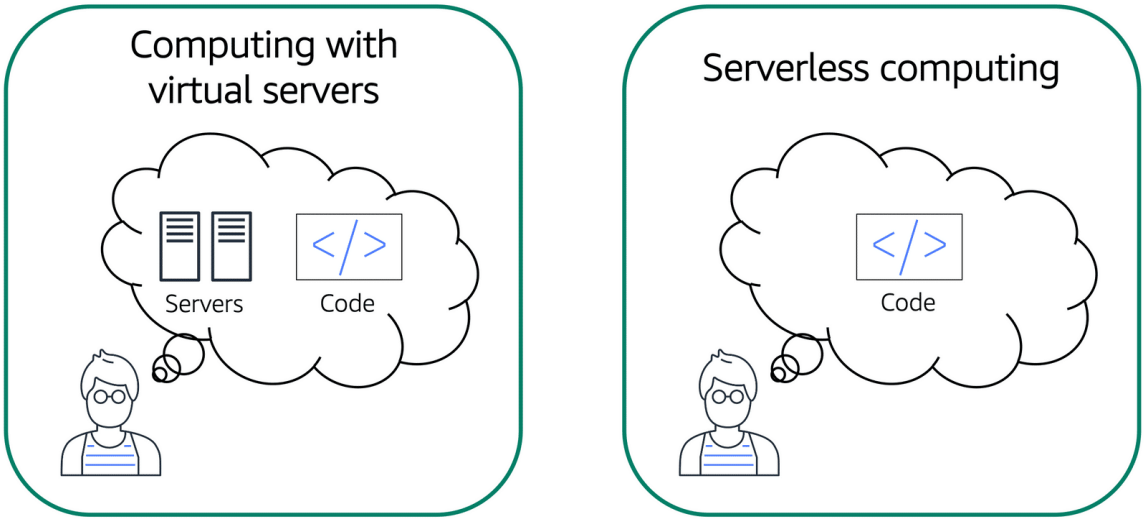
Serverless computing

Earlier in this module, you learned about Amazon EC2, a service that lets you run virtual servers in the cloud. If you have applications that you want to run in Amazon EC2, you must do the following:

Provision instances (virtual servers).

Upload your code.

Continue to manage the instances while your application is running.



Comparison between computing with virtual servers (thinking about servers and code) and serverless computing (thinking only about code).

The term “serverless” means that your code runs on servers, but you do not need to provision or manage these servers. With serverless computing, you can focus more on innovating new products and features instead of maintaining servers.

Another benefit of serverless computing is the flexibility to scale serverless applications automatically. Serverless computing can adjust the applications' capacity by modifying the units of consumptions, such as throughput and memory.

An AWS service for serverless computing is **AWS Lambda**.

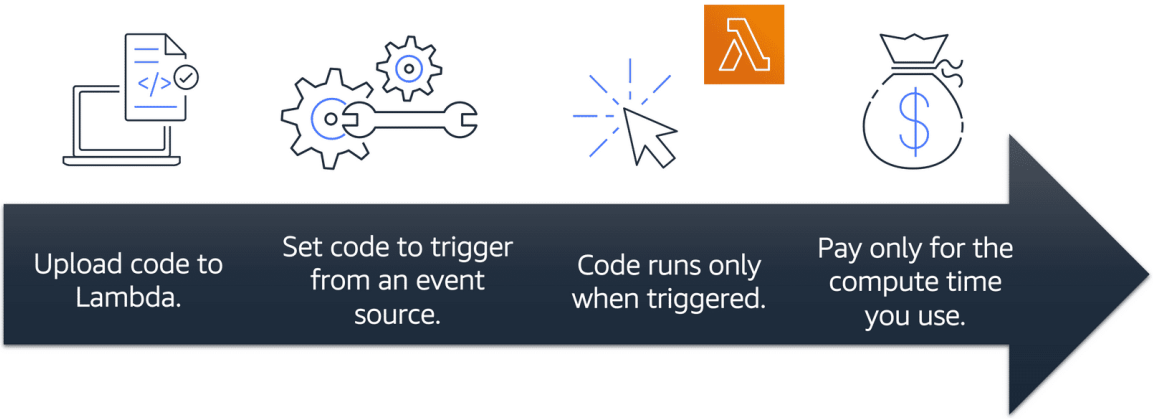
AWS Lambda

AWS Lambda(opens in a new tab) is a service that lets you run code without needing to provision or manage servers.

While using AWS Lambda, you pay only for the compute time that you consume. Charges apply only when your code is running. You can also run code for virtually any type of application or backend service, all with zero administration.

For example, a simple Lambda function might involve automatically resizing uploaded images to the AWS Cloud. In this case, the function triggers when uploading a new image.

How AWS Lambda works



You upload your code to Lambda.

You set your code to trigger from an event source, such as AWS services, mobile applications, or HTTP endpoints.

Lambda runs your code only when triggered.

You pay only for the compute time that you use. In the previous example of resizing images, you would pay only for the compute time that you use when uploading new images. Uploading the images triggers Lambda to run code for the image resizing function.

In AWS, you can also build and run **containerized** applications.

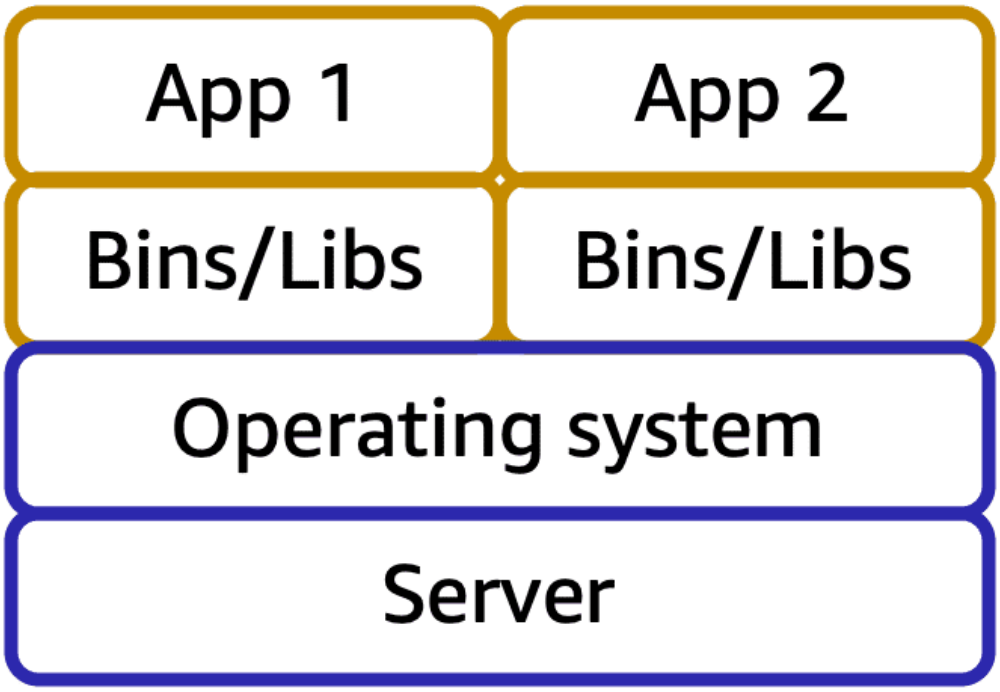
Containers

Containers provide you with a standard way to package your application's code and dependencies into a single object. You can also use containers for processes and workflows in which there are essential requirements for security, reliability, and scalability.

To review an example on how containers work, choose the arrow buttons.

Step 1

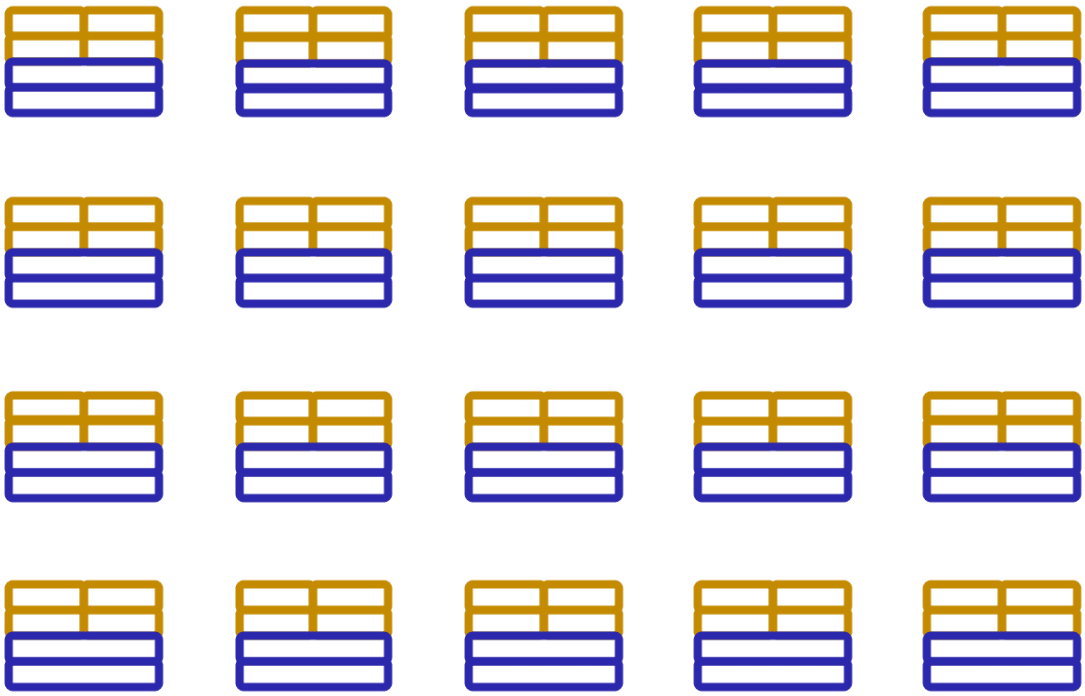
One host with multiple containers



Suppose that a company's application developer has an environment on their computer that is different from the environment on the computers used by the IT operations staff. The developer wants to ensure that the application's environment remains consistent regardless of deployment, so they use a containerized approach. This helps to reduce time spent debugging applications and diagnosing differences in computing environments.

Step 2

Tens of hosts with hundreds of containers



When running containerized applications, it's important to consider scalability. Suppose that instead of a single host with multiple containers, you have to manage tens of hosts with hundreds of containers. Alternatively, you have to manage possibly hundreds of hosts with thousands of containers. At a large scale, imagine how much time it might take for you to monitor memory usage, security, logging, and so on.

Summary

Container orchestration services help you to deploy, manage, and scale your containerized applications. Next, you will learn about two services that provide container orchestration: Amazon Elastic Container Service and Amazon Elastic Kubernetes Service.

START AGAIN

Amazon Elastic Container Service (Amazon ECS)

Amazon Elastic Container Service (Amazon ECS)(opens in a new tab) is a highly scalable, high-performance container management system that enables you to run and scale containerized applications on AWS.

Amazon ECS supports Docker containers. Docker(opens in a new tab) is a software platform that enables you to build, test, and deploy applications quickly. AWS supports the use of open-source Docker Community Edition and subscription-based Docker Enterprise Edition. With Amazon ECS, you can use API calls to launch and stop Docker-enabled applications.

Amazon Elastic Kubernetes Service (Amazon EKS)

Amazon Elastic Kubernetes Service (Amazon EKS)(opens in a new tab) is a fully managed service that you can use to run Kubernetes on AWS.

Kubernetes(opens in a new tab) is open-source software that enables you to deploy and manage containerized applications at scale. A large community of volunteers maintains Kubernetes, and AWS

actively works together with the Kubernetes community. As new features and functionalities release for Kubernetes applications, you can easily apply these updates to your applications managed by Amazon EKS.

AWS Fargate

AWS Fargate(opens in a new tab) is a serverless compute engine for containers. It works with both Amazon ECS and Amazon EKS.

When using AWS Fargate, you do not need to provision or manage servers. AWS Fargate manages your server infrastructure for you. You can focus more on innovating and developing your applications, and you pay only for the resources that are required to run your containers.

Hey everyone, I hope you've been enjoying the course so far. We're going to do check-ins like this one after each major topic to review what you've learned.

First things first, what is cloud computing and what does AWS offer? We define cloud computing as the on-demand delivery of IT resources over the internet with pay-as-you-go pricing. This means that you can make requests for IT resources like compute, networking, storage, analytics, or other types of resources, and then they're made available for you on demand. You don't pay for the resource upfront. Instead, you just provision and pay at the end of the month.

AWS offers services and many categories that you use together to create your solutions. We've only covered a few services so far, you learned about Amazon EC2. With EC2, you can dynamically spin up and spin down virtual servers called EC2 instances. When you launch an EC2 instance, you choose the instance family. The instance family determines the hardware the instance runs on.

And you can have instances that are built for your specific purpose. The categories are general purpose, compute optimized, memory optimized, accelerated computing, and storage optimized.

You can scale your EC2 instances either vertically by resizing the instance, or horizontally by launching new instances and adding them to the pool. You can set up automated horizontal scaling, using Amazon EC2 Auto Scaling.

Once you've scaled your EC2 instances out horizontally, you need something to distribute the incoming traffic across those instances. This is where the Elastic Load Balancer comes into play.

EC2 instances have different pricing models. There is On-Demand, which is the most flexible and has no contract, spot pricing, which allows you to utilize unused capacity at a discounted rate, Savings Plans or Reserved Instances, which allow you to enter into a contract with AWS to get a discounted rate when you commit to a certain level of usage, and Savings Plans which apply to AWS Lambda, and AWS Fargate, as well as EC2 instances.

We also covered messaging services. There is Amazon Simple Queue Service or SQS. This service allows you to decouple system components. Messages remain in the queue until they are either consumed or deleted. Amazon Simple Notification Service or SNS, is used for sending messages like emails, text messages, push notifications, or even HTTP requests. Once a message is published, it is sent to all of these subscribers.

You also learned that AWS has different types of compute services beyond just virtual servers like EC2. There are container services like Amazon Elastic Container Service, or ECS. And there's Amazon Elastic Kubernetes Service, or EKS. Both of which are container orchestration tools. You can use these tools with EC2 instances, but if you don't want to manage that, you don't need to.

You can use AWS Fargate, which allows you to run your containers on top of a serverless compute platform. Then there is AWS Lambda, which allows you to just upload your code, and configure it to run based on triggers. And you only get charged for when the code is actually running. No containers, no virtual machines. Just code and configuration.

Hopefully that sums it up. Catch you in the next one.

I want to talk to you about high availability. Say you wanna grab a cup of coffee at the cafe, but today is not just a normal day in our city. Today, there is a parade coming to march down our street in celebration of all of the wonderful cloud migrations that have been successful. This parade is going to march right in front of our shop. This is great because who doesn't love to look at floats and balloons and have someone throw candy at them from the street? But this is also bad because while the parade is coming down the street, our customers who are driving to come get coffee will be diverted and won't be able to stop by the shop. This will drive sales down and also make customers unhappy.

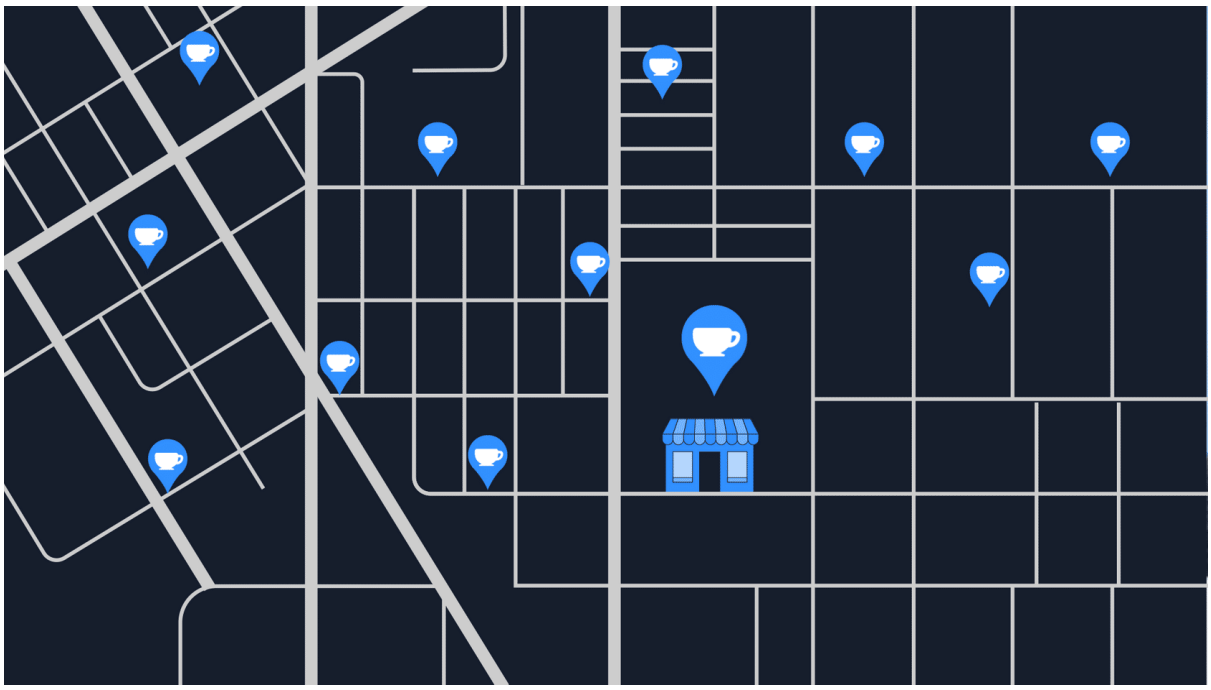
Luckily for us, we thought ahead. We thought long ago what would happen if for some reason our customers couldn't make it into the shop temporarily? Like if there was a parade or if there's a flood or some other event that blocks us from getting into the shop? Well, no matter the reason, we need to be highly available for our customers.

So, I'll let you in on a secret. This isn't our coffee shop's only location. The cafe is actually a chain and we have locations all around the city. That way, if a parade comes down the street, or if there was a power outage on our block, no worries. Customers can still get their coffee by visiting one of our shops just a few blocks away. We still make money and they get their coffee. All is well.

AWS has done a similar thing with how the AWS global infrastructure is set up. It's not good enough to have one giant data center where all of the resources are. If something were to happen to that data center, like a power outage or a natural disaster, everyone's applications would go down all at once. You need high availability and fault tolerance.

Turns out, it's not even good enough to have two giant data centers. Instead, AWS operates in all sorts of different areas around the world called Regions. We're going to talk about this in depth in upcoming videos. In the meantime, I'm gonna sit here and relax, because I know my business is highly available regardless of any parades blocking the street.

Building a global footprint



To understand the AWS global infrastructure, consider the coffee shop. If an event such as a parade, flood, or power outage impacts one location, customers can still get their coffee by visiting a different location only a few blocks away.

This is similar to how the AWS global infrastructure works.

AWS Global Infrastructure

To understand the global infrastructure AWS, I wanna begin with your fundamental business need. You have an application that you have to run, or content you need stored, or data you need analyzed. Basically you have stuff that has to live and operate somewhere. Now historically, businesses had to run applications in their own data centers, 'cause they didn't have a choice. Once AWS became available, companies like yours could now run their applications in other data centers they didn't actually own.

But the conversation is so much more than that, 'cause I want you to understand a fundamental problem with any data center, doesn't matter who built it or who owns it. Events can happen that could cause you to lose connection to that building. If you run your own data center, you have to figure out how to answer the question of what will you do when disaster strikes your building. You could run a second data center, but real estate prices alone could stop you, much less all the duplicate expense of hardware, employees, electricity, heating and cooling, security. Most businesses simply end up just storing backups somewhere, and then hope for the disaster to never come. And hope is not a good business plan.

AWS answers the question of what happens when disaster strikes by building our data centers in large groups we call Regions, and here's how it's designed.

Throughout the globe, AWS builds Regions to be closest to where the business traffic demands. Locations like Paris, Tokyo, Sao Paulo, Dublin, Ohio. Inside each Region, we have multiple data centers that have all the compute, storage, and other services you need to run your applications. Each Region can be connected to each other Region through a high speed fiber network, controlled by AWS, a truly global operation from corner to corner if you need it to be. Now before we get into the architecture of how each Region is built, it's important to know that you, the business decision maker, gets to choose which Region you want to run out of. And each Region is isolated from every other Region in the sense that absolutely no data goes in or out of your environment in that Region without you explicitly granting permission for that data to be moved. This is a critical security conversation to have.

For example, you might have government compliance requirements that your financial information in Frankfurt cannot leave Germany. Well this is absolutely the way AWS operates outta the box. Any data stored in the Frankfurt Region never leaves the Frankfurt Region, or data in the London region never leaves London, or Sydney never leaves Sydney, unless you explicitly, with the right credentials and permissions, request the data be exported.

Regional data sovereignty is part of the critical design of AWS Regions. With data being subject to the local laws and statutes of the country where the Region lives. So with that understanding, that your data, your application, lives and runs in a Region, one of the first decisions you get to make is which Region do you pick? There's four business factors that go into choosing a Region.

Number one, compliance. Before any of the other factors, you must first look at your compliance requirements. Do you have a requirement your data must live in UK boundaries? Then you should choose the London Region, full stop. None of the rest of the options matter. Or let's say you must run inside Chinese borders. Well then, you should choose on of our Chinese Regions. Most businesses, though, are not governed by such strict regulations. So if you don't have a compliance or regulatory control that dictates your Region, then you can look at the other factors.

Number two, proximity. How close you are to your customer base is a major factor because speed of light, still the law of the universe. If most of your customers live in Singapore, consider running out of the Singapore Region. Now you certainly can run out of Virginia, but the time it takes for the information to be sent, or latency, between the US and Singapore is always going to be a factor. Now we may be developing quantum computing, but quantum networking, still some ways away. The time it takes light to travel around the world is always a consideration. Locating close to your customer base, usually the right call.

Number three, feature availability. Sometimes the closest Region may not have all the AWS features you want. Now here's one of the cool things about AWS. We are constantly innovating on behalf of our customers. Every year, AWS releases thousands of new features and products specifically to answer customer requests and needs. But sometimes those brand new services take a lot of new physical hardware that AWS has to build out to make the service operational. And sometimes that means we have to build the service out one Region at a time. So let's say your developers wanted to play with Amazon Braket, our new quantum computing platform. Well then, they have to run in the Regions that already have the hardware installed. Eventually, can we expect it to be in every Region? Well, yeah, that's a good expectation, but if you wanna use it today, then that might be your deciding factor.

Number four, pricing. Even when the hardware is equal one Region to the next, some locations are just more expensive to operate in, for example, Brazil. Now Brazil's tax structure is such that it costs AWS significantly more to operate the exact same services there than in many other countries. The exact same workload in Sao Paulo might be, let's say, 50% more expensive to run than out of Oregon in the United States. Price can be determined by many factors, so AWS has very transparent granular pricing that we'll continue to discuss in this class. But know that each Region has a different price sheet. So if budget is your primary concern, even if your customers live in Brazil, you might still wanna operate in a different country, again, if price is your primary motivation.

So four key factors to choose a Region: Compliance, proximity, feature availability, and pricing. When we come back, we'll look at the moving parts inside a Region.

Selecting a Region

When determining the right Region for your services, data, and applications, consider the following four business factors.

To learn more Regions, expand each of the following four categories.

Compliance with data governance and legal requirements

Depending on your company and location, you might need to run your data out of specific areas. For example, if your company requires all of its data to reside within the boundaries of the UK, you would choose the London Region.

Not all companies have location-specific data regulations, so you might need to focus more on the other three factors.

Proximity to your customers

Selecting a Region that is close to your customers will help you to get content to them faster. For example, your company is based in Washington, DC, and many of your customers live in Singapore. You might consider running your infrastructure in the Northern Virginia Region to be close to company headquarters, and run your applications from the Singapore Region.

Available services within a Region

Sometimes, the closest Region might not have all the features that you want to offer to customers. AWS is frequently innovating by creating new services and expanding on features within existing services. However, making new services available around the world sometimes requires AWS to build out physical hardware one Region at a time.

Suppose that your developers want to build an application that uses Amazon Braket (AWS quantum computing platform). As of this course, Amazon Braket is not yet available in every AWS Region around the world, so your developers would have to run it in one of the Regions that already offers it.

Pricing

Suppose that you are considering running applications in both the United States and Brazil. The way Brazil's tax structure is set up, it might cost 50% more to run the same workload out of the São Paulo Region compared to the Oregon Region. You will learn in more detail that several factors determine pricing, but for now know that the cost of services can vary from Region to Region.

if a Region is where all of the pieces and parts of your application live, some of you might be thinking that we never actually solved the problem that we presented in the

last video. Let me restate the problem. You don't want to run your application in a single building because a single building can fail for any number of unavoidable reasons.

You may be thinking, if my business needs to be disaster proof, then it can't run in just one location. Well, you're absolutely correct. AWS agrees with that statement and that's why our Regions are not one location. To begin with, AWS has data centers, lots of data centers, all around the world, and each Region is made up of multiple data centers.

AWS calls a single data center or a group of data centers, an Availability Zone or AZ. Each Availability Zone is one or more discrete data centers with redundant power, networking, and connectivity. When you launch an Amazon EC2 instance, it launches a virtual machine on a physical hardware that is installed in an Availability Zone. This means each AWS Region consists of multiple isolated and physically separate Availability Zones within a geographic Region.

But we don't build Availability Zones right next to each other because if a large scale incident were to occur, like a natural disaster, for example, you could lose connectivity to everything in that Availability Zone. The question what happens in case of a disaster matters and if you are familiar with disaster recovery planning, you might even have an idea of where we are going with this.

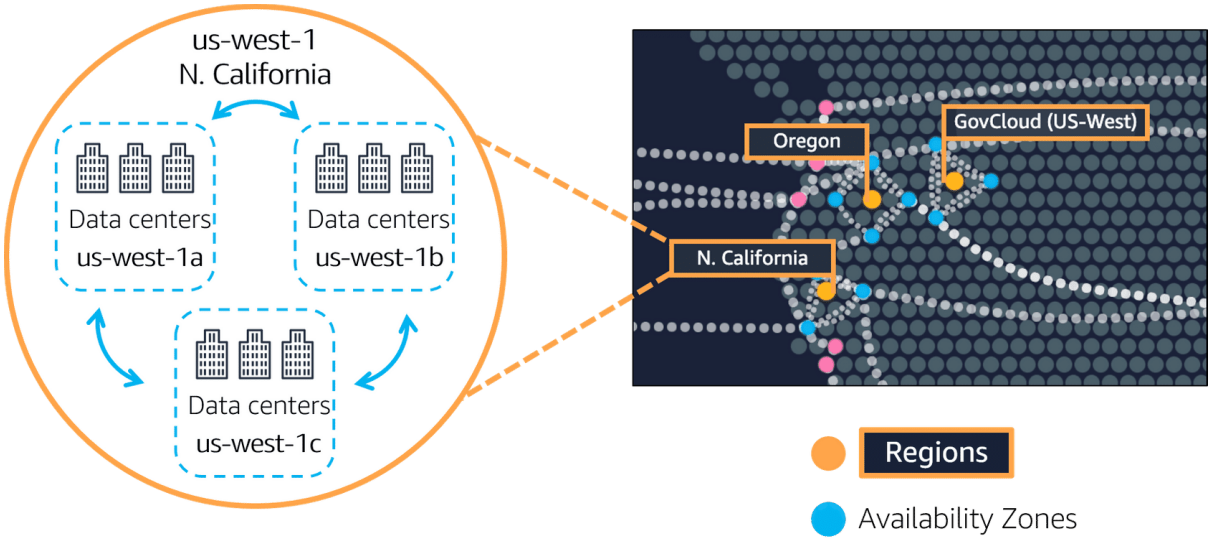
If you only run one EC2 instance, it only runs in one building, or one Availability Zone and a large scale disaster occurs, will your application still be able to run and serve your business? The obvious solution to this is to run multiple EC2 instances, just like we showed in the scaling example earlier. But the main thing is don't run them in the same building. Don't even run them in the same street, push them as far apart as you can before the speed of light tells you to stop if you still want low latency communication. Turns out that the speed of light will let us move these Availability Zones tens of miles apart from each other and still keep single digit millisecond latency between these Availability Zones. Now, if a disaster strikes, your application continues just fine because this disaster only knocked over some of your capacity, not all.

And as we saw in the last section, you can rapidly spin up more capacity in the remaining Availability Zones, thus allowing your business to continue operating without interruption. And as a best practice with AWS, we always recommend you run across at least two Availability Zones in a Region. This means redundantly deploying your infrastructure in two different AZs.

But there's more to Regions than just places to run EC2. Many of the AWS services run at the Region level, meaning they run synchronously across multiple AZs without any additional effort on your part. Take the ELB we talked about previously. This is actually a regional construct. It runs across all Availability Zones, communicating with the EC2 instances that are running in a specific Availability Zone. Regional services are by definition already highly available at no additional cost of effort on your part.

So as you plan for high availability, any service that is listed as a regionally scoped service, you'll already have that box checked. When we come back, let's look at going outside the Regions for additional solutions.

Availability Zones



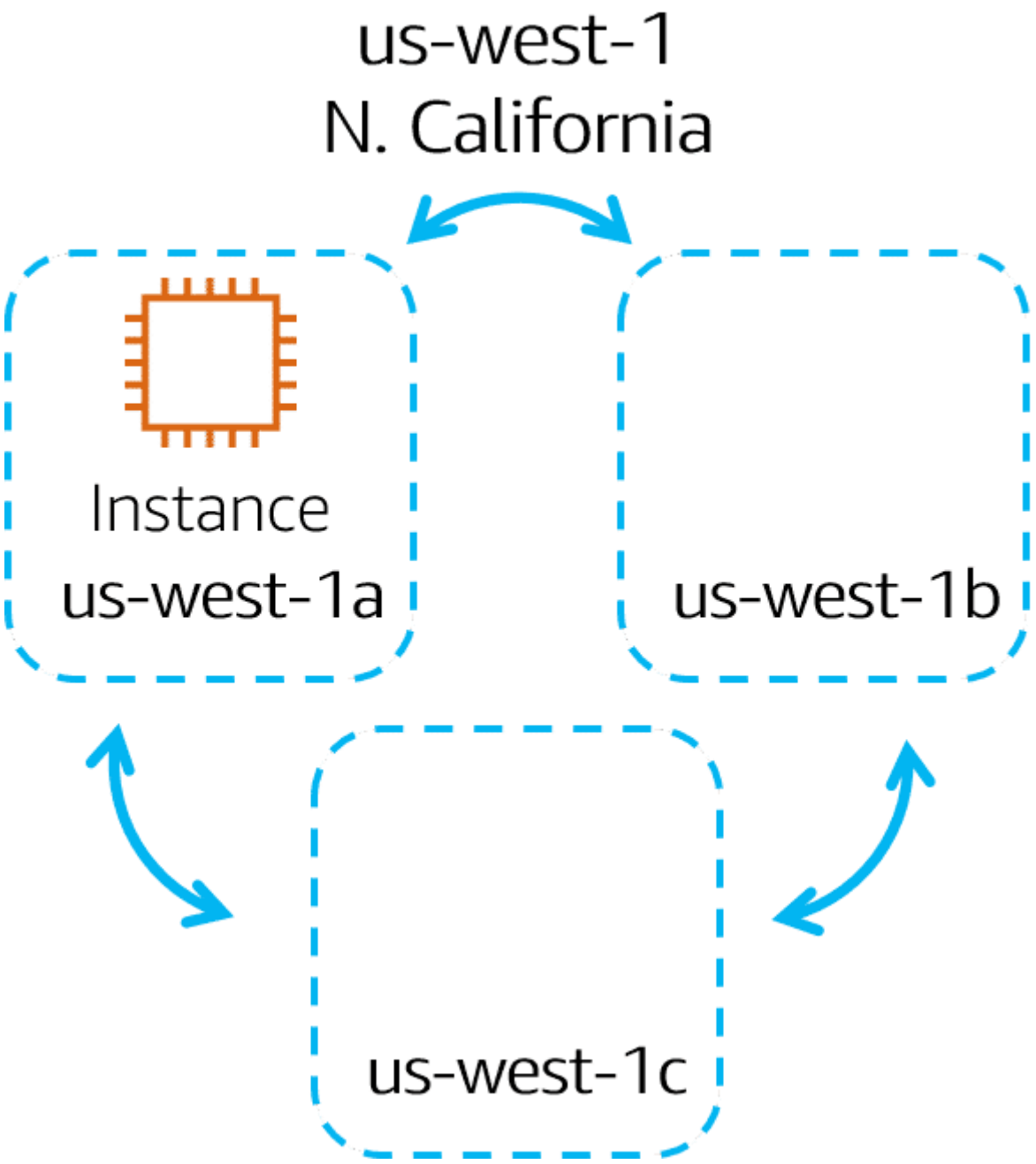
Spotlight on the us-west-1 Region. Northern California, Oregon, and GovCloud (US-West) are separate Regions. The Northern California Region is called us-west-1, and this Region contains three AZs (1a, 1b, and 1c). Then, within each AZ there are three data centers.

An **Availability Zone** is a single data center or a group of data centers within a Region. Availability Zones are located tens of miles apart from each other. This is close enough to have low latency (the time between when content requested and received) between Availability Zones. However, if a disaster occurs in one part of the Region, they are distant enough to reduce the chance that multiple Availability Zones are affected.

To review an example of running Amazon EC2 instances in multiple Availability Zones, choose the arrow buttons.

Step 1

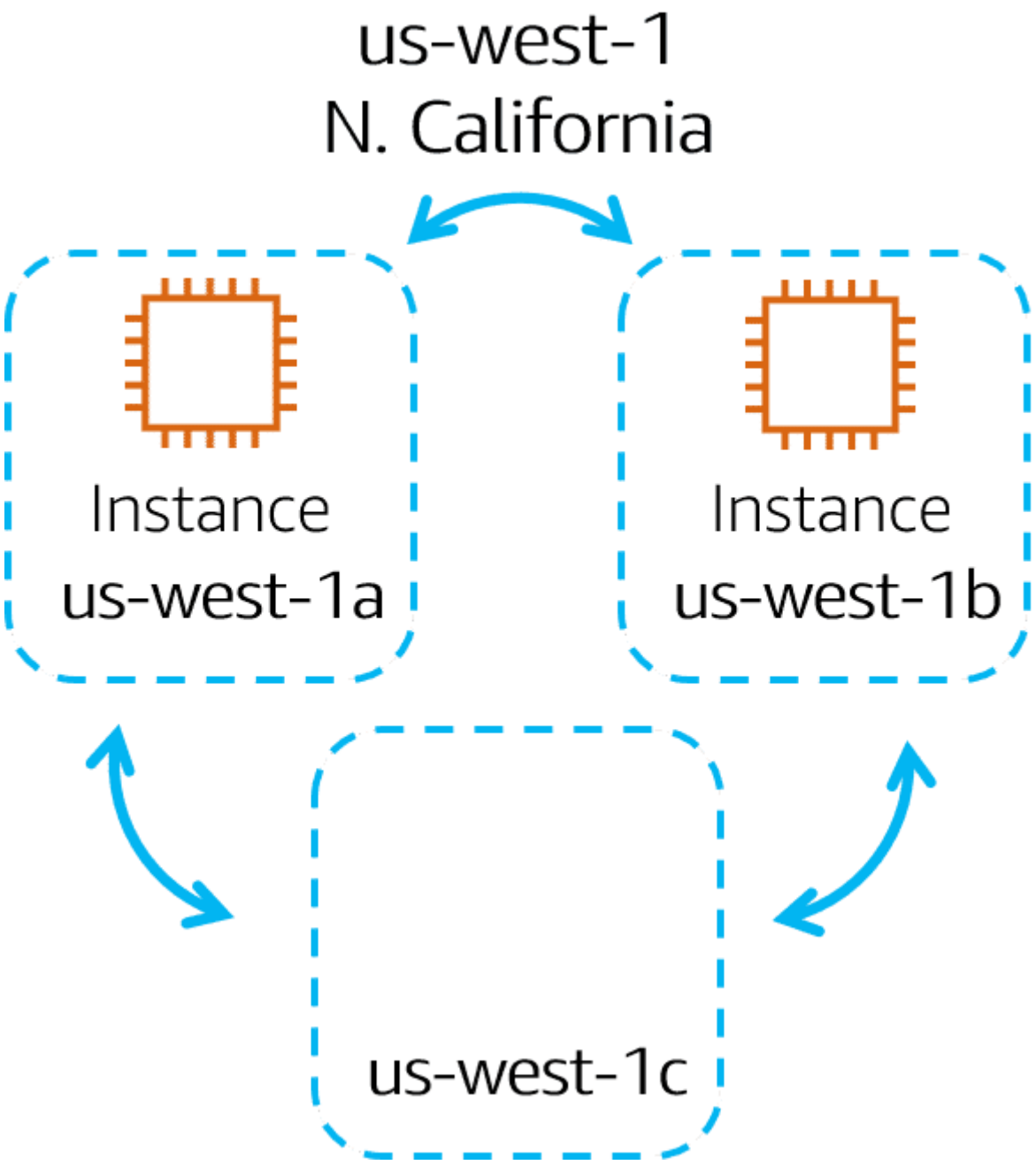
Amazon EC2 instance in a single Availability Zone



Suppose that you're running an application on a single Amazon EC2 instance in the Northern California Region. The instance is running in the us-west-1a Availability Zone. If us-west-1a were to fail, you would lose your instance.

Step 2

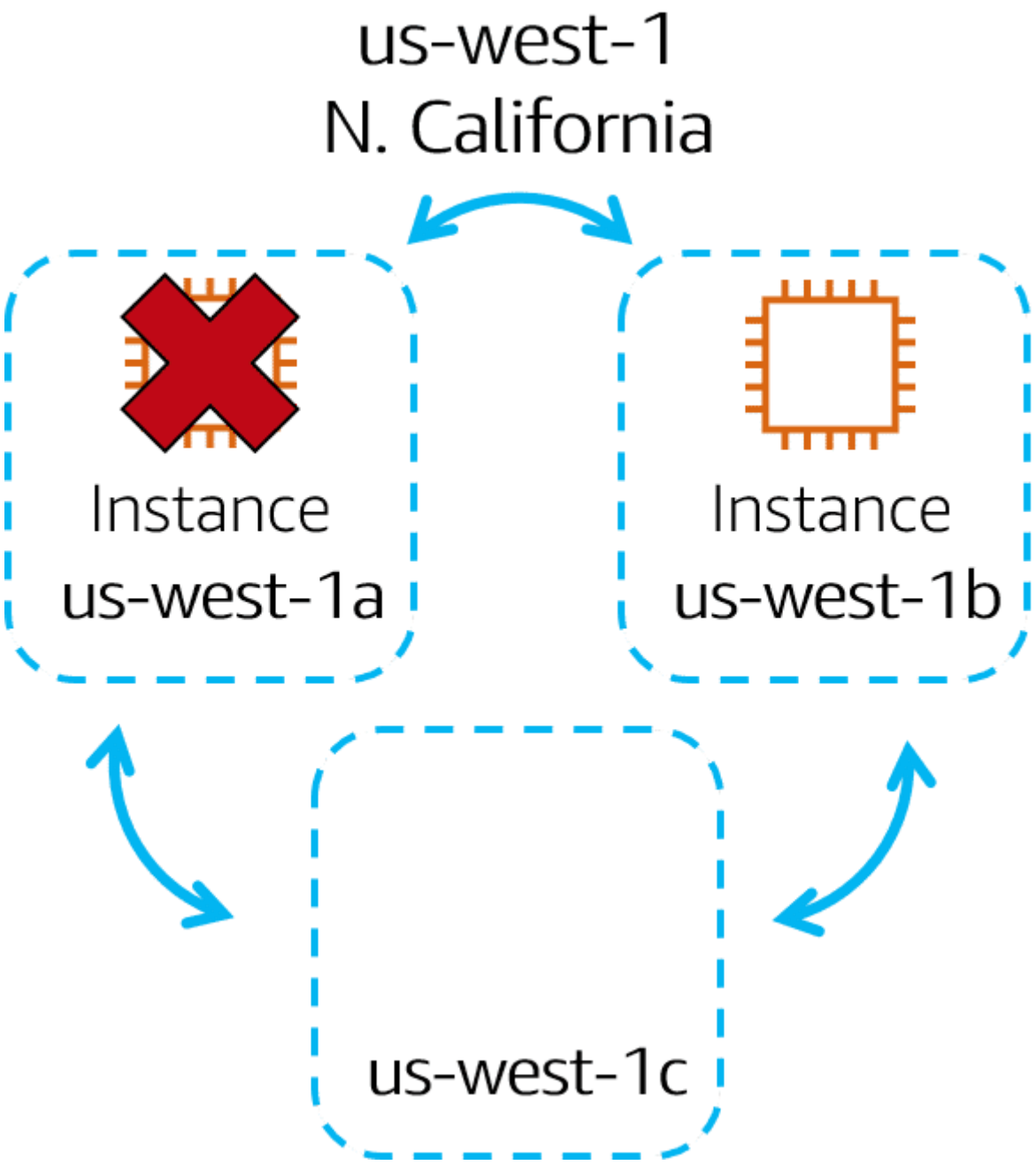
Amazon EC2 instances in multiple Availability Zones



A best practice is to run applications across at least two Availability Zones in a Region. In this example, you might choose to run a second Amazon EC2 instance in us-west-1b.

Step 3

Availability Zone failure



If us-west-1a were to fail, your application would still be running in us-west-1b.

Edge Locations

One of the great things about the AWS global infrastructure, is the way it's engineered to help you better serve your customers. Remember when selecting a Region, one of the major criteria was proximity to your customers, but what if you have customers all over the world or in cities that are not close to one of our Regions? Well, think about our coffee shop. If you have a good customer base in a new city, you can build a satellite store to service those customers.

You don't need to build an entire new headquarters or from an IT perspective, if you have customers in Mumbai who need access to your data, but the data is hosted out of the Tokyo Region, rather than having all the Mumbai-based customers, send requests all the way to Tokyo, to access the data, just place a copy locally or cache a copy in Mumbai. Caching copies of data closer to the customers all around the world uses the concept of content delivery networks, or CDNs.

CDNs are commonly used, and on AWS, we call our CDN Amazon CloudFront. Amazon CloudFront is a service that helps deliver data, video, applications, and APIs to customers around the world with low latency and high transfer speeds. Amazon CloudFront uses what are called Edge locations, all around the world, to help accelerate communication with users, no matter where they are.

Edge locations are separate from Regions, so you can push content from inside a Region to a collection of Edge locations around the world, in order to accelerate communication and content delivery. AWS Edge locations, also run more than just CloudFront. They run a domain name service, or DNS, known as Amazon Route 53, helping direct customers to the correct web locations with reliably low latency.

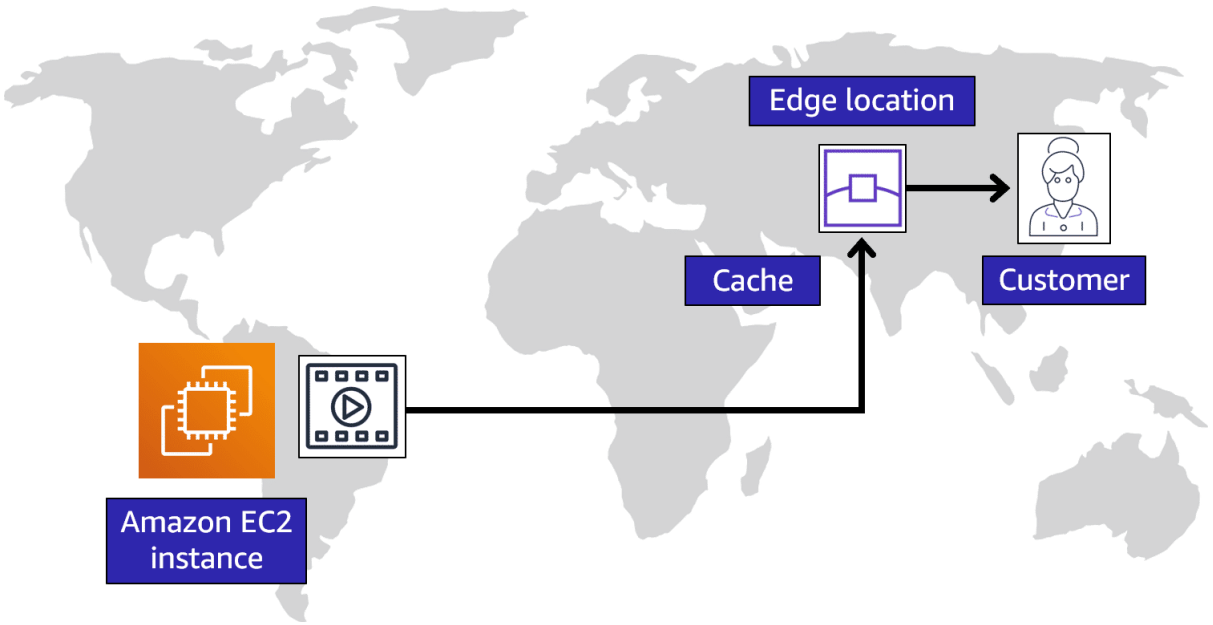
But what if your business wants to use, AWS services inside their own building? Well sure. AWS can do that for you. Introducing AWS Outposts, where AWS will basically install a fully operational mini Region, right inside your own data center. That's owned and operated by AWS, using 100% of AWS functionality, but isolated within your own building. It's not a solution most customers need, but if you have specific problems that can only be solved by staying in your own building, we understand, AWS Outposts can help.

All right, there is so much more that we can say about AWS global infrastructure, but let's keep it simple and stop here. So here's the key points. Number one, Regions are geographically isolated areas, where you can access services needed to run your enterprise. Number two, Regions contain Availability Zones, that allow you to run across physically separated buildings, tens of miles of separation, while keeping your application logically unified. Availability Zones help you solve high availability and disaster recovery scenarios, without any additional effort on your part, and number three, AWS Edge locations run Amazon CloudFront to help get content closer to your customers, no matter where they are in the world.

Edge locations

An **edge location** is a site that Amazon CloudFront uses to store cached copies of your content closer to your customers for faster delivery.

To learn more, choose each of the numbered markers.



Origin

Suppose that your company's data is stored in Brazil, and you have customers who live in China. To provide content to these customers, you don't need to move all the content to one of the Chinese Regions.

Edge location

Instead of requiring your customers to get their data from Brazil, you can cache a copy locally at an edge location that is close to your customers in China.

Customer

When a customer in China requests one of your files, Amazon CloudFront retrieves the file from the cache in the edge location and delivers the file to the customer. The file is delivered to the customer faster because it came from the edge location near China instead of the original source in Brazil.

How to Provision AWS Resources

We've been talking about a few different AWS resources as well as the AWS global infrastructure. You may be wondering, how do I actually interact with these services? And the answer is APIs. In AWS, everything is an API call. An API is an application programming interface. And what that means is, there are pre determined ways for you to interact with AWS services. And you can invoke or call these APIs to provision, configure, and manage your AWS resources.

For example, you can launch an EC2 instance or you can create an AWS Lambda function. Each of those would be different requests and different API calls to AWS. You

can use the AWS Management Console, the AWS Command Line Interface, the AWS Software Development Kits, or various other tools like AWS CloudFormation, to create requests to send to AWS APIs to create and manage AWS resources.

First, let's talk about the AWS Management Console. The AWS Management Console is browser-based. Through the console, you can manage your AWS resources visually and in a way that is easy to digest. This is great for getting started and building your knowledge of the services. It's also useful for building out test environments or viewing AWS bills, viewing monitoring and working with other non technical resources. The AWS Management Console is most likely the first place you will go when you are learning about AWS.

However, once you are up and running in a production type environment, you don't want to rely on the point and click style that the console gives you to create and manage your AWS resources. For example, in order to create an Amazon EC2 Instance, you need to click through various screens, setting all the configurations you want, and then you launch your instance. If later, you wanted to launch another EC2 Instance, you would need to go back into the console and click through those screens again to get it up and running. By having humans do this sort of manual provisioning, you're opening yourself up to potential errors. It's pretty easy to forget to check a checkbox or misspell something when you are doing everything manually.

The answer to this problem is to use tools that allow you to script or program the API calls. One tool you can use is the AWS Command Line Interface or CLI. The CLI allows you to make API calls using the terminal on your machine. This is different than the visual navigation style of the Management Console. Writing commands using the CLI makes actions scriptable and repeatable. So, you can write and run your commands to launch an EC2 Instance. And if you want to launch another, you can just run the pre-written command again. This makes it less susceptible to human error. And you can have these scripts run automatically, like on a schedule or triggered by another process.

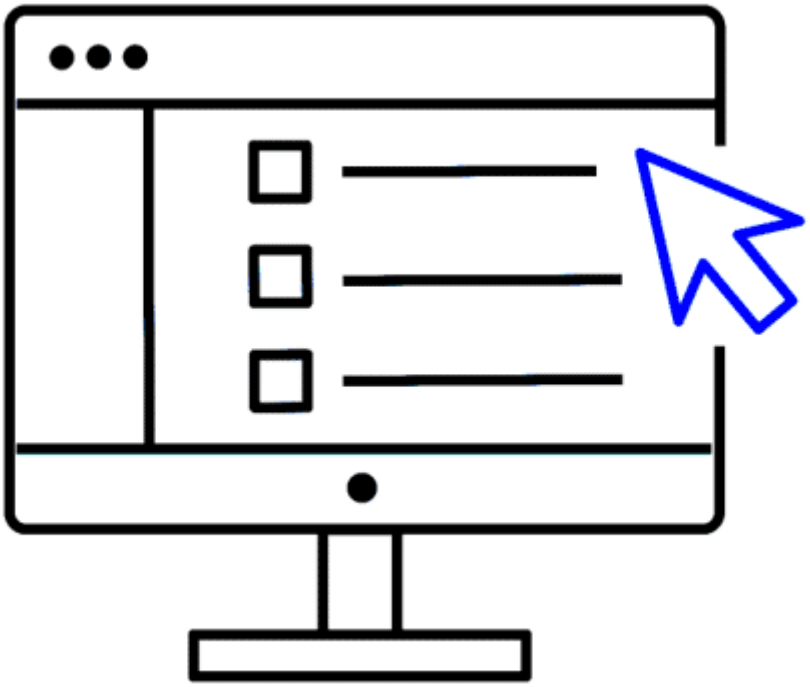
Automation is very important to having a successful and predictable cloud deployment over time. Another way to interact with AWS is through the AWS Software Development Kits or SDKs. The SDKs allow you to interact with AWS resources through various programming languages. This makes it easy for developers to create programs that use AWS without using the low level APIs, as well as avoiding that manual resource creation that we just talked about. More on that in a bit.

Ways to interact with AWS services

To learn about ways to interact with AWS services choose each of the following three tabs.

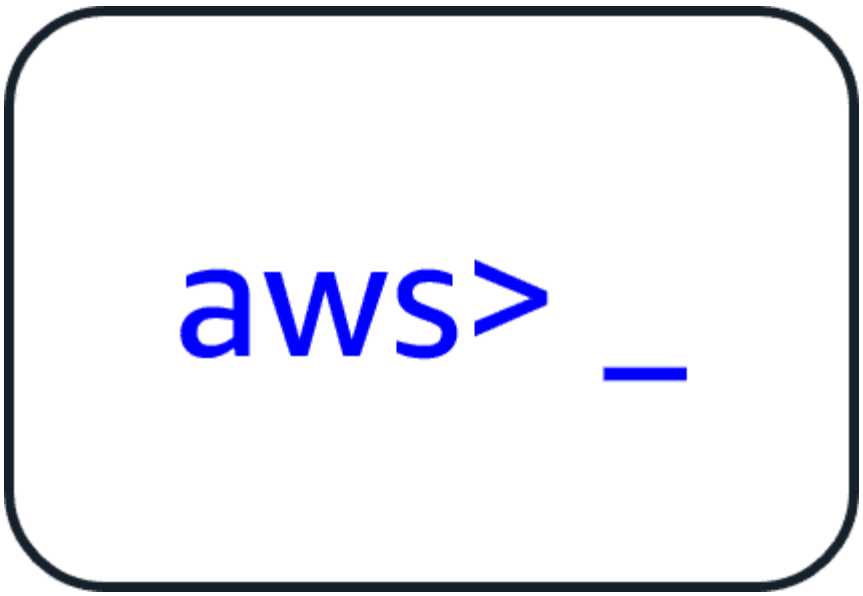
The **AWS Management Console** is a web-based interface for accessing and managing AWS services. You can quickly access recently used services and search for other services by name, keyword, or acronym. The console includes wizards and automated workflows that can simplify the process of completing tasks.

You can also use the AWS Console mobile application to perform tasks such as monitoring resources, viewing alarms, and accessing billing information. Multiple identities can stay logged into the AWS Console mobile app at the same time.



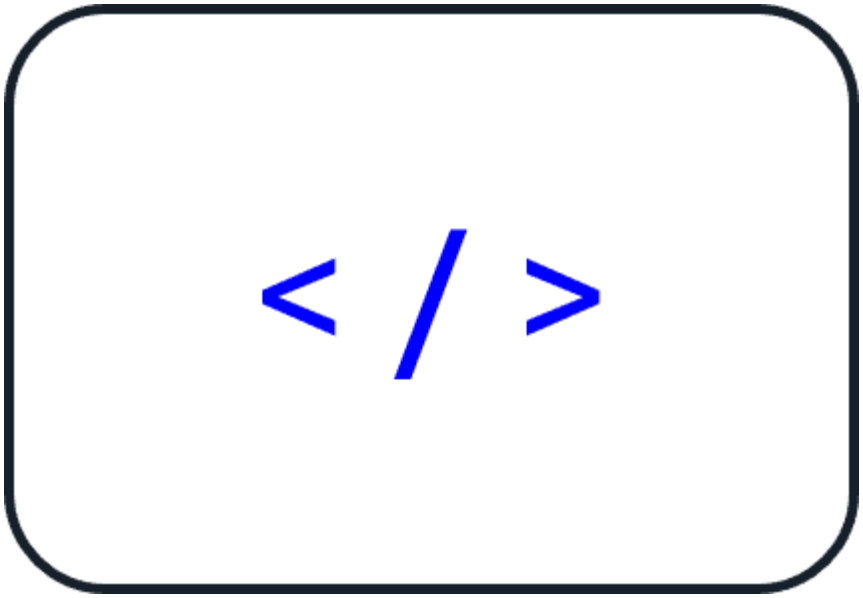
To save time when making API requests, you can use the **AWS Command Line Interface (AWS CLI)**. AWS CLI enables you to control multiple AWS services directly from the command line within one tool. AWS CLI is available for users on Windows, macOS, and Linux.

By using AWS CLI, you can automate the actions that your services and applications perform through scripts. For example, you can use commands to launch an Amazon EC2 instance, connect an Amazon EC2 instance to a specific Auto Scaling group, and more.



Another option for accessing and managing AWS services is the **software development kits (SDKs)**. SDKs make it easier for you to use AWS services through an API designed for your programming language or platform. SDKs enable you to use AWS services with your existing applications or create entirely new applications that will run on AWS.

To help you get started with using SDKs, AWS provides documentation and sample code for each supported programming language. Supported programming languages include C++, Java, .NET, and more.



All right, let's continue to talk about how to interact with AWS. You have the AWS Management Console, the CLI, and the SDKs, which are all sort of do it yourself ways to provision and manage your AWS environment. If you want to provision a resource, you can log into the console and point and click, you can write some commands, or you can

write some programs to do it. There are also other ways you can manage your AWS environment using managed tools like AWS Elastic Beanstalk, and AWS CloudFormation.

AWS Elastic Beanstalk is a service that helps you provision Amazon EC2-based environments. Instead of clicking around the console or writing multiple commands to build out your network, EC2 instances, scaling and Elastic Load Balancers, you can instead provide your application code and desired configurations to the AWS Elastic Beanstalk service, which then takes that information and builds out your environment for you. AWS Elastic Beanstalk also makes it easy to save environment configurations, so they can be deployed again easily. AWS Elastic Beanstalk gives you the convenience of not having to provision and manage all of these pieces separately, while still giving you the visibility and control of the underlying resources. You get to focus on your business application, not the infrastructure.

Another service you can use to help create automated and repeatable deployments is AWS CloudFormation. AWS CloudFormation is an infrastructure as code tool that allows you to define a wide variety of AWS resources in a declarative way using JSON or YAML text-based documents called CloudFormation templates. A declarative format like this allows you to define what you want to build without specifying the details of exactly how to build it. CloudFormation lets you define what you want and the CloudFormation engine will worry about the details on calling APIs to get everything built out.

It also isn't just limited to EC2-based solutions. CloudFormation supports many different AWS resources from storage, databases, analytics, machine learning, and more. Once you define your resources in a CloudFormation template, CloudFormation will parse the template and begin provisioning all the resources you defined in parallel. CloudFormation manages all the calls to the backend AWS APIs for you. You can run the same CloudFormation template in multiple accounts or multiple regions, and it will create identical environments across them. There is less room for human error as it is a totally automated process.

To recap, the AWS Management Console is great for learning and providing a visual for the user. The AWS Management Console is a manual tool. So right off the bat, it isn't a great option for automation. You can instead use the CLI to script your interactions with AWS using the terminal. You can use the SDKs to write programs to interact with AWS for you or you can use manage tools like AWS Elastic Beanstalk or AWS CloudFormation.

AWS Elastic Beanstalk

With **AWS Elastic Beanstalk**, you provide code and configuration settings, and Elastic Beanstalk deploys the resources necessary to perform the following tasks:

- Adjust capacity
- Load balancing
- Automatic scaling
- Application health monitoring

AWS CloudFormation

With **AWS CloudFormation**, you can treat your infrastructure as code. This means that you can build an environment by writing lines of code instead of using the AWS Management Console to individually provision resources.

AWS CloudFormation provisions your resources in a safe, repeatable manner, enabling you to frequently build your infrastructure and applications without having to perform manual actions. It determines the right operations to perform when managing your stack and rolls back changes automatically if it detects errors.

Module 3 Summary

In Module 3, you learned about the following concepts:

- AWS Regions and Availability Zones
- Edge locations and Amazon CloudFront
- The AWS Management Console, AWS CLI, and SDKs
- AWS Elastic Beanstalk
- AWS CloudFormation

Module 4 Introduction

Learning objectives

In this module, you will learn how to:

- Describe the basic concepts of networking.
- Describe the difference between public and private networking resources.
- Explain a virtual private gateway using a real life scenario.
- Explain a virtual private network (VPN) using a real life scenario.
- Describe the benefit of AWS Direct Connect.
- Describe the benefit of hybrid deployments.
- Describe the layers of security used in an IT strategy.
- Describe the services customers use to interact with the AWS global network.

Connectivity to AWS

A VPC, or Virtual Private Cloud, is essentially your own private network in AWS. A VPC allows you to define your private IP range for your AWS resources, and you place things like EC2 instances and ELBs inside of your VPC.

Now you don't just go throw your resources into a VPC and move on. You place them into different subnets. Subnets are chunks of IP addresses in your VPC that allow you to group resources together. Subnets, along with networking rules we will cover later, control whether resources are either publicly or privately available. What we haven't told you yet is there are actually ways you can control what traffic gets into your VPC at all. What I mean by this is, for some VPCs, you might have internet-facing resources that the public should be able to reach, like a public website, for example.

However, in other scenarios, you might have resources that you only want to be reachable if someone is logged into your private network. This might be internal services, like an HR application or a backend database. First, let's talk about public-facing resources. In order to allow traffic from the public internet to flow into and out of your VPC, you must attach what is called an internet gateway, or IGW, to your VPC.

An internet gateway is like a doorway that is open to the public. Think of our coffee shop. Without a front door, the customers couldn't get in and order their coffee, so we install a front door and the people can then go in and out of that door when coming and going from our shop. The front door in this example is like an internet gateway. Without it, no one can reach the resources placed inside of your VPC.

Next, let's talk about a VPC with all internal private resources. We don't want just anyone from anywhere to be able to reach these resources. So we don't want an internet gateway attached to our VPC. Instead, we want a private gateway that only allows people in if they are coming from an approved network, not the public internet. This private doorway is called a virtual private gateway, and it allows you to create a VPN connection between a private network, like your on-premises data center or internal corporate network to your VPC.

To relate this back to the coffee shop, this would be like having a private bus route going from my building to the coffee shop. If I want to go get coffee, I first must badge into the building, thus authenticating my identity, and then I can take the secret bus route to the internal coffee shop that only people from my building can use. So if you want to establish an encrypted VPN connection to your private internal AWS resources, you would need to attach a virtual private gateway to your VPC.

Now the problem with our super secret bus route is that it still uses the open road. It's susceptible to traffic jams and slowdowns caused by the rest of the world going about their business. The same thing is true for VPN connections. They are private and they are encrypted, but they still use a regular internet connection that has bandwidth that is being shared by many people using the internet.

So what I've done to make things more reliable and less susceptible to slowdowns is I made a totally separate magic doorway that leads directly from the studio into the coffee shop. No one else driving

around on the road can slow me down because this is my direct doorway; no one else can use it. What, did you not have a secret magic doorway into your favorite coffee shop? All right, moving on. The point being is you still want a private connection, but you want it to be dedicated and shared with no one else. You want the lowest amount of latency possible with the highest amount of security possible.

With AWS, you can achieve that using what is called AWS Direct Connect. Direct Connect allows you to establish a completely private, dedicated fiber connection from your data center to AWS. You work with a Direct Connect partner in your area to establish this connection, because like my magic doorway, AWS Direct Connect provides a physical line that connects your network to your AWS VPC. This can help you meet high regulatory and compliance needs, as well as sidestep any potential bandwidth issues. It's also important to note that one VPC might have multiple types of gateways attached for multiple types of resources all residing in the same VPC, just in different subnets.

Amazon Virtual Private Cloud (Amazon VPC)

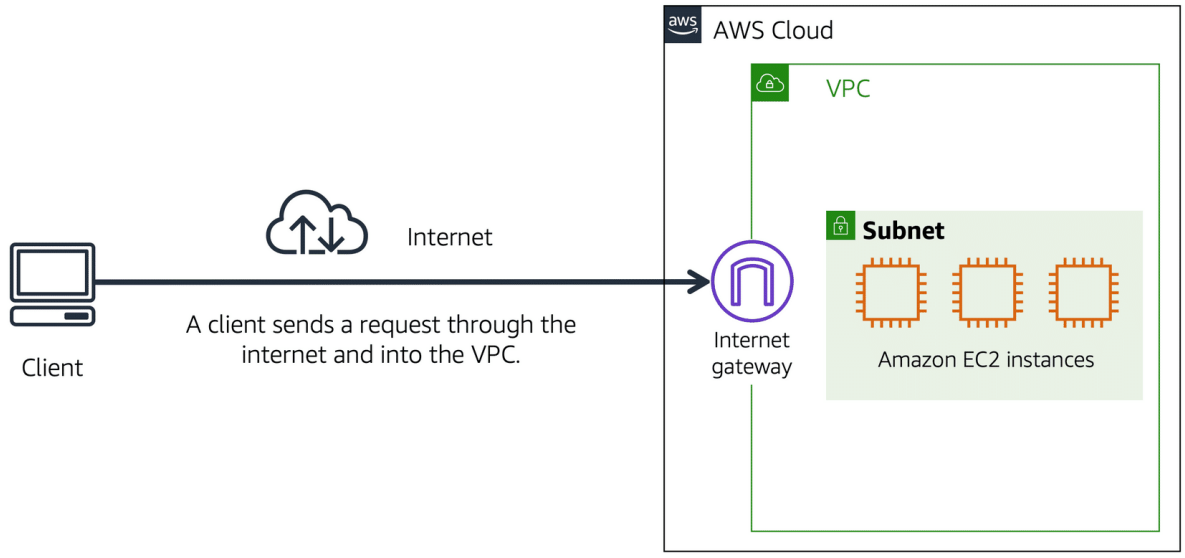
Imagine the millions of customers who use AWS services. Also, imagine the millions of resources that these customers have created, such as Amazon EC2 instances. Without boundaries around all of these resources, network traffic would be able to flow between them unrestricted.

A networking service that you can use to establish boundaries around your AWS resources is **Amazon Virtual Private Cloud (Amazon VPC)**(opens in a new tab).

Amazon VPC enables you to provision an isolated section of the AWS Cloud. In this isolated section, you can launch resources in a virtual network that you define. Within a virtual private cloud (VPC), you can organize your resources into subnets. A **subnet** is a section of a VPC that can contain resources such as Amazon EC2 instances.

Internet gateway

To allow public traffic from the internet to access your VPC, you attach an **internet gateway** to the VPC.



Internet gateway icon attached to a VPC that holds three EC2 instances. An arrow connects the client to the gateway over the internet indicating that the client's request has gained access to the VPC.

An internet gateway is a connection between a VPC and the internet. You can think of an internet gateway as being similar to a doorway that customers use to enter the coffee shop. Without an internet gateway, no one can access the resources within your VPC.

What if you have a VPC that includes only private resources?

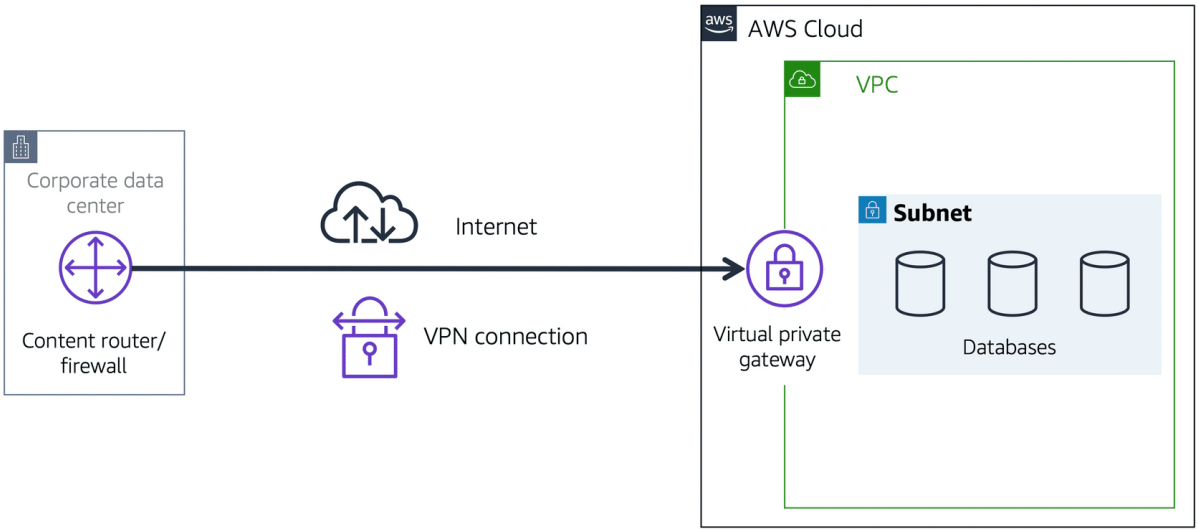
Virtual private gateway

To access private resources in a VPC, you can use a **virtual private gateway**.

Here's an example of how a virtual private gateway works. You can think of the internet as the road between your home and the coffee shop. Suppose that you are traveling on this road with a bodyguard to protect you. You are still using the same road as other customers, but with an extra layer of protection.

The bodyguard is like a virtual private network (VPN) connection that encrypts (or protects) your internet traffic from all the other requests around it.

The virtual private gateway is the component that allows protected internet traffic to enter into the VPC. Even though your connection to the coffee shop has extra protection, traffic jams are possible because you're using the same road as other customers.



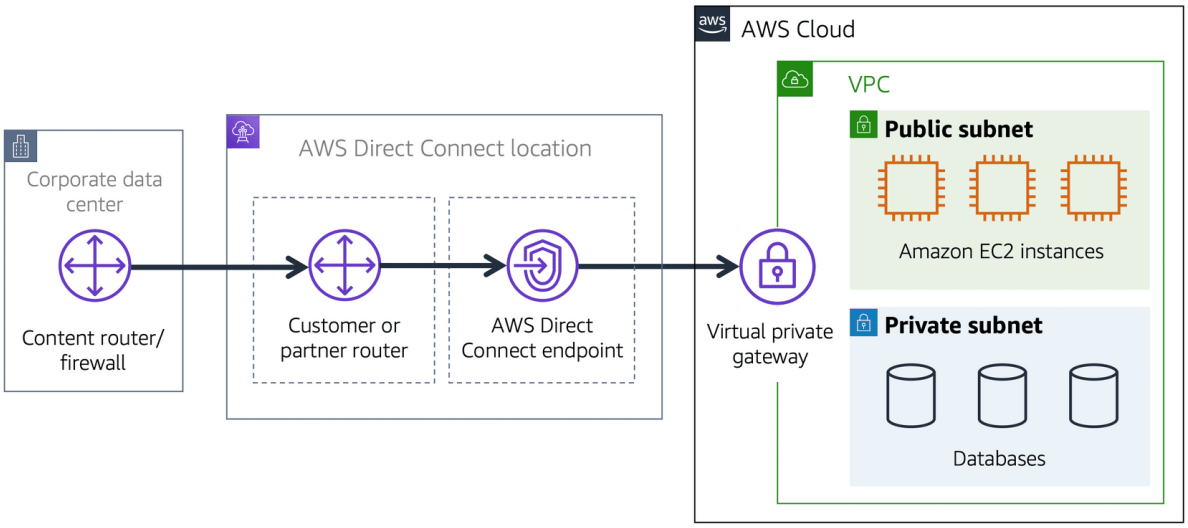
A virtual private gateway enables you to establish a virtual private network (VPN) connection between your VPC and a private network, such as an on-premises data center or internal corporate network. A virtual private gateway allows traffic into the VPC only if it is coming from an approved network.

AWS Direct Connect

AWS Direct Connect(opens in a new tab) is a service that lets you to establish a dedicated private connection between your data center and a VPC.

Suppose that there is an apartment building with a hallway directly linking the building to the coffee shop. Only the residents of the apartment building can travel through this hallway.

This private hallway provides the same type of dedicated connection as AWS Direct Connect. Residents are able to get into the coffee shop without needing to use the public road shared with other customers.



A corporate data center routes network traffic to an AWS Direct Connect location. That traffic is then routed to a VPC through a virtual private gateway. All network traffic between the corporate data center and VPC flows through this dedicated private connection.

The private connection that AWS Direct Connect provides helps you to reduce network costs and increase the amount of bandwidth that can travel through your network.

Subnets and Network Access Control Lists

Welcome to your VPC. You can think of it as a hardened fortress where nothing goes in or out without explicit permission. You have a gateway on the VPC that only permits traffic in or out of the VPC. But that only covers perimeter, and that's only one part of network security that you should be focusing on as part of your IT strategy.

AWS has a wide range of tools that cover every layer of security: network hardening, application security, user identity, authentication and authorization, distributed denial-of-service or DDoS prevention, data integrity, encryption, much more. We're gonna talk about a few of these key pieces. And if you really wanna know more about security, please follow the links in this page to be directed to more information and additional training on how to lock your infrastructure down tighter than Aunt Robin's secret lemon pie recipe, and ain't nobody getting that recipe.

Today, I wanna talk about a few aspects of network hardening looking at what happens inside the VPC. Now, the only technical reason to use subnets in a VPC is to control access to the gateways. The public subnets have access to the internet gateway; the private subnets do not. But subnets can also control traffic permissions. Packets are messages from the internet, and every packet that crosses the subnet boundaries gets checked against something called a network access control list or network ACL. This check is to see if the packet has permissions to either leave or enter the subnet based on who it was sent from and how it's trying to communicate.

You can think of network ACLs as passport control officers. If you're on the approved list, you get through. If you're not on the list, or if you're explicitly on the do-not-enter list, then you get blocked. Network ACLs check traffic going into and leaving a subnet, just like passport control. The list gets checked on your way into a country and on the way out. And just because you were let in doesn't necessarily mean they're gonna let you out. Approved traffic can be sent on its way, and potentially harmful traffic, like attempts to gain control of a system through administrative requests, they get blocked before they ever touch the target. You can't hack what you can't touch.

Now, this sounds like great security, but it doesn't answer all of the network control issues. Because a network ACL only gets to evaluate a packet if it crosses a subnet boundary, in or out. It doesn't evaluate if a packet can reach a specific EC2 instance or not. Sometimes, you'll have multiple EC2 instances in the same subnet, but they might have different rules around who can send them messages, what port those messages are allowed to be sent to. So you need instance level network security as well.

To solve instance level access questions, we introduce security groups. Every EC2 instance, when it's launched, automatically comes with a security group. And by default, the security group does not allow any traffic into the instance at all. All ports are blocked; all IP addresses sending packets are blocked. That's very secure, but perhaps not very useful. If you want an instance to actually accept traffic from the outside, like say, a message from a front end instance or a message from the Internet. So obviously, you can modify the security group to accept a specific type of traffic. In the case of a website, you want web-based traffic or HTTPS to be accepted but not other types of traffic, say an operating system or administration requests.

If NACLs are a passport control, a security group is like the doorman at your building, the building being the EC2 Instance, in this case. The doorman will check a list to ensure that someone is allowed to enter the building but won't bother check the list on the way out. With security groups, you allow specific traffic in and by default, all traffic is allowed out. Now, wait a minute, Blaine. You just described two different engines each doing the exact same job. Let good packets in, keep bad packets out. The key difference between a security group and a network ACL is the security group is stateful, meaning, as we talked about, it has some kind of a memory when it comes to who to allow in or out, and the network ACL is stateless, which remembers nothing and checks every single packet that crosses its border regardless of any circumstances.

You know, this metaphor is important to understand. So I wanna illustrate the round trip of a packet as it goes from one instance to another instance in a different subnet. Now, this traffic management, it doesn't care about the contents of the packet itself. In fact, it doesn't even open the envelope, it can't. All it can do is check to see if the sender is on the approved list.

All right. Let's start with instance A. We wanna send a packet from instance A to instance B in a different subnet, same VPC, different subnets. So instance A sends the packet. Now, the first thing that happens is that packet meets the boundary of the security group of instance A. By default, all outbound traffic is allowed from a security group. So you can walk right by the doormen and leave, cool, right. The packet made it past the security group of instance A. Now it has to leave the subnet boundary. At the boundary, the passport must now make it through passport control, the network ACL. The network ACL doesn't care about what the security group allowed. It has its own list of who can pass and who can't. If the target address is allowed, you can keep going on your journey, which it is.

So now we have exited the original subnet and now the packet goes to the target subnet where instance B lives. Now at this target subnet, once again, we have passport control. Just because the packet was allowed out of its home country does not mean that it can enter the destination country or subnet in this case. They both have unique passport officers with their own checklists. You have to be approved on both lists, or you could get turned away at the border. Well, turns out the packet is on the approved list for this subnet, so it's allowed to enter through the network ACL into the subnet. Almost there. Now, we're approaching the target instance, instance B. Every EC2 Instance has their own security group. You wanna enter this instance, the doorman will need to check to see if you're allowed in, and we're on the list. The packet has reached the target instance.

Once the transaction's complete, now it's just time to come home. It's the return traffic pattern. It's the most interesting, because this is where the stateful versus stateless nature of the different engines come into play. Because the packet still has to be evaluated at each checkpoint. Security groups, by default, allow all return traffic. So they don't have to check a list to see if they're allowed out. Instead, they automatically allow the return traffic to pass by no matter what. Passport control here at the subnet boundary, these network ACLs do not remember state. They don't care that the packet came in here. It might be on a you-can't-leave list. Every ingress and egress is checked with the appropriate list. The package return address has to be on their approved list to make it across the border. Made it to the border of the origin subnet, but we have to negotiate passport network ACL control here as well. Stateless controls, means it always checks its list.

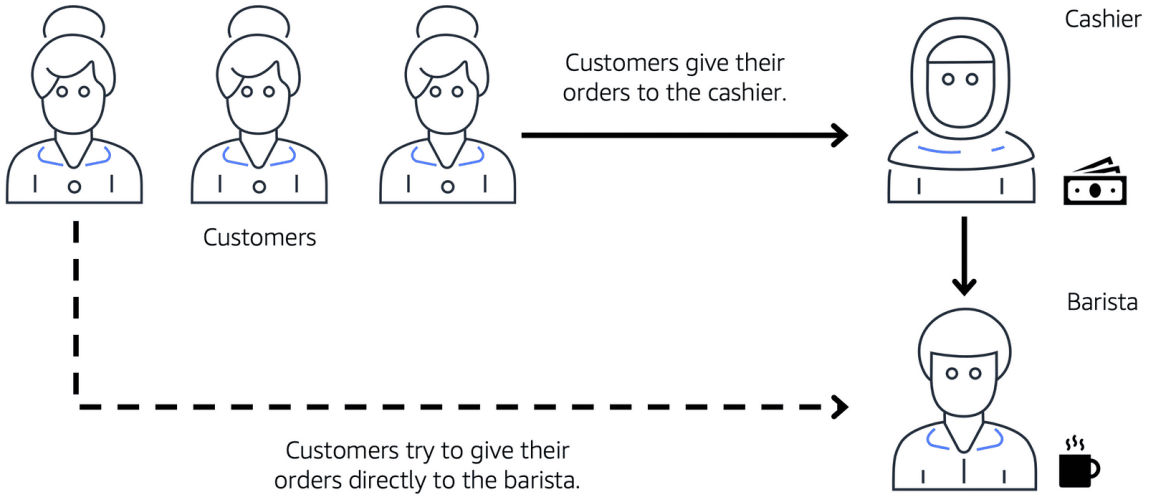
The packet pass the network ACL, the subnet level, which means the packets now made it back to instance A but the security group, the doorman is still in charge here. The key difference though is these security groups are stateful. The security group recognizes the packet from before. So it doesn't need to check to see if it's allowed in. Come on home.

Now, this may seem like we spent a lot of effort just getting a packet from one instance to another and back. You might be concerned about all the network overhead this might generate. The reality is all of these exchanges happen instantly as part of how AWS Networking actually works. If you wanna know the technology that makes all that possible, well, then you need to sign up for a completely different set of trainings. Good network security will take advantage of both network ACLs and security groups, because security in-depth is critical for modern architectures.

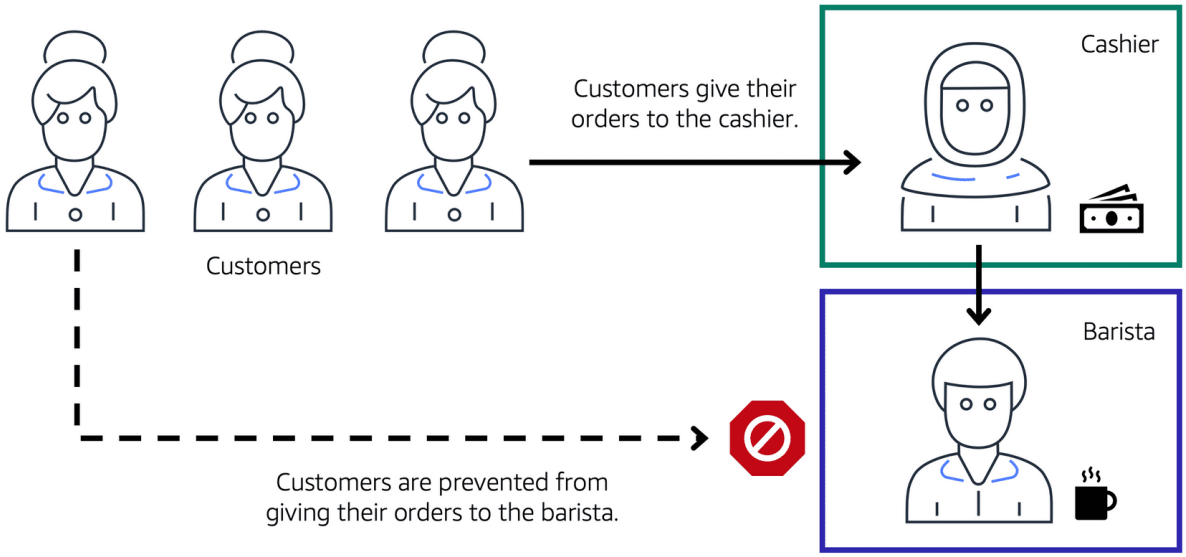
To learn more about the role of subnets within a VPC, review the following example from the coffee shop.

First, customers give their orders to the cashier. The cashier then passes the orders to the barista. This process allows the line to keep running smoothly as more customers come in.

Suppose that some customers try to skip the cashier line and give their orders directly to the barista. This disrupts the flow of traffic and results in customers accessing a part of the coffee shop that is restricted to them.



To fix this, the owners of the coffee shop divide the counter area by placing the cashier and the barista in separate workstations. The cashier's workstation is public facing and designed to receive customers. The barista's area is private. The barista can still receive orders from the cashier but not directly from customers.



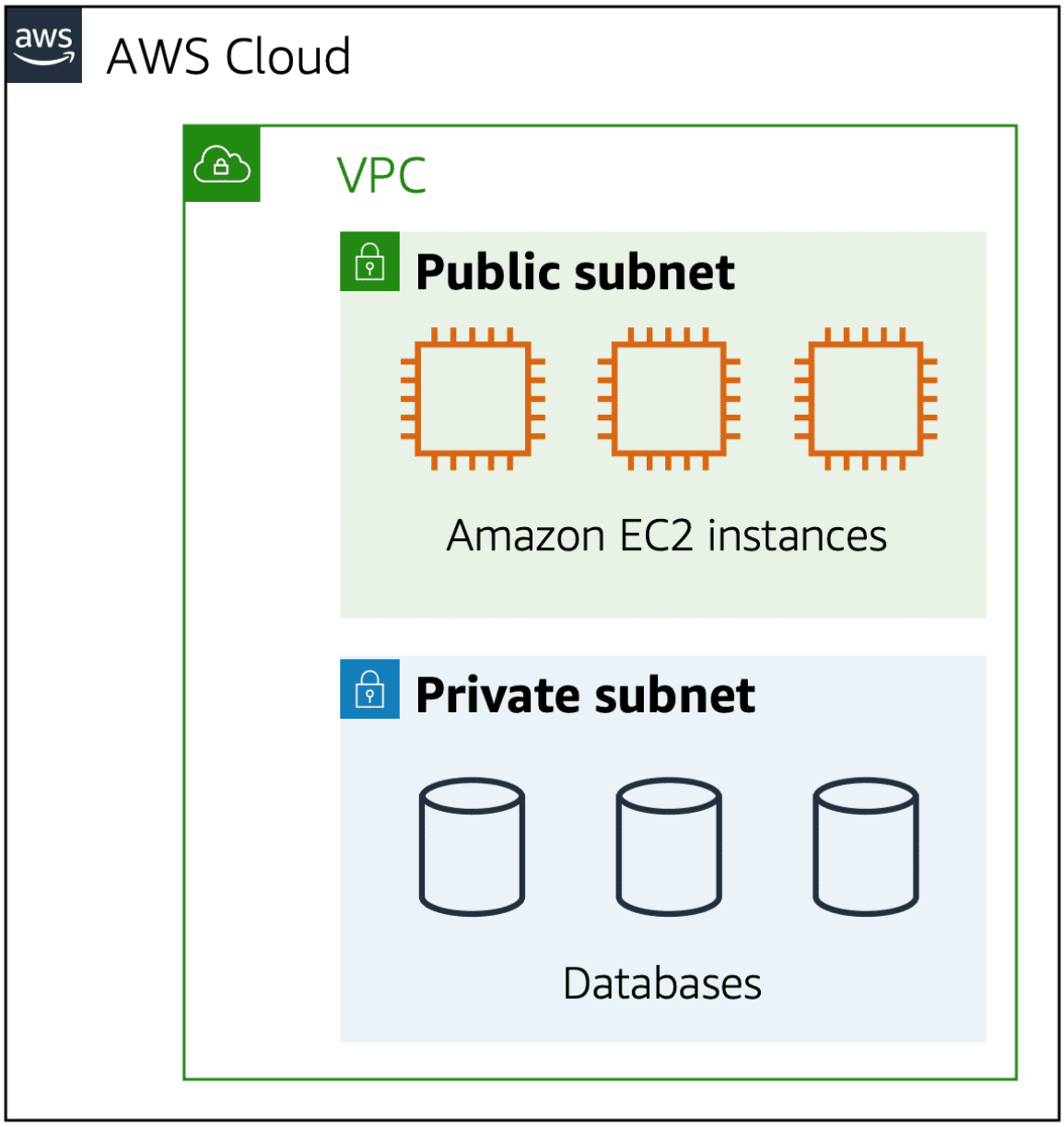
A cashier, a barista, and three customers in line. The icon for the first customer in line has an arrow pointing to cashier showing that the customer gives their order to the cashier. Then the cashier icon has an arrow pointing to barista icon showing that the cashier forwards the customer's order to the barista. The last customer in line tries to give their order directly to the barista, but they're blocked from doing so.

This is similar to how you can use AWS networking services to isolate resources and determine exactly how network traffic flows.

In the coffee shop, you can think of the counter area as a VPC. The counter area divides into two separate areas for the cashier's workstation and the barista's workstation. In a VPC, **subnets** are separate areas that are used to group together resources.

Subnets

A subnet is a section of a VPC in which you can group resources based on security or operational needs. Subnets can be public or private.



Public subnets contain resources that need to be accessible by the public, such as an online store's website.

Private subnets contain resources that should be accessible only through your private network, such as a database that contains customers' personal information and order histories.

In a VPC, subnets can communicate with each other. For example, you might have an application that involves Amazon EC2 instances in a public subnet communicating with databases that are located in a private subnet.

Network traffic in a VPC

When a customer requests data from an application hosted in the AWS Cloud, this request is sent as a packet. A **packet** is a unit of data sent over the internet or a network.

It enters into a VPC through an internet gateway. Before a packet can enter into a subnet or exit from a subnet, it checks for permissions. These permissions indicate who sent the packet and how the packet is trying to communicate with the resources in a subnet.

The VPC component that checks packet permissions for subnets is a **network access control list (ACL)**(opens in a new tab).

Network ACLs

A network ACL is a virtual firewall that controls inbound and outbound traffic at the subnet level.

For example, step outside of the coffee shop and imagine that you are in an airport. In the airport, travelers are trying to enter into a different country. You can think of the travelers as packets and the passport control officer as a network ACL. The passport control officer checks travelers' credentials when they are both entering and exiting out of the country. If a traveler is on an approved list, they are able to get through. However, if they are not on the approved list or are explicitly on a list of banned travelers, they

cannot come in.

Each AWS account includes a default network ACL. When configuring your VPC, you can

use your account's default network ACL or create custom network ACLs.

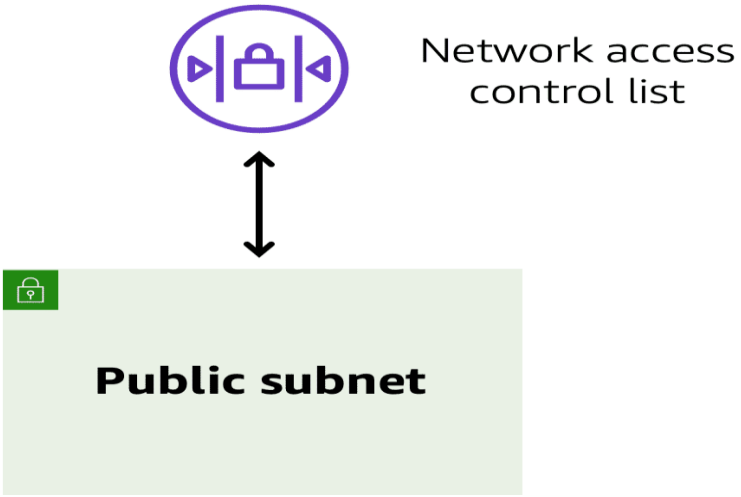
By default, your account's default network ACL allows all inbound and outbound traffic,

but you can modify it by adding your own rules. For custom network ACLs, all inbound

and outbound traffic is denied until you add rules to specify which traffic to allow.

Additionally, all network ACLs have an explicit deny rule. This rule ensures that if a

packet doesn't match any of the other rules on the list, the packet is

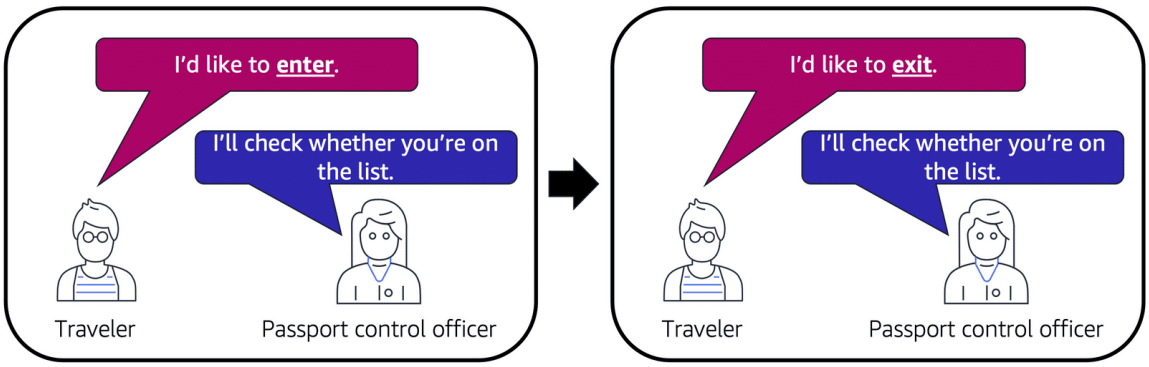


denied.

Stateless packet filtering

Network ACLs perform stateless packet filtering. They remember nothing and check packets that cross the subnet border each way: inbound and outbound.

Recall the previous example of a traveler who wants to enter into a different country. This is similar to sending a request out from an Amazon EC2 instance and to the internet.When a packet response for that request comes back to the subnet, the network ACL does not remember your previous request. The network ACL checks the packet response against its list of rules to determine whether to allow or deny.



After a packet has entered a subnet, it must have its permissions evaluated for resources within the subnet, such as Amazon EC2 instances.

The VPC component that checks packet permissions for an Amazon EC2 instance is a **security group**(opens in a new tab).

Security groups

A security group is a virtual firewall that controls inbound and outbound traffic for an Amazon EC2 instance.

By default, a security group denies all inbound traffic and allows all outbound traffic. You can add custom rules to configure which traffic should be allowed; any other traffic would then be denied

For this example, suppose that you are in an apartment building with a door attendant who greets guests in the lobby. You can think of the guests as packets and the door attendant as a security group. As guests arrive, the door attendant checks a list to ensure they can enter the building. However, the door attendant does not check the list again when guests are exiting the building

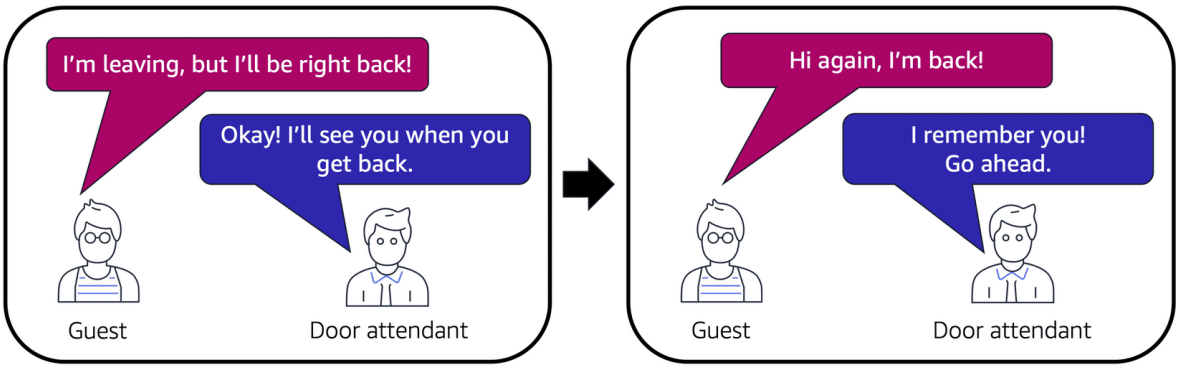
If you have multiple Amazon EC2 instances within the same VPC, you can associate them with the same security group or use different security groups for each instance.

Stateful packet filtering

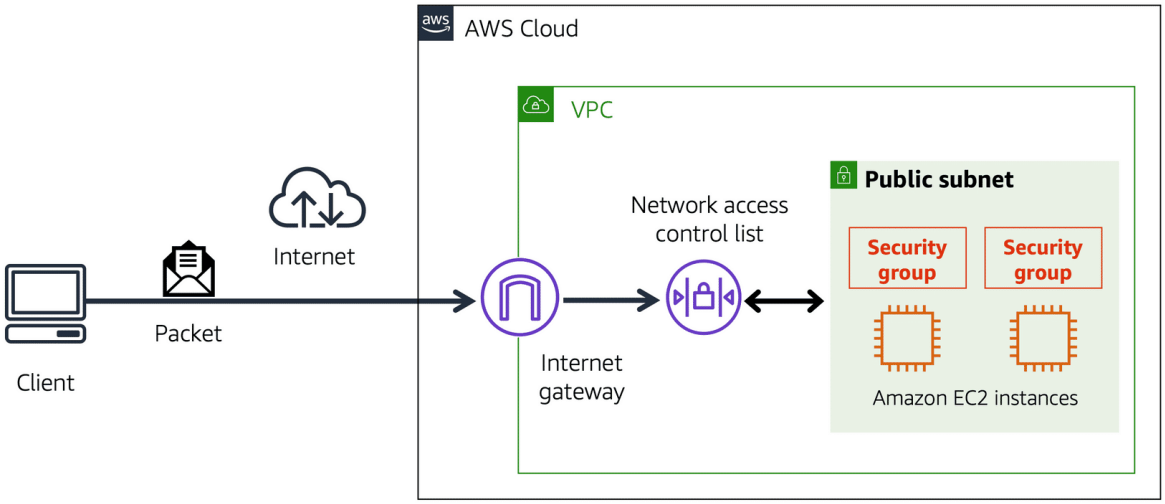
Security groups perform **stateful** packet filtering. They remember previous decisions made for incoming packets.

Consider the same example of sending a request out from an Amazon EC2 instance to the internet.

When a packet response for that request returns to the instance, the security group remembers your previous request. The security group allows the response to proceed, regardless of inbound security group rules.



With both network ACLs and security groups, you can configure custom rules for the traffic in your VPC. As you continue to learn more about AWS security and networking, make sure to understand the differences between network ACLs and security groups.



A packet travels over the internet from a client, to the internet gateway and into the VPC. Then the pack goes through the network access control list and accesses the public subnet, where two EC2 instances are located.

VPC component recall

Recall the purpose of the following four VPC components. Compare your response by choosing each VPC component flashcard.

To practice recalling VPC components, select each of the following flashcards by choosing them.

Private subnet

Isolate databases containing customers' personal information.

Virtual private gateway

Create a VPN connection between the VPC and the internal corporate network.

Public subnet

Support the customer-facing website.

AWS Direct Connect

Establish a dedicated connection between the on-premises data center and the VPC.

Global Networking

We've been talking a lot about how you interact with your AWS infrastructure. But how do your customers interact with your AWS infrastructure? Well, if you have a website hosted at AWS, then customers usually enter your website into their browser, hit Enter, some magic happens, and the site opens up.

But how does this magic work? Well, just like this magic coin that I have here, you know, you take a bite, and it's, it's back. Well, not quite like that, but I'm going to take you through two services, which would help in the website case. The first one being Route 53. Route 53 is AWS's domain name service, or DNS, and it's highly available and scalable. But wait, what is DNS, you say? Think of DNS as a translation service. But instead of translating between languages, it translates website names into IP, or Internet Protocol, addresses that computers can read.

For example, when we enter a website address into our browser, it contacts Route 53 to obtain the IP address of the site, say, 192.1.1.1, then it routes your computer or browser to that address. If we go further, Route 53 can direct traffic to different endpoints using several different routing policies, such as latency-based routing, geolocation DNS, geoproximity, and weighted round robin. If we take geolocation DNS, that means we direct traffic based on where the customer is located. So traffic coming from say North America is routed to the Oregon Region, and traffic in Ireland is routed to the Dublin Region, as an example.

You can even use Route 53 to register domain names, so you can buy and manage your own domain names right on AWS.

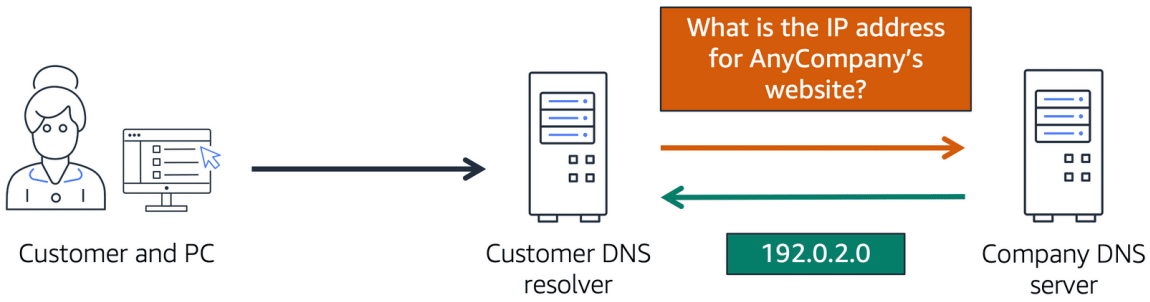
Speaking of websites, there is another service which can help speed up delivery of website assets to customers, Amazon CloudFront. If you remember, we talked about Edge locations earlier in the course, these locations are serving content as close to customers as possible, and one part of that, is the content delivery network, or CDN. For those who need reminding, a CDN is a network that helps to deliver edge content to users based on their geographic location.

If we go back to our North America versus Ireland example, say we have a user in Seattle, and they want to access a website, to speed this up, we host the site in Oregon, and we deploy our static web assets, like images and GIFs in CloudFront in North America. This means they get content delivered as close to them as possible, North America in this case, when they access the site. But for our Irish users, it doesn't make sense to deliver those assets out of Oregon, as the latency is not favorable. Thus we deploy those same static assets in CloudFront, but this time in the Dublin Region. That means they can access the same content, but from a location closer to them, which in turn improves latency.

Domain Name System (DNS)

Suppose that AnyCompany has a website hosted in the AWS Cloud. Customers enter the web address into their browser, and they are able to access the website. This happens because of **Domain Name System (DNS)** resolution. DNS resolution involves a customer DNS resolver communicating with a company DNS server.

You can think of DNS as being the phone book of the internet. DNS resolution is the process of translating a domain name to an IP address.



A client connects to a DNS resolver looking for a domain. The resolver forwards the request to the DNS server, which returns the IP address to the resolver.

For example, suppose that you want to visit AnyCompany's website.

1. 1

   When you enter the domain name into your browser, this request is sent to a customer DNS resolver.

2. 2

   The customer DNS resolver asks the company DNS server for the IP address that corresponds to AnyCompany's website.

3. 3

   The company DNS server responds by providing the IP address for AnyCompany's website, 192.0.2.0.

Amazon Route 53

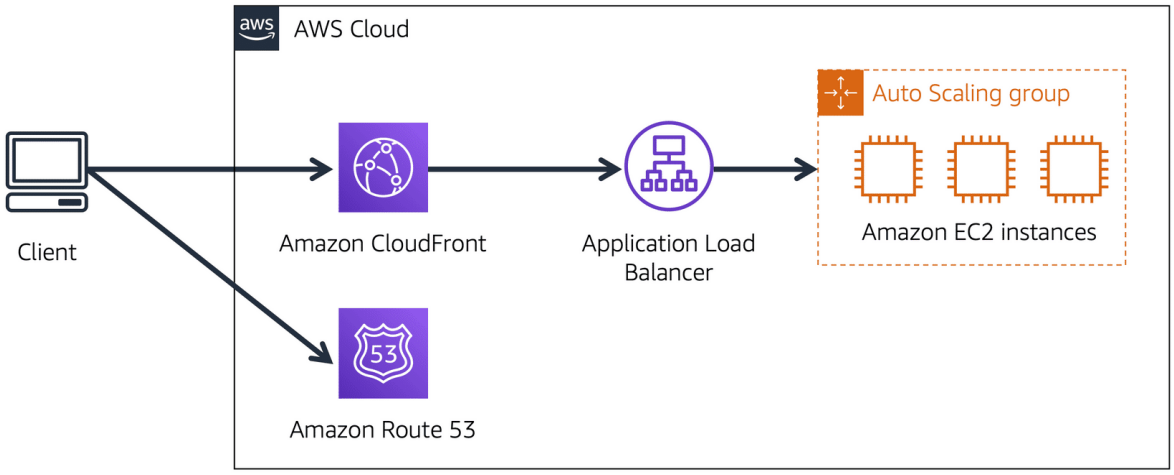
Amazon Route 53(opens in a new tab) is a DNS web service. It gives developers and businesses a reliable way to route end users to internet applications hosted in AWS.

Amazon Route 53 connects user requests to infrastructure running in AWS (such as Amazon EC2 instances and load balancers). It can route users to infrastructure outside of AWS.

Another feature of Route 53 is the ability to manage the DNS records for domain names. You can register new domain names directly in Route 53. You can also transfer DNS records for existing domain names managed by other domain registrars. This enables you to manage all of your domain names within a single location.

In the previous module, you learned about Amazon CloudFront, a content delivery service. The following example describes how Route 53 and Amazon CloudFront work together to deliver content to customers.

Example: How Amazon Route 53 and Amazon CloudFront deliver content



Suppose that AnyCompany's application is running on several Amazon EC2 instances. These instances are in an Auto Scaling group that attaches to an Application Load Balancer.

Suppose that AnyCompany's application is running on several Amazon EC2 instances. These instances are in an Auto Scaling group that attaches to an Application Load Balancer.

1. 1

   A customer requests data from the application by going to AnyCompany's website.

2. 2

   Amazon Route 53 uses DNS resolution to identify AnyCompany.com's corresponding IP address, 192.0.2.0. This information is sent back to the customer.

3. 3

   The customer's request is sent to the nearest edge location through Amazon CloudFront.

4. 4

   Amazon CloudFront connects to the Application Load Balancer, which sends the incoming packet to an Amazon EC2 instance.

Module 5 Introduction

Learning objectives

In this module, you will learn how to:

- Summarize the basic concept of storage and databases.
- Describe the benefits of Amazon Elastic Block Store (Amazon EBS).
- Describe the benefits of Amazon Simple Storage Service (Amazon S3).
- Describe the benefits of Amazon Elastic File System (Amazon EFS).
- Summarize various storage solutions.
- Describe the benefits of Amazon Relational Database Service (Amazon RDS).
- Describe the benefits of Amazon DynamoDB.
- Summarize various database services

Instance Stores and Amazon Elastic Block Store (Amazon EBS)

When you're using Amazon EC2 to run your business applications, those applications need access to CPU, memory, network, and storage. EC2 instances give you access to all those different components, and right now, let's focus on the storage access. As applications run, they will oftentimes need access to block-level storage.

You can think of block-level storage as a place to store files. A file being a series of bytes that are stored in blocks on disc. When a file is updated, the whole series of blocks aren't all overwritten. Instead, it updates just the pieces that change. This makes it an efficient storage type when working with applications like databases, enterprise software, or file systems.

When you use your laptop or personal computer, you are accessing block-level storage. All block-level storage is, in this case, is your hard drive. EC2 instances have hard drives as well. And there are a few different types.

When you launch an EC2 instance, depending on the type of the EC2 instance you launched, it might provide you with local storage called instance store volumes. These volumes are physically attached to the host, your EC2 instances running on top of. And you can write to it just like a normal hard drive. The catch here is that since this volume is attached to the underlying physical host, if you stop or terminate your EC2 instance, all data written to the instance store volume will be deleted. The reason for this, is that

if you start your instance from a stop state, it's likely that EC2 instance will start up on another host. A host where that volume does not exist. Remember EC2 instances are virtual machines, and therefore the underlying host can change between stopping and starting an instance.

Because of this ephemeral or temporary nature of instance store volumes, they are useful in situations where you can lose the data being written to the drive. Such as temporary files, scratch data, and data that can be easily recreated without consequence.

All right, I'm telling you not to write important data to the drives that come with EC2 instances. I'm sure that sounds a bit scary because obviously you'll need a place to write data that persists outside of the life cycle of an EC2 instance. You don't want your entire database getting deleted every time you stop an EC2 instance. Don't worry, this is where a service called Amazon Elastic Block Store, or EBS, comes into play.

With EBS, you can create virtual hard drives, that we call EBS volumes, that you can attach to your EC2 instances. These are separate drives from the local instance store volumes, and they aren't tied directly to the host that you're EC2 is running on. This means, that the data that you write to an EBS volume can persist between stops and starts of an EC2 instance.

EBS volumes come in all different sizes and types. How this works, is you define the size, type and configurations of the volume you need. Provision the volume, and then attach it to your EC2 instance. From there, you can configure your application to write to the volume and you're good to go. If you stop and then start the EC2 instance, the data in the volume remains.

Since the use case for EBS volumes is to have a hard drive that is persistent, that your applications can write to, it's probably important that you back that data up. EBS allows you to take incremental backups of your data called snapshots. It's very important that you take regular snapshots of your EBS volumes This way, if a drive ever becomes corrupted, you haven't lost your data. And you can restore that data from a snapshot.

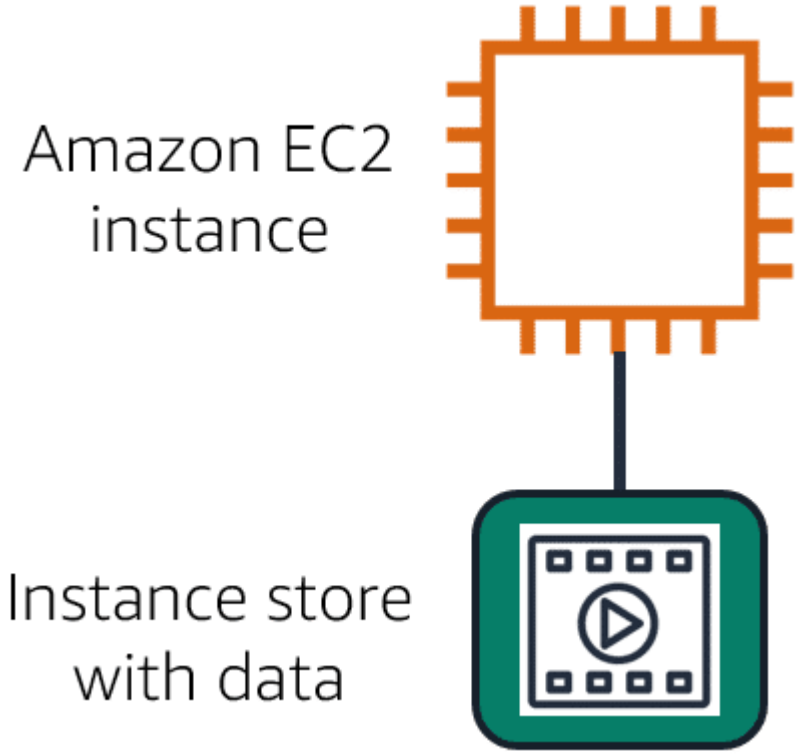
Instance stores

Block-level storage volumes behave like physical hard drives.

An **instance store**(opens in a new tab) provides temporary block-level storage for an Amazon EC2 instance. An instance store is disk storage that is physically attached to the host computer for an EC2 instance, and therefore has the same lifespan as the instance. When the instance is terminated, you lose any data in the instance store.

Step 1

An Amazon EC2 instance with an attached instance store is running.

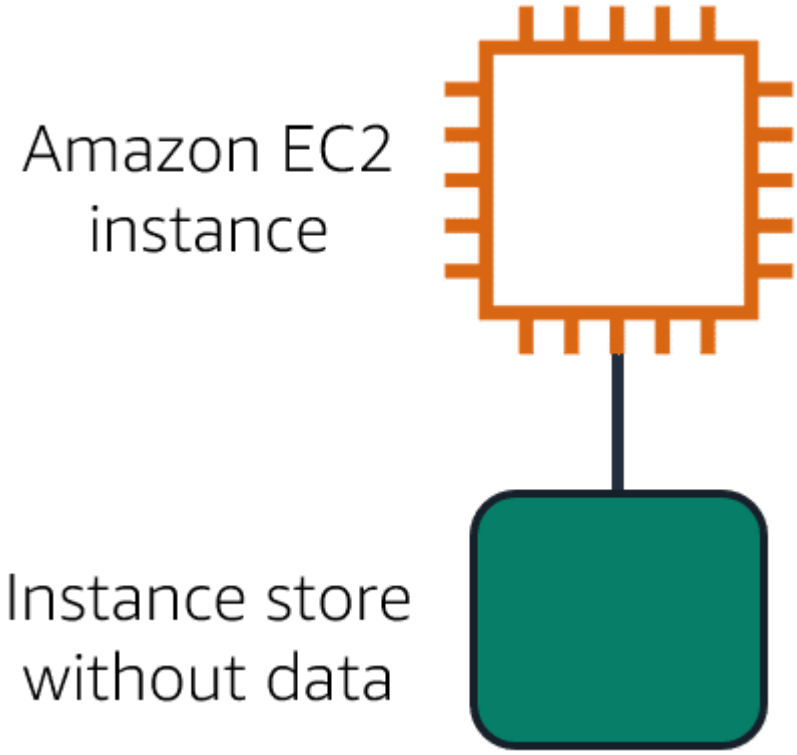


Step 2

The instance is stopped or terminated.

Step 3

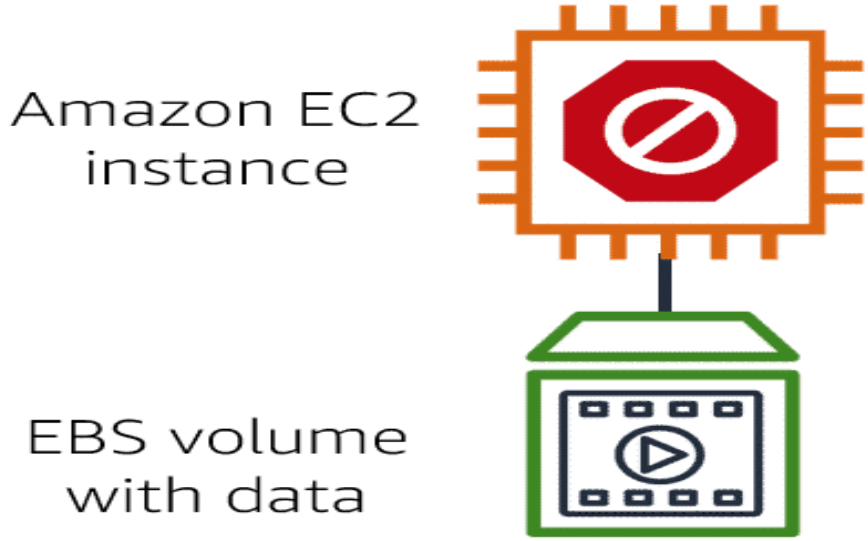
All data on the attached instance store is deleted.



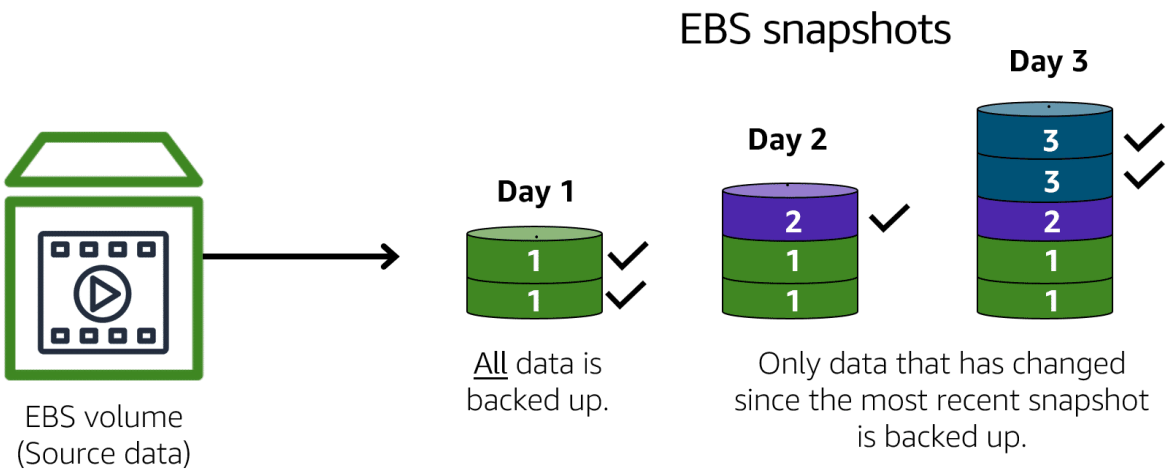
Amazon Elastic Block Store (Amazon EBS)(opens in a new tab) is a service that provides block-level storage volumes that you can use with Amazon EC2 instances. If you stop or terminate an Amazon EC2 instance, all the data on the attached EBS volume remains available.

To create an EBS volume, you define the configuration (such as volume size and type) and provision it. After you create an EBS volume, it can attach to an Amazon EC2 instance.

Because EBS volumes are for data that needs to persist, it's important to back up the data. You can take incremental backups of EBS volumes by creating Amazon EBS snapshots.



Amazon EBS snapshots



Incremental backups of EBS volumes with Amazon EBS snapshots. On Day 1, two volumes are backed up. Day 2 adds one new volume and the new volume is backed up. Day 3 adds two more volumes for a total of five volumes. Only the two new volumes are backed up.

An **EBS snapshot**(opens in a new tab) is an incremental backup. This means that the first backup taken of a volume copies all the data. For subsequent backups, only the blocks of data that have changed since the most recent snapshot are saved.

Incremental backups are different from full backups, in which all the data in a storage volume copies each time a backup occurs. The full backup includes data that has not changed since the most recent backup.

Amazon Simple Storage Service (Amazon S3)

Welcome to this video on Amazon Simple Storage Service, or Amazon S3. From the name, you've probably guessed that it's a storage service and it's, well, simple. Most businesses have data that needs to be stored somewhere. For the coffee shop, this could be receipts, images, Excel spreadsheets, employee training videos, and even text files, among others. Storing these files is where Amazon S3 comes in handy because it's a data store that allows you to store and retrieve a virtually unlimited amount of data at any scale.

Data is stored as objects, but instead of storing them in a file directory, you store them in what we call buckets. Think of a file sitting on your hard drive. That's an object. And think of a file directory. That's the bucket. The maximum object size that you can upload is 5 terabytes. You can also version objects to protect them from accidental deletion. What this means is that you always retain the previous versions of an object, like a paper trail. You can even create multiple buckets and store them across different classes or tiers of data. You can then create permissions to limit who can see or even access objects.

And you can even stage data between different tiers. These tiers offer mechanisms for different storage use cases, such as data that needs to be accessed frequently compared to, audit data that needs to be retained for several years. Let's go through the notable ones.

The first storage class is called S3 Standard and comes with 11 nines of durability. That means an object stored in S3 Standard has a 99.999999999 percent – that's a lot of nines – probability that it will remain intact after a period of 1 year. That's pretty high. Furthermore, data is stored in such a way that AWS can sustain the concurrent loss of data in two separate storage facilities. This is because data is stored in at least three facilities. So multiple copies reside across locations. Another useful way to use Amazon S3 is static website hosting, where a static website is a collection of HTML files and each file is akin to a physical page of the actual site. You can do this by simply uploading all your HTML, static web assets, and so forth into a bucket and then checking a box to host it as a static website. You can then enter the bucket's URL and bam! Instant website. And we say static, but that doesn't mean you can't have animations and moving parts to your website. That’s a pretty awesome way to start up that coffee blog.

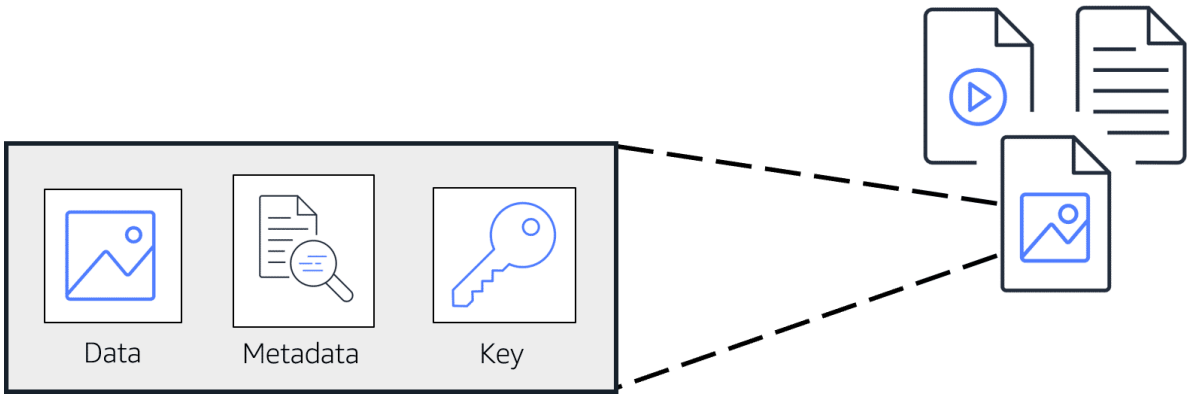
The next storage class is called S3 Standard-Infrequent Access or S3 Standard-IA. It’s used for data that is accessed less frequently but requires rapid access when needed. This means it's a perfect place to store backups, disaster recovery files, or any object that requires long-term storage.

Another storage class or tier lends itself to that example we had earlier about audit data. Say, we need to retain data for several years for auditing purposes. And we don't need it to be retrieved very rapidly. Well, then you can use S3 Glacier Flexible Retrieval to archive that data. To use S3 Glacier Flexible Retrieval, you can simply move data to it, or you can create vaults and then populate them with archives. And if you have compliance requirements around retaining data for, say, a certain period of time, you can employ an S3 Glacier vault lock policy and lock your vault. You can specify controls such as write once, read many, or WORM, in a vault lock policy and lock the policy from future edits. Once locked, the policy can no longer be changed. You also have three options for retrieval. These range from minutes to hours, and you have the option of uploading directly to S3 Glacier Flexible Retrieval or using S3 Lifecycle policies.

In fact, I don't think we mentioned lifecycle policies up until this point. But they are policies you can create that can move data automatically between tiers. For example, say we need to keep an object in S3 Standard for 90 days, and then we want to move it to S3 Standard-IA for the next 30 days. Then after 120 days total, we want to move it to S3 Glacier Flexible Retrieval. With Lifecycle policies, you create that configuration without changing your application code, and it will perform those moves for you automatically. It's another example of a managed AWS service, helping take that burden off you, so you can focus more on your business needs.

There are also other storage classes like S3 One Zone-Infrequent Access, S3 Glacier Instant Retrieval, and S3 Glacier Deep Archive that you can use. Happy storing.

Object storage



In **object storage**, each object consists of data, metadata, and a key.

The data might be an image, video, text document, or any other type of file. Metadata contains information about what the data is, how it is used, the object size, and so on. An object's key is its unique identifier.

Recall that when you modify a file in block storage, only the pieces that are changed are updated. When a file in object storage is modified, the entire object is updated.

Amazon Simple Storage Service (Amazon S3)

Amazon Simple Storage Service (Amazon S3)(opens in a new tab) is a service that provides object-level storage. Amazon S3 stores data as objects in buckets.

You can upload any type of file to Amazon S3, such as images, videos, text files, and so on. For example, you might use Amazon S3 to store backup files, media files for a website, or archived documents. Amazon S3 offers unlimited storage space. The maximum file size for an object in Amazon S3 is 5 TB.

When you upload a file to Amazon S3, you can set permissions to control visibility and access to it. You can also use the Amazon S3 versioning feature to track changes to your objects over time.

Amazon S3 storage classes

With Amazon S3, you pay only for what you use. You can choose from a range of storage classes(opens in a new tab) to select a fit for your business and cost needs. When selecting an Amazon S3 storage class, consider these two factors:

- How often you plan to retrieve your data
- How available you need your data to be

To learn more about Amazon S3 storage classes, expand each of the following eight categories.

S3 Standard

- Designed for frequently accessed data
- Stores data in a minimum of three Availability Zones

Amazon S3 Standard provides high availability for objects. This makes it a good choice for a wide range of use cases, such as websites, content distribution, and data analytics. Amazon S3 Standard has a higher cost than other storage classes intended for infrequently accessed data and archival storage.

S3 Standard-Infrequent Access (S3 Standard-IA)

- Ideal for infrequently accessed data
- Similar to Amazon S3 Standard but has a lower storage price and higher retrieval price

Amazon S3 Standard-IA is ideal for data infrequently accessed but requires high availability when needed. Both Amazon S3 Standard and Amazon S3 Standard-IA store data in a minimum of three Availability Zones. Amazon S3 Standard-IA provides the same level of availability as Amazon S3 Standard but with a lower storage price and a higher retrieval price.

S3 One Zone-Infrequent Access (S3 One Zone-IA)

- Stores data in a single Availability Zone
- Has a lower storage price than Amazon S3 Standard-IA

Compared to S3 Standard and S3 Standard-IA, which store data in a minimum of three Availability Zones, S3 One Zone-IA stores data in a single Availability Zone. This makes it a good storage class to consider if the following conditions apply:

- You want to save costs on storage.
- You can easily reproduce your data in the event of an Availability Zone failure.

S3 Intelligent-Tiering

- Ideal for data with unknown or changing access patterns
- Requires a small monthly monitoring and automation fee per object

In the S3 Intelligent-Tiering storage class, Amazon S3 monitors objects' access patterns. If you haven't accessed an object for 30 consecutive days, Amazon S3 automatically moves it to the infrequent access tier, S3 Standard-IA. If you access an object in the infrequent access tier, Amazon S3 automatically moves it to the frequent access tier, S3 Standard.

S3 Glacier Instant Retrieval

- Works well for archived data that requires immediate access
- Can retrieve objects within a few milliseconds

When you decide between the options for archival storage, consider how quickly you must retrieve the archived objects. You can retrieve objects stored in the S3 Glacier Instant Retrieval storage class within milliseconds, with the same performance as S3 Standard.

S3 Glacier Flexible Retrieval

- Low-cost storage designed for data archiving

- Able to retrieve objects within a few minutes to hours

S3 Glacier Flexible Retrieval is a low-cost storage class that is ideal for data archiving. For example, you might use this storage class to store archived customer records or older photos and video files. You can retrieve your data from S3 Glacier Flexible Retrieval from 1 minute to 12 hours.

S3 Glacier Deep Archive

- Lowest-cost object storage class ideal for archiving
- Able to retrieve objects within 12 hours

S3 Deep Archive supports long-term retention and digital preservation for data that might be accessed once or twice in a year. This storage class is the lowest-cost storage in the AWS Cloud, with data retrieval from 12 to 48 hours. All objects from this storage class are replicated and stored across at least three geographically dispersed Availability Zones.

S3 Outposts

- Creates S3 buckets on Amazon S3 Outposts
- Makes it easier to retrieve, store, and access data on AWS Outposts

Amazon S3 Outposts delivers object storage to your on-premises AWS Outposts environment. Amazon S3 Outposts is designed to store data durably and redundantly across multiple devices and servers on your Outposts. It works well for workloads with local data residency requirements that must satisfy demanding performance needs by keeping data close to on-premises applications.

Comparing Amazon EBS and Amazon S3

AWS Cloud Practitioners, welcome to the clash of the storage class! In the block storage corner, weighing in at sizes up to 16 tebibytes each, with a unique ability to survive the termination of their Amazon EC2 instances, they are solid state, they are spinning platters, they are Amazon Elastic Block Storage!

In the regional object storage corner, weighing in at unlimited storage, with individual objects at 5,000 gigabytes in size, they specialize in write once/read many, they are 99 .999 999 999% durable, they are Amazon Simple Storage Service!

Head-to-head, each storage class boasts the best dynamically distributed design for different storage demands. Which storage class will ultimately be victorious in this thunderdome slugfest? To understand who wins, you need to clarify a use case.

Round one. Let's say you're running a photo analysis website where users upload a photo of themselves, and your application finds the animals that look just like them. You have potentially millions of animal pictures that all need to be indexed and possibly viewed by thousands of people at once. This is the perfect use case for S3. S3 is already web enabled. Every object already has a URL that you can control access rights to who can see or manage the image. It's regionally distributed, which means that it has 11 nines of durability, so no need to worry about backup strategies. S3 is your backup strategy. Plus the cost savings is substantial overrunning the same storage load on EBS. With the additional advantage of being serverless, no Amazon EC2 instances are needed. Sounds like S3 is the judge's winner here for this round.

But wait, round two, you have an 80-gigabyte video file that you're making edit corrections on. To know the best storage class here, we need to understand the difference between object storage and block storage. Object storage treats any file as a complete, discreet object. Now this is great for documents, and images, and video files that get uploaded and consumed as entire objects, but every time there's a change to the object, you must re-upload the entire file. There are no delta updates. Block storage breaks those files down to small component parts or blocks. This means, for that 80-gigabyte file, when you make an edit to one scene in the film and save that change, the engine only updates the blocks where those bits live. If you're making a bunch of micro edits, using EBS, elastic block storage, is the perfect use case. If you were using S3, every time you saved the changes, the system would have to upload all 80 gigabytes, the whole thing, every time. EBS clearly wins round two.

This means, if you are using complete objects or only occasional changes, S3 is victorious. If you are doing complex read, write, change functions, then, absolutely, EBS is your knockout winner. Your winner depends on your individual workload. Each service is the right service for specific needs. Once you understand what you need, you will know which service is your champion!

Amazon Elastic File System (Amazon EFS)

Next up on the list of storage services is Amazon Elastic File System, or what we call EFS. EFS is a managed file system. It's extremely common for businesses to have shared file systems across their applications. For example, you might have multiple servers running analytics on large amounts of data being stored in a shared file system. This data traditionally has been hosted on premises. In this on-premises data center, you would have to ensure that the storage you have can keep up with the amount of data that you are storing. Make sure backups are taken, and that the data is stored redundantly as well as manage all of the servers hosting that data.

Luckily with AWS, you don't need to worry about buying all of that hardware and keeping the whole file system running from an operational standpoint. With EFS, you can keep existing file systems in place but let AWS do all the heavy lifting of the scaling and the replication. EFS allows you to have multiple instances accessing the data in EFS at the same time. It scales up and down as needed without you needing to do anything to make that scaling happen. Super nice, right? Well, you might be thinking, Amazon EBS also lets me store files that I can access from EC2 instances. What exactly is the difference here?

[Blaine] AWS Cloud Practitioners, welcome back!

[Morgan] All right, we don't need to do all of that. The answer is really simple. Amazon EBS volumes attach to EC2 instances and are an Availability Zone-level resource. In order to attach EC2 to EBS, you need to be in the same AZ. You can save files on it. You

can also run a database on top of it. Or store applications on it. It's a hard drive. If you provision a two terabyte EBS volume and fill it up, it doesn't automatically scale to give you more storage. So that's EBS. Amazon EFS can have multiple instances reading and writing from it at the same time.

But it isn't just a blank hard drive that you can write to. It is a true file system for Linux. It is also a regional resource. Meaning any EC2 instance in the Region can write to the EFS file system. As you write more data to EFS, it automatically scales. No need to provision any more volumes.

File storage

In **file storage**, multiple clients (such as users, applications, servers, and so on) can access data that is stored in shared file folders. In this approach, a storage server uses block storage with a local file system to organize files. Clients access data through file paths.

Compared to block storage and object storage, file storage is ideal for use cases in which a large number of services and resources need to access the same data at the same time.

Amazon Elastic File System (Amazon EFS)(opens in a new tab) is a scalable file system used with AWS Cloud services and on-premises resources. As you add and remove files, Amazon EFS grows and shrinks automatically. It can scale on demand to petabytes without disrupting applications.

Comparing Amazon EBS and Amazon EFS

Select each of the cards below to review a comparison of Amazon EBS and Amazon EFS

Amazon EBS

An Amazon EBS volume stores data in a **single** Availability Zone.

To attach an Amazon EC2 instance to an EBS volume, both the Amazon EC2 instance and the EBS volume must reside within the same Availability Zone.

Amazon EFS

Amazon EFS is a regional service. It stores data in and across **multiple** Availability Zones.

The duplicate storage enables you to access data concurrently from all the Availability Zones in the Region where a file system is located. Additionally, on-premises servers can access Amazon EFS using AWS Direct Connect.

Amazon Relational Database Service (Amazon RDS)

So you're storing data about your coffee shop in various systems. But you're finding that you need to maintain relationships between various types of data. And by relationships, I mean, if say, a customer orders the same drink multiple times, maybe you want to offer them a promotional discount on their next purchase. And you need a way to keep track of this relationship somewhere. In this case, it's best to use a relational database management system, or RDBMS. Essentially, It means we store data in a way such that it relates to other pieces of data.

For example, if we had a customer entry or record, we store that in a customer table. We then could have an entry for the physical address, which we store on a corresponding address table. We then relate the two via a common attribute and can query the data that is housed in both tables.

The most common way to query the data is by writing queries in SQL. And this runs on a variety of database systems. Speaking of database systems, what are some of the more well known ones that AWS supports? Well, there's MySQL, PostgreSQL, Oracle, Microsoft SQL Server, and many more. If you have an on-premises environment, you're probably running one of those and they're most likely housed in your data center.

But is there a way to easily move them to the cloud? Well, the simple answer is yes, you can do what we call a Lift-and-Shift, and migrate your database to run on Amazon EC2. This means you have control over the same variables you do, in your on-premises environment, such as OS, memory, CPU, storage capacity, and so forth. It's a quick entry to the cloud, and you can migrate them using standard practices or using something like Database Migration Service, which we'll cover in a later video.

The other option for running your databases in the cloud is to use a more managed service called Amazon Relational Database Service, or RDS. It supports all the major database engines, like the ones we mentioned earlier, but this service comes with added benefits. These include automated patching, backups, redundancy, failover, disaster recovery, all of which you normally have to manage for yourself. This makes it an extremely attractive option to AWS customers, as it allows you to focus on business problems and not maintaining databases. Which if you're a database admin, can be pretty time consuming and difficult.

So how do we make it even easier for you to run database workloads on the cloud? Well, we go one further and have them migrate or deploy to Amazon Aurora. It's our most managed relational database option. It comes in two forms, MySQL and PostgreSQL. And is priced is 1/10th the cost of commercial grade databases. That's a pretty cost effective database. The other benefits are things like your data is replicated across facilities, so you have six copies at any given time. You can also deploy up to 15 read replicas, so you can offload your reads and scale performance. Additionally, there's continuous backups to S3, so you always have a backup ready to restore. You also get point in time recovery, so you can recover data from a specific period.

And there you have it, relational databases in a nutshell.

Relational databases

In a **relational database**, data is stored in a way that relates it to other pieces of data.

An example of a relational database might be the coffee shop's inventory management system. Each record in the database would include data for a single item, such as product name, size, price, and so on.

Relational databases use **structured query language (SQL)** to store and query data. This approach allows data to be stored in an easily understandable, consistent, and scalable way. For example, the coffee shop owners can write a SQL query to identify all the customers whose most frequently purchased drink is a medium latte.

Example of data in a relational database:

ID	Product name	Size	Price
1	Medium roast ground coffee	12 oz.	$5.30
2	Dark roast ground coffee	20 oz.	$9.27

Amazon Relational Database Service

Amazon Relational Database Service (Amazon RDS)(opens in a new tab) is a service that enables you to run relational databases in the AWS Cloud.

Amazon RDS is a managed service that automates tasks such as hardware provisioning, database setup, patching, and backups. With these capabilities, you can spend less time completing administrative tasks and more time using data to innovate your applications. You can integrate Amazon RDS with other services to

fulfill your business and operational needs, such as using AWS Lambda to query your database from a serverless application.

Amazon RDS provides a number of different security options. Many Amazon RDS database engines offer encryption at rest (protecting data while it is stored) and encryption in transit (protecting data while it is being sent and received)

Amazon RDS database engines

Amazon RDS is available on six database engines, which optimize for memory, performance, or input/output (I/O). Supported database engines include:

- Amazon Aurora
- PostgreSQL
- MySQL
- MariaDB
- Oracle Database
- Microsoft SQL Server

Amazon Aurora

Amazon Aurora(opens in a new tab) is an enterprise-class relational database. It is compatible with MySQL and PostgreSQL relational databases. It is up to five times faster than standard MySQL databases and up to three times faster than standard PostgreSQL databases.

Amazon Aurora helps to reduce your database costs by reducing unnecessary input/output (I/O) operations, while ensuring that your database resources remain reliable and available.

Consider Amazon Aurora if your workloads require high availability. It replicates six copies of your data across three Availability Zones and continuously backs up your data to Amazon S3.

Amazon DynamoDB

Let's talk about Amazon DynamoDB. At its most basic level, DynamoDB is a database. It's a serverless database, meaning you don't need to manage the underlying instances or infrastructure powering it.

With DynamoDB, you create tables. A DynamoDB table, is just a place where you can store and query data. Data is organized into items, and items have attributes. Attributes are just different features of your data. If you have one item in your table, or 2 million

items in your table, DynamoDB manages the underlying storage for you. And you don't need to worry about the scaling of the system, up or down.

DynamoDB stores this data redundantly across availability zones and mirrors the data across multiple drives under the hood for you. This makes the burden of operating a highly available database, much lower.

DynamoDB, beyond being massively scalable, is also highly performant. DynamoDB has a millisecond response time. And when you have applications with potentially millions of users, having scalability and reliable lightning fast response times is important.

Now, DynamoDB isn't a normal database. In the sense that it doesn't use the very widely used query language, sequel, or SQL. Relational databases, like a standard MySQL Database, require that you have a well defined schema, in place. That might consist of one, or many tables that might relate to each other. You then use SQL to query the data.

This works great for a lot of use cases, and has been the standard type of database historically. However, these types of rigid SQL databases, can have performance and scaling issues when under stress. The rigid schema also makes it so that you cannot have any variation in the types of data that you store in a table. So, it might not be the best fit for a dataset that is a little bit less rigid, and is being accessed at a very high rate.

This is where non-relational, or NoSQL, databases come in. DynamoDB is a non-relational database. Non-relational databases tend to have simple flexible schemas, not complex rigid schemas, laying out multiple tables that all relate to each other.

With DynamoDB, you can add and remove attributes from items in the table, at any time. Not every item in the table has to have the same attributes. This is great for datasets that have some variation from item to item. Because of this flexibility, you cannot run complex SQL queries on it. Instead, you would write queries based on a small subset of attributes that are designated as keys.

Because of this, the queries that you run are non-relational databases tend to be simpler, and focus on a collection of items from one table, not queries than span multiple tables. This query pattern, along with other factors, including the way that the underlying system is designed, allows DynamoDB to be very quick in response time, and highly scalable.

So, things to remember: DynamoDB is a non-relational, NoSQL database. It is purpose built. Meaning it has specific use cases, and it isn't the best fit for every workload out there. It has millisecond response time. It's fully managed, and it's highly scalable. One awesome example is on Prime Day in 2019, across the 48 hours of Prime Day, there were 7.11 trillion calls to the DynamoDB API, peaking at 45.4 million requests per second. That's insanely scalable, all without the underlying database management. That's pretty cool.

Nonrelational databases

In a **nonrelational database**, you create tables. A table is a place where you can store and query data.

Nonrelational databases are sometimes referred to as “NoSQL databases” because they use structures other than rows and columns to organize data. One type of structural approach for nonrelational databases is key-value pairs. With key-value pairs, data is organized into items (keys), and items have attributes (values). You can think of attributes as being different features of your data.

In a key-value database, you can add or remove attributes from items in the table at any time. Additionally, not every item in the table has to have the same attributes.

Example of data in a nonrelational database:

Key	Value
1	**Name**: John Doe **Address**: 123 Any Street **Favorite drink**: Medium latte
2	**Name**: Mary Major **Address**: 100 Main Street **Birthday**: July 5, 1994

Amazon DynamoDB

Amazon DynamoDB(opens in a new tab) is a key-value database service. It delivers single-digit millisecond performance at any scale.

To learn about features of DynamoDB, select each flashcard by choosing them.

Serverless

DynamoDB is serverless, which means that you do not have to provision, patch, or manage servers.

You also do not have to install, maintain, or operate software.

Automatic scaling

As the size of your database shrinks or grows, DynamoDB automatically scales to adjust for changes in capacity while maintaining consistent performance.

This makes it a suitable choice for use cases that require high performance while scaling.

AWS Cloud Practitioners, welcome back to the championship chase of the database! In the relational corner, engineered to remove undifferentiated heavy lifting from your database administrators with automatic high availability and recovery provided. You control the data, you control the schema, you control the network. You are running Amazon RDS. Yes, Yeah.

The NoSQL corner, using a key value pair that requires no advanced schema, able to operate as a global database at the touch of a button. It has massive throughput. It has petabyte scale potential. It has granular API access. It is Amazon DynamoDB.

Head to head. Each database class is engineered to exactly enhance exciting existential environments you envision. Which database will ultimately be victorious in this new world knock out night fight? Once again, the winner will depend on your use case.

Round one, Relational databases have been around since the moment businesses started using computers. Being able to build complex analysis of data spread across multiple tables, is the strength of any relational system. In this round, you have a sales supply chain management system that you have to analyze for weak spots. Using RDS is the clear winner here because it's built for business analytics, because you need complex relational joins. Round one easily goes to RDS.

Round two, the use case, pretty much anything else. Now that sounds weird, but despite what your standalone legacy database vendor would have you believe, most of what people use expensive relational databases for, has nothing to do with complex relationships. In fact, a lot of what people put into these databases ends up just being look-up tables.

For this round, imagine you have an employee contact list: names, phone numbers, emails, employee IDs. Well, this is all single table territory. I could use a relational database for this, but the things that make relational databases great, all of that complex functionality, creates overhead and lag and expense if you're not actually using it. This is where non-relational databases, Dynamo DB, delivers the knockout punch. By eliminating all the overhead, DynamoDB allows you to build powerful, incredibly fast databases where you don't need complex joint functionality. DynamoDB comes out the undisputed champion.

Once again, the winner depends on your individual workload. Each service is the right service for specific needs. And once you understand what you need, you will know again, which service is your champion.

Amazon Redshift

We just spent a lot of time discussing the kinds of workflow that require fast, reliable, current data. Databases that can handle 1,000s of transactions per second, storage that is highly available and massively durable. But sometimes, we have a business need that goes outside what is happening right now to what did happen. This data analysis is the realm of a whole

different class of databases. Sure, you could use the one size fits all model of a single database for everything, but modern databases that are engineered for high speed, real time ingestion, and queries may not be the best fit.

Let me explain. In order to handle the velocity of real time read/write functionality, most relational databases tend to function fabulously at certain capacities. How much content it actually stores. The problem with historical analytics, data that answers questions like, "Show me how production has improved since we started", is the data collection never stops. In fact, with modern telemetry and the explosion of IoT, the volume of data will overwhelm even the beefiest traditional relational database.

It gets worse. Not just the volume, but the variety of data can be a problem. You want to run business intelligence or BI projects against data coming from different data stores like your inventory, your financial, and your retail sales systems? A single query against multiple databases sounds nice, but traditional databases don't handle them easily.

Once data becomes too complex to handle with traditional relational databases, you've entered the world of data warehouses. Data warehouses are engineered specifically for this kind of big data, where you are looking at historical analytics as opposed to operational analysis.

Now, let's be clear. Historical may be as soon as: show me last hour's sales numbers across all the stores. The key thing is, the data is now set. We're not selling any more from the last hour because that is now in the past. Compare that question to, "How many bags of coffee do we still have in our inventory right now?" Which could be changing as we speak. As long as your business question is looking backwards at all, then a data warehouse is the right solution for that line of business intelligence.

Now there are many data warehouse solutions out on the market. If you already have a favorite one, running it on AWS is just a matter of getting the data transferred. But beyond that, there may still be a lot of undifferentiated heavy lifting that goes into keeping a data warehouse tuned, resilient, and continuously scaling. Wouldn't it be nice if your data warehouse team could focus on the data instead of the unavoidable care and feeding of the engine?

Introducing Amazon Redshift. This is data warehousing as a service. It's massively scalable. Redshift nodes in multiple petabyte sizes is very common. In fact, in cooperation with Amazon Redshift Spectrum, you can directly run a single SQL query against exabytes of unstructured data running in data lakes.

But it's more than just being able to handle massively larger data sets. Redshift uses a variety of innovations that allow you to achieve up to 10 times higher performance than traditional databases, when it comes to these kinds of business intelligence workloads.

I'm not gonna go into details of how the magic works. We have whole classes that you or your data teams can take that explain how it is built, and why it can return such improved results. The key for you is to understand that when you need big data BI solutions, Redshift allows you to get started with a single API call. Less time waiting for results, more time getting answers.

Amazon Redshift

Amazon Redshift(opens in a new tab) is a data warehousing service that you can use for big data analytics. It offers the ability to collect data from many sources and helps you to understand relationships and trends across your data.

AWS Database Migration Service

We've been talking about databases and the various database options on AWS. But what happens if you have a database that's on-premises or in the cloud already? Does that mean you have to start from scratch or does AWS have a magical way to help you migrate your existing database?

Thankfully, AWS offers a service called Amazon Database Magic. I mean, Amazon Database Migration Service, or DMS, to help customers do just that. DMS helps customers migrate existing databases onto AWS in a secure and easy fashion. You essentially migrate data between a source and a target database. The best part is that the source database remains fully operational during the migration, minimizing downtime to applications that rely on that database. Better yet is that the source and target databases don't have to be of the same type.

But let's start with databases that are of the same type. These migrations are known as homogenous and can be from MySQL to Amazon RDS for MySQL, Microsoft SQL Server to Amazon RDS for SQL Server or even Oracle to Amazon RDS for Oracle. The process is fairly straightforward since schema structures, data types, and database code is compatible between source and target.

As I mentioned, the source database can be located on-premises, running on Amazon EC2 Instances, or it can be an Amazon RDS database. The target itself can be a database in Amazon EC2 or Amazon RDS. In this case, you create a migration task with connections to the source and target databases. Then start the migration with a click of the button. AWS Database Migration Service takes care of the rest.

The second type of migration occurs when source and target databases are of different types. This is called heterogeneous migrations, and it's a two-step process. Since the schema structures, data types, and database code are different between source and target, we first need to convert them using the AWS Schema Conversion Tool. This will convert the source schema and code to match that of the target database. The next step is then to use DMS to migrate data from the source database to the target database.

But these are not the only use cases for DMS. Others include development and test database migrations, database consolidation, and even continuous database replication. Development and test migration is when you want to develop this to test against production data, but without affecting production users. In this case, you use DMS to migrate a copy of your production database to your dev or test environments, either once-off or continuously.

Database consolidation is when you have several databases and want to consolidate them into one central database.

Finally, continuous replication is when you use DMS to perform continuous data replication. This could be for disaster recovery or because of geographic separation.

If you'd like to learn more about any of these, please check out our resources section. And there you have it, folks, DMS in a nutshell.

AWS Database Migration Service (AWS DMS)

AWS Database Migration Service (AWS DMS)(opens in a new tab) enables you to migrate relational databases, nonrelational databases, and other types of data stores.

With AWS DMS, you move data between a source database and a target database. The source and target databases(opens in a new tab) can be of the same type or different types. During the migration, your source database remains operational, reducing downtime for any applications that rely on the database.

For example, suppose that you have a MySQL database that is stored on premises in an Amazon EC2 instance or in Amazon RDS. Consider the MySQL database to be your source database. Using AWS DMS, you could migrate your data to a target database, such as an Amazon Aurora database.

Other use cases for AWS DMS

To review other use cases for AWS DMS, select each of the following flashcards.

-Development and test database migrations

Enabling developers to test applications against production data without affecting production users

-Database consolidation

Combining several databases into a single database

-Continuous replication

Sending ongoing copies of your data to other target sources instead of doing a one-time migration.

Additional Database Services

Amazon DocumentDB

Amazon DocumentDB(opens in a new tab) is a document database service that supports MongoDB workloads. (MongoDB is a document database program.)

Amazon Neptune

Amazon Neptune(opens in a new tab) is a graph database service.

You can use Amazon Neptune to build and run applications that work with highly connected datasets, such as recommendation engines, fraud detection, and knowledge graphs.

Amazon Quantum Ledger Database (Amazon QLDB)

Amazon Quantum Ledger Database (Amazon QLDB)(opens in a new tab) is a ledger database service.

You can use Amazon QLDB to review a complete history of all the changes that have been made to your application data.

Amazon Managed Blockchain

Amazon Managed Blockchain(opens in a new tab) is a service that you can use to create and manage blockchain networks with open-source frameworks.

Blockchain is a distributed ledger system that lets multiple parties run transactions and share data without a central authority.

Amazon ElastiCache

Amazon ElastiCache(opens in a new tab) is a service that adds caching layers on top of your databases to help improve the read times of common requests.

It supports two types of data stores: Redis and Memcached.

Amazon DynamoDB Accelerator

Amazon DynamoDB Accelerator (DAX)(opens in a new tab) is an in-memory cache for DynamoDB.

It helps improve response times from single-digit milliseconds to microseconds.

Module 6 Introduction

Looks like we're getting deeper into our AWS account. Things are running well. And I want to let you in on the security measures we have in place. By this, I mean, we have to describe the various security mechanisms we offer on the AWS Cloud, like the shared responsibility model.

With the shared responsibility model, AWS controls security of the cloud and customers control security in the cloud. We, as AWS, control the data centers, security of our services, and all the layers in this section. The next part are the workloads that AWS customers run in the cloud and those are the customer's responsibility to secure. It's something we share with customers to ensure security in the cloud.

Let's take a look at the various other security services, mechanisms, and features that AWS has to offer in this module. Stay tuned

AWS Shared Responsibility Model

When it comes to securing your business on AWS, it's important to ask the question: Who is ultimately responsible for the security? Is it A: You, the customer? or B: AWS? And the correct answer is: yes. Both. Both are ultimately responsible for making sure that you are secure.

Now, if there's any security experts watching this right now you're probably shaking your head saying you can't have two different entities with the ultimate responsibility over a single object. That's not security, that's wishful thinking. At AWS, we agree completely. But for us, we don't look at your environment as a single object. Instead we see it as a collection of parts that build on each other. AWS is responsible for the security of some of the objects. Responsible 100% for those. The others, you are responsible 100% for their security. This is what's known as the shared responsibility model.

It's no different than securing a house. The builder constructed the house with four walls and a door. It's their responsibility to make sure the walls are strong and the doors are solid. It's your responsibility, the homeowner, to close and lock the doors.

It really is that simple on AWS as well. Take EC2, for instance. EC2 lives in a physical building, a data center that must be secured. It has a network and a hypervisor that supports your instances and their individual operating systems. On top of that operating system, you have your application, and that supports your data. So for EC2 and every service AWS offers, there's a similar stack of parts that build on top of each other. AWS is 100% responsible for some, you are responsible for the others.

So, starting with the physical layer. This is iron and concrete and fences and security guards. Someone has to own the concrete. Someone has to staff the physical perimeter, 24/7. This is AWS. On top of the physical layer we have our network and our hypervisor. Now, I'm not gonna go into details on how this is all secured, but basically we have reinvented those technologies to make them faster, stronger, tamper-proof.

But you don't have to just take our word for it. We have numerous third party auditors who have gone through the code and the way we build our infrastructure, and can provide the right documentation you need for your security compliance structures. Now, on top of all that, on EC2, you now get to pick what operating system you want to run.

This is the magic dividing line that separates our responsibility. AWS' responsibility and your responsibility. This is your operating system. You're 100% in charge of this. AWS does not have any backdoor into your system here. You and you alone have the only encryption key to log onto the root of this OS or to create any user accounts there. I mean, no more than a construction company would keep copies of your front door key, AWS cannot enter your operating system. And here's a hint. If someone from AWS calls and asks you for your OS key, it is not AWS.

Now that means your operations team is 100% responsible for keeping the operating system patched. If AWS discovers there are some new vulnerabilities in your version of Windows, let's say, we can certainly notify your account owner but we cannot deploy a patch. This is a really good thing for your security. This means no one can deploy anything that might break your system without your team being the ones that do it. Now, on top of that OS, you can run whatever applications you want. You own them. You maintain them.

Which takes us to the most important part of the stack, your data. Data. This is always your domain to control. And sometimes you might want to have your data open for everyone to see, like pictures on a retail website. Other times like banking or healthcare, yeah, not so much, not so much. AWS provides everyone with the tool set they need for their data to open it up to some authorized individuals, to everyone, to just a single person under specific conditions, or even lock it down so no one can access it. Plus, the ability to have ubiquitous encryption. That way even if you accidentally left your front door open, all anyone would see is unreadable encrypted content.

The AWS shared responsibility model is about making sure both sides understand exactly what tasks are ours. Basically, AWS is responsible for the security of the cloud and you are responsible for the security in the cloud. Together, you have an environment you can trust.

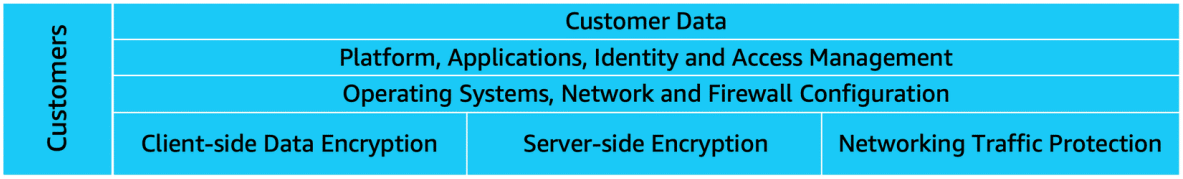
The AWS shared responsibility model

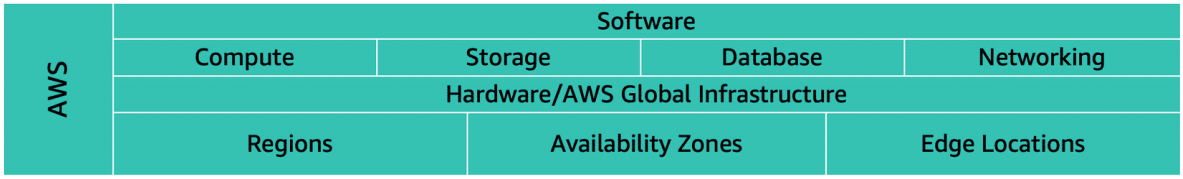
Throughout this course, you have learned about a variety of resources that you can create in the AWS Cloud. These resources include Amazon EC2 instances, Amazon S3 buckets, and Amazon RDS databases. Who is responsible for keeping these resources secure: you (the customer) or AWS?

The answer is both. The reason is that you do not treat your AWS environment as a single object. Rather, you treat the environment as a collection of parts that build upon each other. AWS is

responsible for some parts of your environment and you (the customer) are responsible for other parts. This concept is known as the **shared responsibility model**(opens in a new tab).

The shared responsibility model divides into customer responsibilities (commonly referred to as "security in the cloud") and AWS responsibilities (commonly referred to as "security of the cloud").





You can think of this model as being similar to the division of responsibilities between a homeowner and a homebuilder. The builder (AWS) is responsible for constructing your house and ensuring that it is solidly built. As the homeowner (the customer), it is your responsibility to secure everything in the house by ensuring that the doors are closed and locked.

To learn more about the shared responsibility model, expand each of the following two categories.

Customers: Security in the cloud

Customers are responsible for the security of everything that they create and put *in* the AWS Cloud.

When using AWS services, you, the customer, maintain complete control over your content. You are responsible for managing security requirements for your content, including which content you choose to store on AWS, which AWS services you use, and who has access to that content. You also control how access rights are granted, managed, and revoked.

The security steps that you take will depend on factors such as the services that you use, the complexity of your systems, and your company's specific operational and security needs. Steps include selecting, configuring, and patching the operating systems that will run on Amazon EC2 instances, configuring security groups, and managing user accounts.

AWS: Security of the cloud

AWS is responsible for security *of* the cloud.

AWS operates, manages, and controls the components at all layers of infrastructure. This includes areas such as the host operating system, the virtualization layer, and even the physical security of the data centers from which services operate.

AWS is responsible for protecting the global infrastructure that runs all of the services offered in the AWS Cloud. This infrastructure includes AWS Regions, Availability Zones, and edge locations.

AWS manages the security of the cloud, specifically the physical infrastructure that hosts your resources, which include:

- Physical security of data centers
- Hardware and software infrastructure
- Network infrastructure
- Virtualization infrastructure

Although you cannot visit AWS data centers to see this protection firsthand, AWS provides several reports from third-party auditors. These auditors have verified its compliance with a variety of computer security standards and regulations.

User Permissions and Access

In the coffee shop, every employee has an identity. They come into work in the morning and they'd log into the system to clock in, use the registers and manage the systems, running the coffee shop, day to day. We have the cash registers, the computers helping run the whole operation. Each person has unique access to these systems based on who they are.

If I have Rudy at the register taking orders and Blaine in the back, checking on the inventory levels on the computer, they have two different logins and two different sets of permissions. Rudy can run the cash register, but if he went to log into the inventory system, he wouldn't be allowed to do that.

You will want to scope your users permissions in AWS in a similar way. When you create an AWS account, you are given what is called the root account user. This root user is the owner of the AWS account and has permission to do anything they want inside of that account. This is like being the owner of the coffee shop.

In this situation, let's say I am the owner of the coffee shop. I can come into the shop, use my credentials to work the register, work the inventory system or any other system in the coffee shop. I cannot be restricted. With the AWS root user, you can access and control any resource in the account. You can spin up databases, EC2 instances, blockchain services, or literally whatever you want. Because that user is so powerful, we recommend that as soon as you create an AWS account and log in with your root user, you turn on multi-factor authentication, or MFA, to ensure that you need not only the email and password, but also a randomized token to log in.

That is great. But even with MFA turned on, in reality you don't want to use the root user for everything. I, as the coffee shop owner, don't give my level of access to all employees. Rudy on the cash register cannot access the inventory system, remember? You control access in a granular way by using the AWS service, AWS Identity and Access Management, or IAM.

In IAM, you can create IAM users. When you create an IAM user, by default, it has no permissions. The user can't even log into the AWS account at first, it has absolutely zero permissions. It can not launch an EC2 instance. It can not create an S3 bucket. Nothing. You have to explicitly give the user permission to do anything in that account. Remember, by default, all actions are denied. You have to explicitly allow any action done by any user. You give people access only to what they need and nothing else. This idea is called the least privilege principle.

The way that you grant or deny permission is to associate what is called an IAM policy to an IAM user. An IAM policy is a JSON document that describes what API calls a user can or cannot make. Let's look at this quick example. In this example, you can see we have a permission statement that has the effect as Allow, the action as s3:ListBucket. And the resource is a unique ID for an S3 bucket. So if I attach this policy to a user and that user could view the bucket "coffee_shop_reports" but perform no other action in this account. There were only two potential options for the effect on any policy. Either allow or deny. For action, you can list any AWS API call and for resource, you would list what AWS resource that specific API call is for. Now, as a business person, you wouldn't need to write these policies, but they are used all over in your AWS accounts.

One way to make it easier to manage your users and their permissions is to organize them into IAM groups. Groups are, well, they are groupings of users. You can attach a policy to a group and all of the users in that group will have those permissions. If you have a bunch of cashiers in the coffee shop, instead of individually granting them all access to the register. Instead, you can grant all cashiers access then just add each individual cashier to the group. Same idea with groups in IAM.

All right, so far with IAM, you have the root user, they can do anything. You have users which can be organized into groups. And you also have policies which are documents that describe permissions that you can then attach to users or groups. There is one other major identity in IAM, and it's called a role.

To understand the idea of roles, let's think about the coffee shop. As we know, Blaine works in the shop and depending on the staffing of the shop day to day, he might work the register or the inventory system or he might be the one cleaning up at the end of the day with access to no systems. I, as the owner, have the authority to assign these different roles to Blaine. His responsibilities and access are variable and change from day to day. Just because he worked on tracking inventory in the system yesterday, doesn't mean that he should be at any time. His role at work changes and is temporary in nature. The same type of idea exists in AWS. You can create identities in AWS that are called roles.

Roles have associated permissions that allow or deny specific actions. And these roles can be assumed for temporary amounts of time. It is similar to a user, but has no username and password. Instead, it is an identity that you can assume to gain access to temporary permissions. You use roles to temporarily grant access to AWS resources, to users, external identities, applications, and even other AWS services. When an identity assumes a role, it abandons all of the previous permissions that it has and it assumes the permissions of that role.
You can actually avoid creating IAM users for every person in your organization by federating users into your account. This means that they could use their regular corporate credentials to log into AWS by mapping their corporate identities to IAM roles. AWS IAM authentication and authorization as a service.

AWS Identity and Access Management (IAM)

AWS Identity and Access Management (IAM)(opens in a new tab) enables you to manage access to AWS services and resources securely.

IAM gives you the flexibility to configure access based on your company's specific operational and security needs. You do this by using a combination of IAM features, which are explored in detail in this lesson:

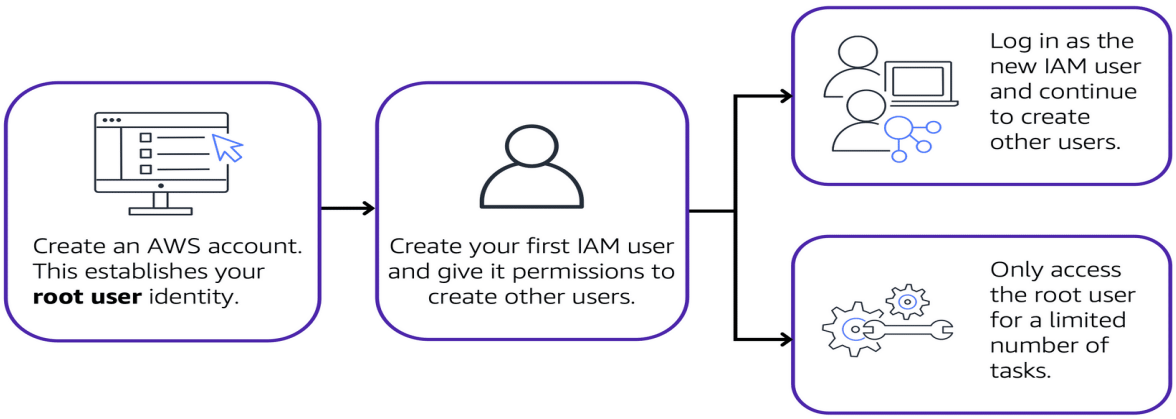
- IAM users, groups, and roles
- IAM policies
- Multi-factor authentication

You will also learn best practices for each of these features.

AWS account root user

When you first create an AWS account, you begin with an identity known as the **root user**(opens in a new tab).

The root user is accessed by signing in with the email address and password that you used to create your AWS account. You can think of the root user as being similar to the owner of the coffee shop. It has complete access to all the AWS services and resources in the account.



Best practice:

Do **not** use the root user for everyday tasks.

Instead, use the root user to create your first IAM user and assign it permissions to create other users.

Then, continue to create other IAM users, and access those identities for performing regular tasks throughout AWS. Only use the root user when you need to perform a limited number of tasks that are only available to the root user. Examples of these tasks include changing your root user email address and changing your AWS support plan. For more information, see "Tasks that require root user credentials" in the AWS Account Management Reference Guide(opens in a new tab).

IAM users

An **IAM user** is an identity that you create in AWS. It represents the person or application that interacts with AWS services and resources. It consists of a name and credentials.

By default, when you create a new IAM user in AWS, it has no permissions associated with it. To allow the IAM user to perform specific actions in AWS, such as launching an Amazon EC2 instance or creating an Amazon S3 bucket, you must grant the IAM user the necessary permissions

Best practice:

We recommend that you create individual IAM users for each person who needs to access AWS.

Even if you have multiple employees who require the same level of access, you should create individual IAM users for each of them. This provides additional security by allowing each IAM user to have a unique set of security credentials.

IAM policies

An **IAM policy** is a document that allows or denies permissions to AWS services and resources.

IAM policies enable you to customize users' levels of access to resources. For example, you can allow users to access all of the Amazon S3 buckets within your AWS account, or only a specific bucket.

Best practice:

Follow the security principle of **least privilege** when granting permissions.

By following this principle, you help to prevent users or roles from having more permissions than needed to perform their tasks.

For example, if an employee needs access to only a specific bucket, specify the bucket in the IAM policy. Do this instead of granting the employee access to all of the buckets in your AWS account.

Example: IAM policy

Here's an example of how IAM policies work. Suppose that the coffee shop owner has to create an IAM user for a newly hired cashier. The cashier needs access to the receipts kept in an Amazon S3 bucket with the ID: AWSDOC-EXAMPLE-BUCKET.

```
{
  "Version": "2012-10-17",
  "Statement": {
    "Effect": "Allow",
    "Action": "s3:ListObject",
    "Resource": "arn:aws:s3:::
AWSDOC-EXAMPLE-BUCKET"
  }
}
```

This example IAM policy allows permission to access the objects in the Amazon S3 bucket with ID: *AWSDOC-EXAMPLE-BUCKET*

In this example, the IAM policy is allowing a specific action within Amazon S3: ListObject. The policy also mentions a specific bucket ID: AWSDOC-EXAMPLE-BUCKET. When the owner attaches this policy to the cashier's IAM user, it will allow the cashier to view all of the objects in the AWSDOC-EXAMPLE-BUCKET bucket.

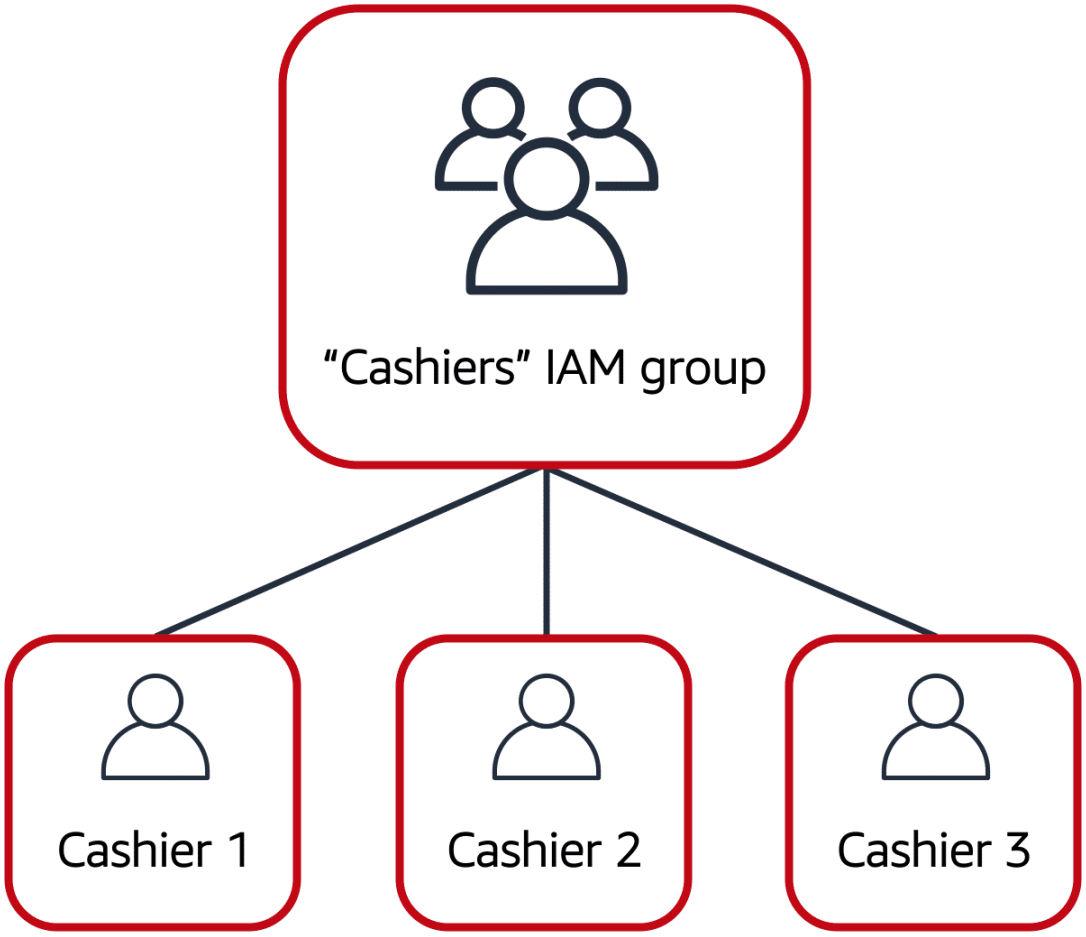
If the owner wants the cashier to be able to access other services and perform other actions in AWS, the owner must attach additional policies to specify these services and actions.

Now, suppose that the coffee shop has hired a few more cashiers. Instead of assigning permissions to each individual IAM user, the owner places the users into an **IAM group**(opens in a new tab).

IAM groups

An IAM group is a collection of IAM users. When you assign an IAM policy to a group, all users in the group are granted permissions specified by the policy.

Here's an example of how this might work in the coffee shop. Instead of assigning permissions to cashiers one at a time, the owner can create a "Cashiers" IAM group. The owner can then add IAM users to the group and then attach permissions at the group level.



Assigning IAM policies at the group level also makes it easier to adjust permissions when an employee transfers to a different job. For example, if a cashier becomes an inventory specialist, the coffee shop owner removes them from the "Cashiers" IAM group and adds them into the "Inventory Specialists" IAM group. This ensures that employees have only the permissions that are required for their current role.

What if a coffee shop employee hasn't switched jobs permanently, but instead, rotates to different workstations throughout the day? This employee can get the access they need through **IAM roles**(opens in a new tab).

Best practice:

IAM roles are ideal for situations in which access to services or resources needs to be granted temporarily, instead of long-term.

To review an example of how IAM roles could be used in the coffee shop example, choose the arrow buttons to display each of the following two steps.

Step 1

First, the coffee shop owner grants the employee permissions to the "Cashier" and "Inventory" roles so they can switch between these two workstations.

Step 1

First, the coffee shop owner grants the employee permissions to the "Cashier" and "Inventory" roles so they can switch between these two workstations.

1. 1

2. 2

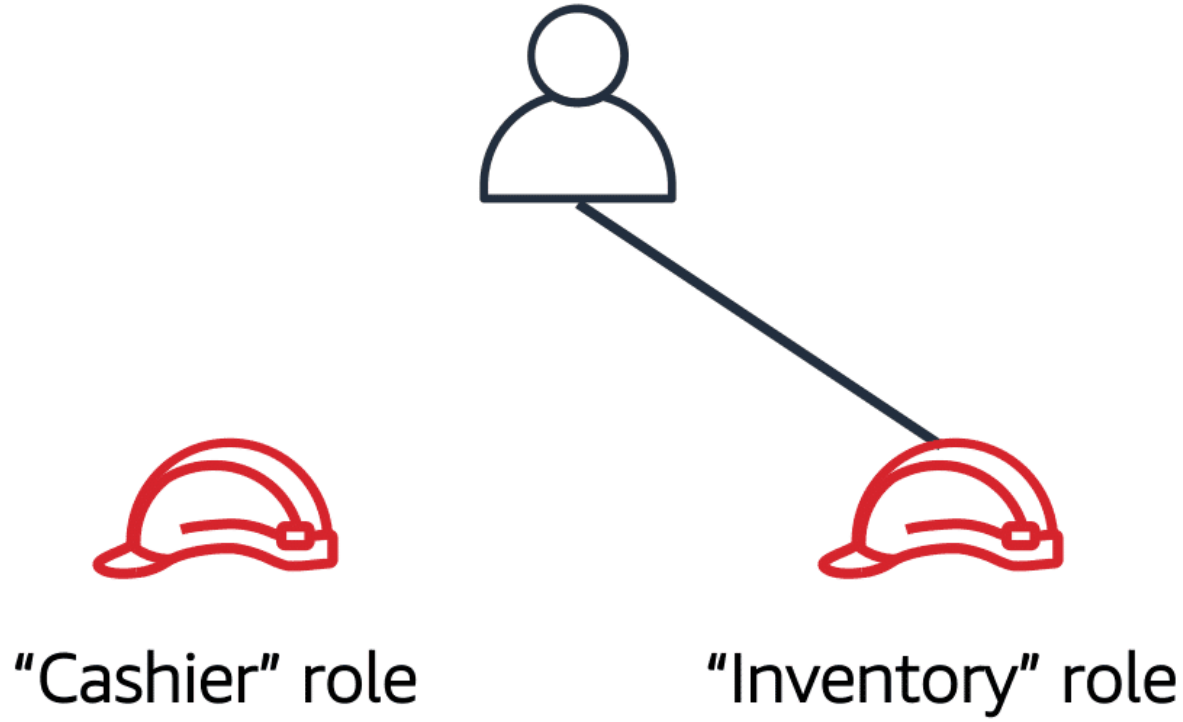
3.

Step 2

"Cashier" role

The employee begins their day by assuming the "Cashier" role. This grants them access to the cash register system.

Conclusion



Later in the day, the employee needs to update the inventory system. They assume the "Inventory" role. This grants the employee access to the inventory system and also revokes their access to the cash register system.

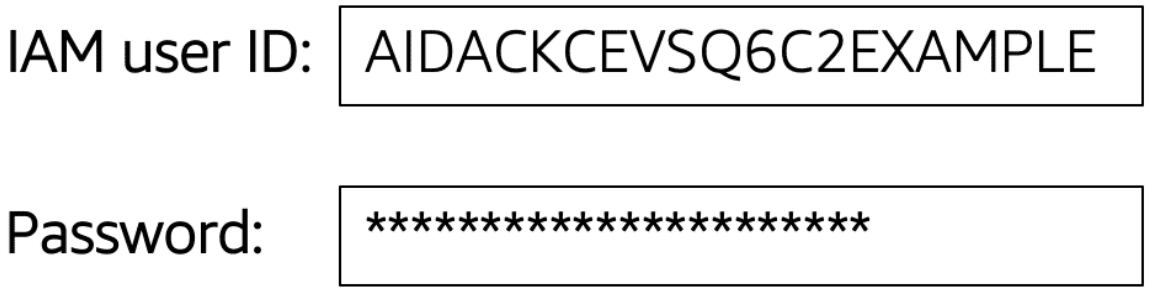
Multi-factor authentication

Have you ever signed in to a website that required you to provide multiple pieces of information to verify your identity? You might have needed to provide your password and then a second form of authentication, such as a random code sent to your phone. This is an example of **multi-factor authentication**(opens in a new tab).

In IAM, multi-factor authentication (MFA) provides an extra layer of security for your AWS account.

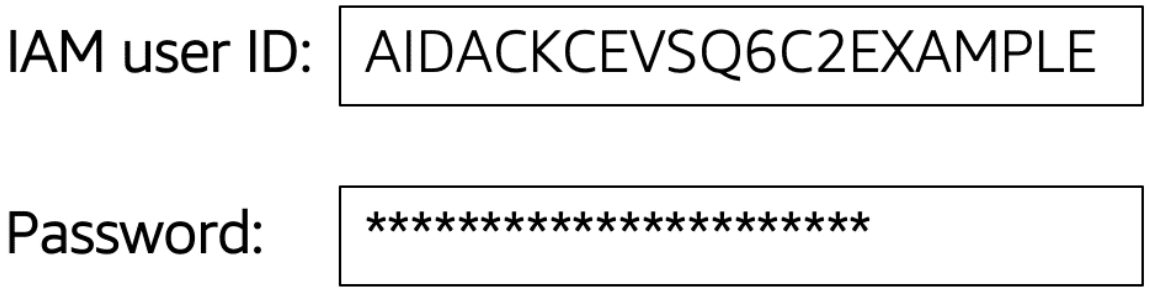
To learn more about how MFA works, choose the arrow buttons to display each of the following two steps.

Step 1



First, a user enters their IAM user ID and password to sign in to an AWS website.

Step 1



First, a user enters their IAM user ID and password to sign in to an AWS website.

1. 1
2. 2
3.

Step 2

Next, the user is prompted for an authentication response from their AWS MFA device. This device could be a hardware security key, a hardware device, or an MFA application on a device such as a smartphone.

Conclusion

When the user has been successfully authenticated, they are able to access the requested AWS services or resources.

You can enable MFA for the root user and IAM users. As a best practice, enable MFA for the root user and all IAM users in your account. By doing this, you can keep your AWS account safe from unauthorized access.

AWS Organizations

With your first foray into the AWS Cloud, you most likely will start with one AWS account and have everything reside in there. Most people start this way, but as your company grows or even begins their cloud journey, it's important to have a separation of duties. For example, you want your developers to have access to development resources, have your accounting staff able to access billing information, or even have business units separate so that they can experiment with AWS services without effecting each other. So you start to add more cards for each person, whoever needs to onboard. And before you know it, you end up with a tangled bowl of AWS account spaghetti, not as tasty as you might imagine.

For example, you'll then have to keep track of Account A, F, and G, or maybe Account B has the wrong permissions and Account C has billing and compliance info. One way to install order and to enforce who is allowed to perform certain functions in what account is to make use of an AWS service called AWS Organizations.

The easiest way to think of Organizations is as a central location to manage multiple AWS accounts. You can manage billing control, access, compliance, security, and share resources across your AWS accounts. Let's outline some of the main features of AWS Organizations, shall we?

The first is centralized management of all your AWS accounts. Think of all those AWS accounts, we had: A, B, C, F, G. Now you can combine them into an organization that enables us to manage the accounts centrally, and wow. Now we've found Accounts D and E in the process. Next up is consolidated billing for all member accounts. This means you can use the primary account of your organization to consolidate and pay for all member accounts. Another advantage of consolidated billing is bulk discounts. Cash money, indeed.

The next feature is that you can implement hierarchical groupings of your accounts to meet security, compliance, or budgetary needs. This means you can group accounts into organizational units, or OUs, kind of like business units, or BUs. For example, if you have accounts that must access only the AWS services that meet certain regulatory requirements, you can put those accounts into one OU, or if you have accounts that fall under the developer OU, you can group them accordingly.

One of the last main features we'll touch upon is that you have control over the AWS services and API actions that each account can access as an administrator of the primary account of an organization. You can use something called service control policies, or SCPs, to specify the maximum permissions for member accounts in the organization. In essence, with SCPs you can restrict which AWS services, resources, and individual API actions, the users and roles in each member account can access.

AWS Organizations

Suppose that your company has multiple AWS accounts. You can use **AWS Organizations**(opens in a new tab) to consolidate and manage multiple AWS accounts within a central location.

When you create an organization, AWS Organizations automatically creates a **root**, which is the parent container for all the accounts in your organization.

In AWS Organizations, you can centrally control permissions for the accounts in your organization by using **service control policies (SCPs)**(opens in a new tab). SCPs enable you to place restrictions on the AWS services, resources, and individual API actions that users and roles in each account can access.

Consolidated billing is another feature of AWS Organizations. You will learn about consolidated billing in a later module.

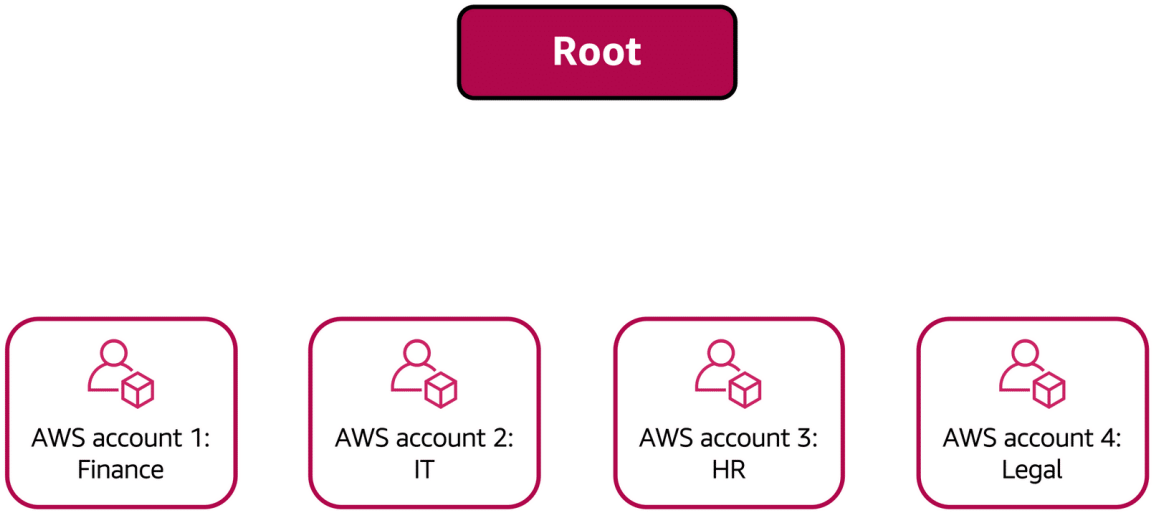
Organizational units

In AWS Organizations, you can group accounts into organizational units (OUs) to make it easier to manage accounts with similar business or security requirements. When you apply a policy to an OU, all the accounts in the OU automatically inherit the permissions specified in the policy.

By organizing separate accounts into OUs, you can more easily isolate workloads or applications that have specific security requirements. For instance, if your company has accounts that can access only the AWS services that meet certain regulatory requirements, you can put these accounts into one OU. Then, you can attach a policy to the OU that blocks access to all other AWS services that do not meet the regulatory requirements.

To review an example of how a company might use AWS Organizations;

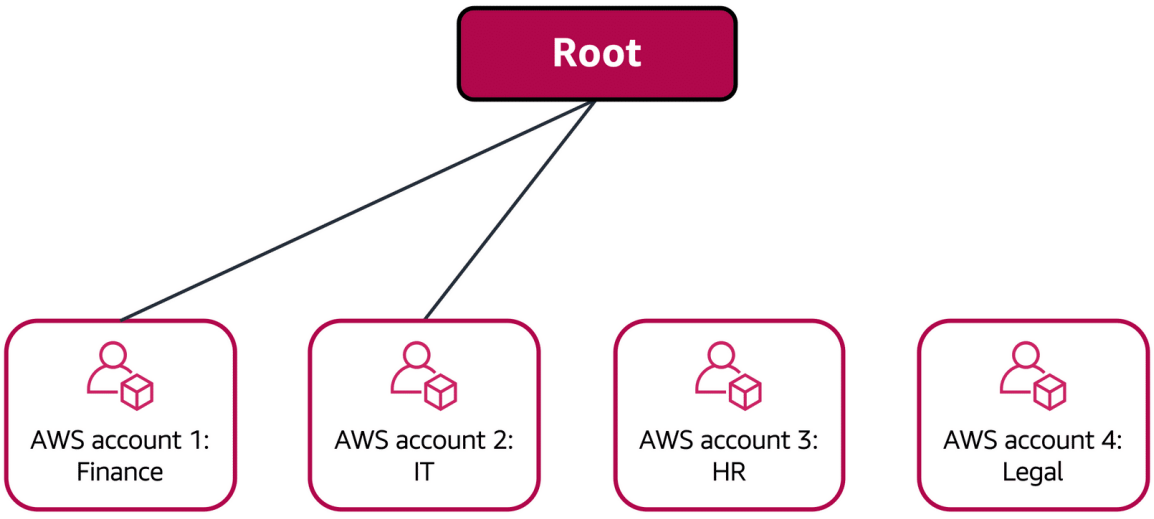
Step 1



Imagine that your company has separate AWS accounts for the finance, information technology (IT), human resources (HR), and legal departments. You decide to consolidate these accounts into a single organization so that you can administer them from a central location. When you create the organization, this establishes the root.

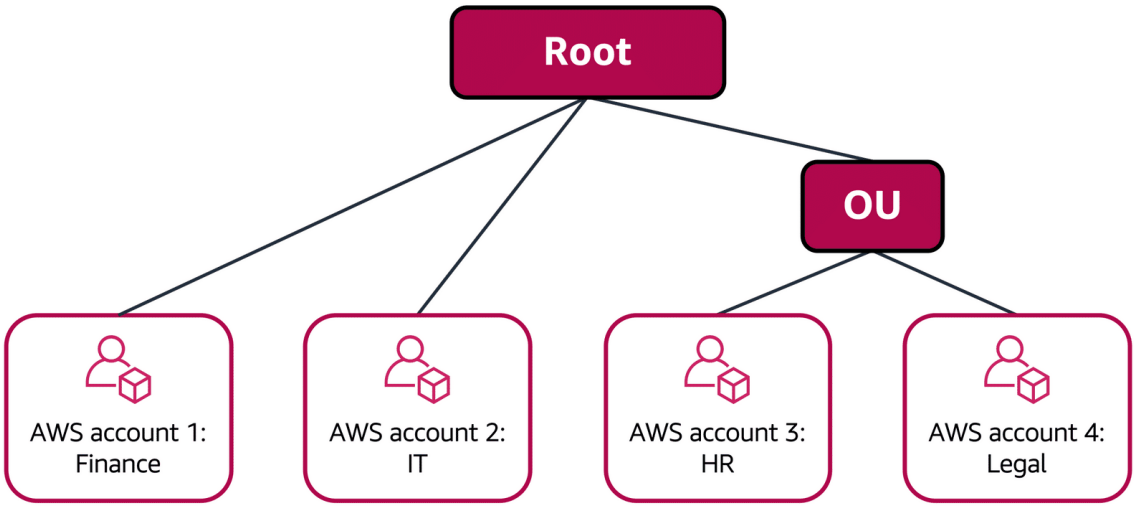
In designing your organization, you consider the business, security, and regulatory needs of each department. You use this information to decide which departments group together in OUs.

Step 2



The finance and IT departments have requirements that do not overlap with those of any other department. You bring these accounts into your organization to take advantage of benefits such as consolidated billing, but you do not place them into any OUs.

Step 3



The HR and legal departments need to access the same AWS services and resources, so you place them into an OU together. Placing them into an OU empowers you to attach policies that apply to both the HR and legal departments' AWS accounts.

Conclusion

Even though you have placed these accounts into OUs, you can continue to provide access for users, groups, and roles through IAM.

By grouping your accounts into OUs, you can easily give them access to the services and resources that they need. You also prevent them from accessing any services or resources that they do not need.

Compliance

For every industry, there are specific standards that need to be upheld, and you will be audited or inspected to ensure that you have met those standards. For example, for a coffee shop, the health inspector will come by and check that everything is up to code and sanitary. Similarly, you could be audited for taxes to see that you have run the back office correctly and have followed the law. You rely on documentation, records and inspections to pass audits and compliance checks as they come along.

You'll need to devise a similar way to meet compliance and auditing in AWS. Depending on what types of solutions you host on AWS, you will need to ensure that you are up to compliance for whatever standards and regulations your business is specifically held to. If you run software that deals with consumer data in the EU, you would need to make

sure that you're in compliance with GDPR, or if you run healthcare applications in the US you will need to design your architectures to meet HIPAA compliance requirements.

Whatever your compliance need is, you'll need some tools to be able to collect documents, records and inspect your AWS environment to check if you meet the compliance regulations that you're under. The first thing to note is, AWS has already built out data center infrastructure and networking following industry best practices for security, and as an AWS customer, you inherit all the best practices of AWS policies, architecture, and operational processes.

AWS complies with a long list of assurance programs that you can find online. This means that segments of your compliance have already been completed, and you can focus on meeting compliance within your own architectures that you build on top of AWS. The next thing to know in regards to compliance and AWS, is that the Region you choose to operate out of, might help you meet compliance regulations. If you can only legally store data in the country that the data is from, you can choose a Region that makes sense for you and AWS will not automatically replicate data across Regions.

You also should be very aware of the fact that you own your data in AWS. As shown in the AWS shared responsibility model, you have complete control over the data that you store in AWS. You can employ multiple different encryption mechanisms to keep your data safe, and that varies from service to service. So, if you need specific standards for data storage, you can devise a way to either reach those requirements by building it yourself on top of AWS or using the features that already exist in many services. For a lot of services, enabling data protection is a configuration setting on the resource.

AWS also offers multiple whitepapers and documents that you can download and use for compliance reports. Since you aren't running the data center yourself, you can essentially request that AWS provides you with documentation proving that they are following best practices for security and compliance.

One place you can access these documents is through a service called AWS Artifact. With AWS Artifact, you can gain access to compliance reports done by third parties who have validated a wide range of compliance standards. Check out the AWS Compliance Center in order to find compliance information all in one place. It will show you compliance enabling services as well as documentation like the AWS Risk and Security Whitepaper, which you should read to ensure that you understand security and compliance with AWS.

To know if you are compliant in AWS, please remember that we follow a shared responsibility. The underlying platform is secure and AWS can provide documentation on what types of compliance requirements they meet, through services like AWS Artifact and whitepapers. But, beyond that, what you build on AWS is up to you. You control the architecture of your applications and the solutions you build, and they need to be built with compliance, security, and the shared responsibility model in mind.

AWS Artifact

Depending on your company's industry, you may need to uphold specific standards. An audit or inspection will ensure that the company has met those standards.

AWS Artifact(opens in a new tab) is a service that provides on-demand access to AWS security and compliance reports and select online agreements. AWS Artifact consists of two main sections: AWS Artifact Agreements and AWS Artifact Reports.

To learn more, expand each of the following two categories.

AWS Artifact Agreements

Suppose that your company needs to sign an agreement with AWS regarding your use of certain types of information throughout AWS services. You can do this through **AWS Artifact Agreements**.

In AWS Artifact Agreements, you can review, accept, and manage agreements for an individual account and for all your accounts in AWS Organizations. Different types of agreements are offered to address the needs of customers who are subject to specific regulations, such as the Health Insurance Portability and Accountability Act (HIPAA).

AWS Artifact Reports

Next, suppose that a member of your company's development team is building an application and needs more information about their responsibility for complying with certain regulatory standards. You can advise them to access this information in **AWS Artifact Reports**.

AWS Artifact Reports provide compliance reports from third-party auditors. These auditors have tested and verified that AWS is compliant with a variety of global, regional, and industry-specific security standards and regulations. AWS Artifact Reports remains up to date with the latest reports released. You can provide the AWS audit artifacts to your auditors or regulators as evidence of AWS security controls.

The following are some of the compliance reports and regulations that you can find within AWS Artifact. Each report includes a description of its contents and the reporting period for which the document is valid.

AWS Artifact provides access to AWS security and compliance documents, such as AWS ISO certifications, Payment Card Industry (PCI) reports, and Service Organization Control (SOC) reports. To learn more about the available compliance reports, visit AWS Compliance Programs(opens in a new tab).

Customer Compliance Center

The **Customer Compliance Center**(opens in a new tab) contains resources to help you learn more about AWS compliance.

In the Customer Compliance Center, you can read customer compliance stories to discover how companies in regulated industries have solved various compliance, governance, and audit challenges.

You can also access compliance whitepapers and documentation on topics such as:

- AWS answers to key compliance questions
- An overview of AWS risk and compliance
- An auditing security checklist

Additionally, the Customer Compliance Center includes an auditor learning path. This learning path is designed for individuals in auditing, compliance, and legal roles who want to learn more about how their internal operations can demonstrate compliance using the AWS Cloud.

Denial-of-Service Attacks

D-D-o-S, DDoS, the distributed denial-of-service. It's an attack on your enterprise's infrastructure, and you've heard of it. Your security team might have written a plan for

it, and you know that many businesses have been devastated by it. But what exactly is it, and more importantly, how can you defend against it?

Now to be clear, this is a 14-hour discussion to really understand it all, but you need to at least know the fundamentals of how the attacks are carried out, and how AWS can automatically defend your infrastructure from these crippling assaults. Now we don't have a lot of time to cover all this, and the clock starts now. The objective of a DDoS attack is to shut down your application's ability to function by overwhelming the system to the point it can no longer operate.

In normal operations, your application takes requests from customers and returns results. In a DDoS attack, the bad actor tries to overwhelm the capacity of your application, basically to deny anyone your services. But a single machine attacking your application has no hope of providing enough of an attack by itself, so the distributed part is that the attack leverages other machines around the internet to unknowingly attack your infrastructure. The bad actor creates an army of zombie bots, brainlessly assaulting your enterprise. The key to a good attack, and I call it that when I should call it powerful. I mean, it's definitely chaotic evil, but the key is to have the assault commander do the smallest amount of work needed, and have the targeted victim receive an unbearable load of resulting work they must process through.

So let me cherry-pick a few specific attack examples that work really well. The UDP flood. It is based on the helpful parts of the internet, like the National Weather Service. Now anyone can send a small request to the Weather Service, and ask, "Give me weather," and in return, the Weather Service's fleet of machines will send back a massive amount of weather telemetry, forecasts, updates, lots of stuff. So the attack here is simple. The bad actor sends a simple request, give me weather. But it gives a fake return address on the request, your return address. So now the Weather Service very happily floods your server with megabytes of rain forecasts, and your system could be brought to a standstill, just sorting through the information it never wanted in the first place. Now that is one example of half a dozen low-level, brute force attacks, all designed to exhaust your network.

Some attacks are much more sophisticated, like the HTTP level attacks, which look like normal customers asking for normal things like complicated product searches over and over and over, all coming from an army of zombified bot machines. They ask for so much attention that regular customers can't get in.

They even try horrible tricks like the Slowloris attack. Mm-hmm. Imagine standing in line at the coffee shop, when someone in front of you takes seven minutes to order their whatever it is they're ordering, and you don't get to order until they finish and get out of your way. Well, Slowloris attack is the exact same thing. Instead of a normal connection, I would like to place an order, the attacker pretends to have a terribly slow connection. You get the picture. Meanwhile, your production servers are standing there waiting for the customer to finish their request so they can dash off and return the result. But until they get the entire packet, they can't move on to the next thread, the next customer. A few Slowloris attackers can exhaust the capacity of your entire front end with almost no effort at all. I could go on monologuing for hours just talking about the elegantly evil

architecture of these attacks, but we are on the clock here, and it is time to stop these attacks cold. And here's the cool solution: You already know the solution.

Everything we've been talking about over this entire course is not only good architecture, but it also helps solve almost all DDoS attack vectors with zero additional effort or cost. First attack, the low level network attacks like the UDP floods. Solution, security groups. The security groups only allow in proper request traffic. Things like weather reports use an entirely different protocol than the ones your customers use. Not on the list, you don't get to talk to the server. And what's more, security groups operate at the AWS network level, not at the EC2 instance level, like an operating system firewall might.

So massive attacks like UDP floods or reflection attacks just get shrugged off by the scale of the entire AWS Regions capacity, not your individual EC2's capacity. This is a case where our size is a huge advantage in your protection. I won't say it's impossible to overwhelm AWS, but the scale it would take, it would be too expensive for these bad actors. Slowloris attacks? Look at our elastic load balancer. Because the ELB handles the http traffic request first, so it waits until the entire message, no matter how fast or slow, is complete before sending it over to the front end web server. I mean, sure, you can try to overwhelm it, but remember how the ELB is scalable and how it runs at the region level?

To overwhelm ELB, you would once again have to overwhelm the entire AWS region. It's not theoretically impossible, but too massively expensive for anyone to pull off. For the sharpest, most sophisticated attacks, AWS also offers specialized defense tools called AWS Shield with AWS WAF. AWS WAF uses a web application firewall to filter incoming traffic for the signatures of bad actors. It has extensive machine learning capabilities, and can recognize new threats as they evolve and proactively help defend your system against an ever-growing list of destructive vectors.

All right, that's it, the clock is almost up. The takeaway is a well-architected system is already defended against most attacks. And by using AWS Shield Advanced, you can turn AWS into your partner against DDoS attacks.

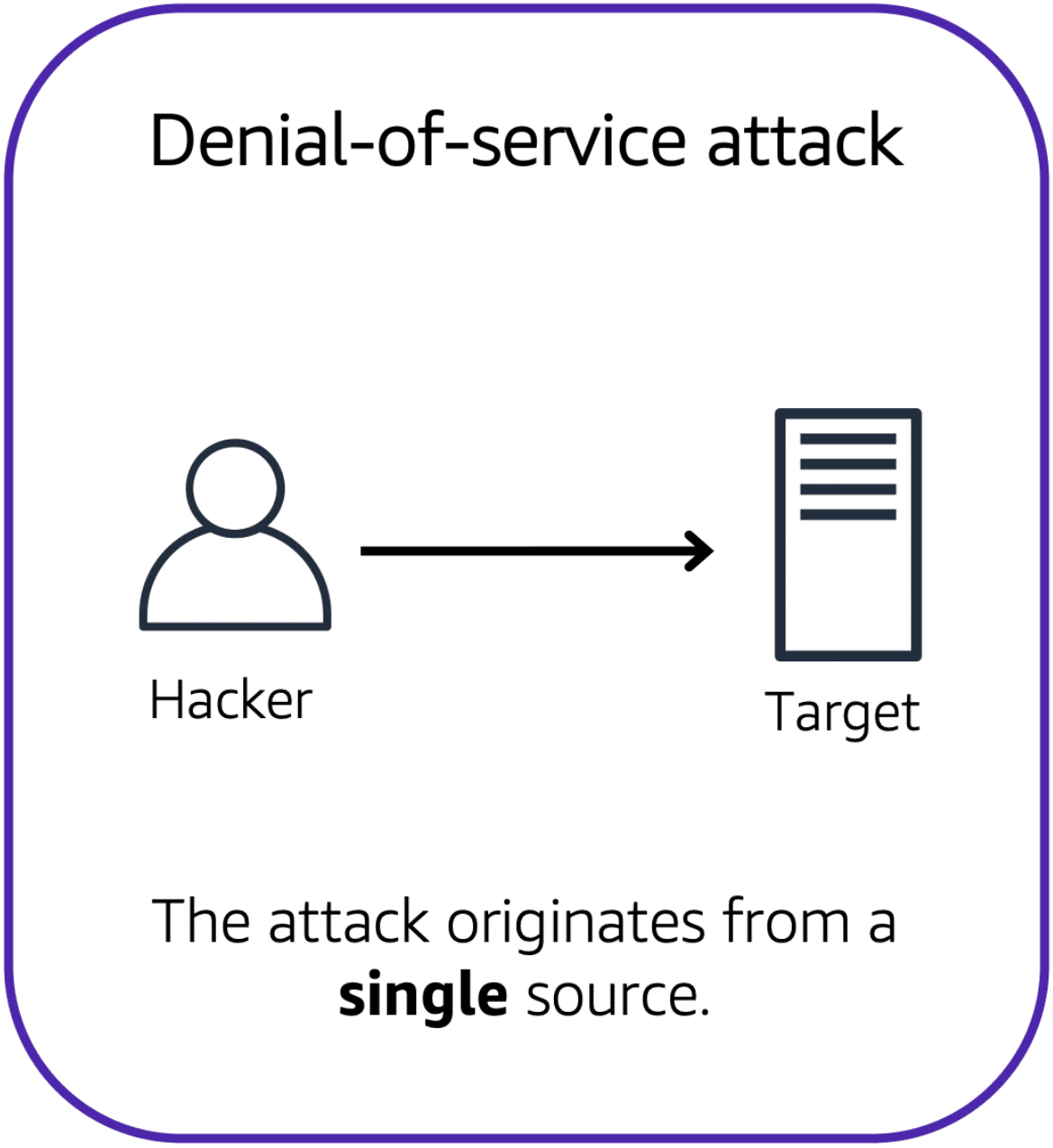
Customers can call the coffee shop to place their orders. After answering each call, a cashier takes the order and gives it to the barista.

However, suppose that a prankster is calling in multiple times to place orders but is never picking up their drinks. This causes the cashier to be unavailable to take other customers' calls. The coffee shop can attempt to stop the false requests by blocking the phone number that the prankster is using.

In this scenario, the prankster's actions are similar to a **denial-of-service attack**.

Denial-of-service attacks

A **denial-of-service (DoS) attack** is a deliberate attempt to make a website or application unavailable to users.



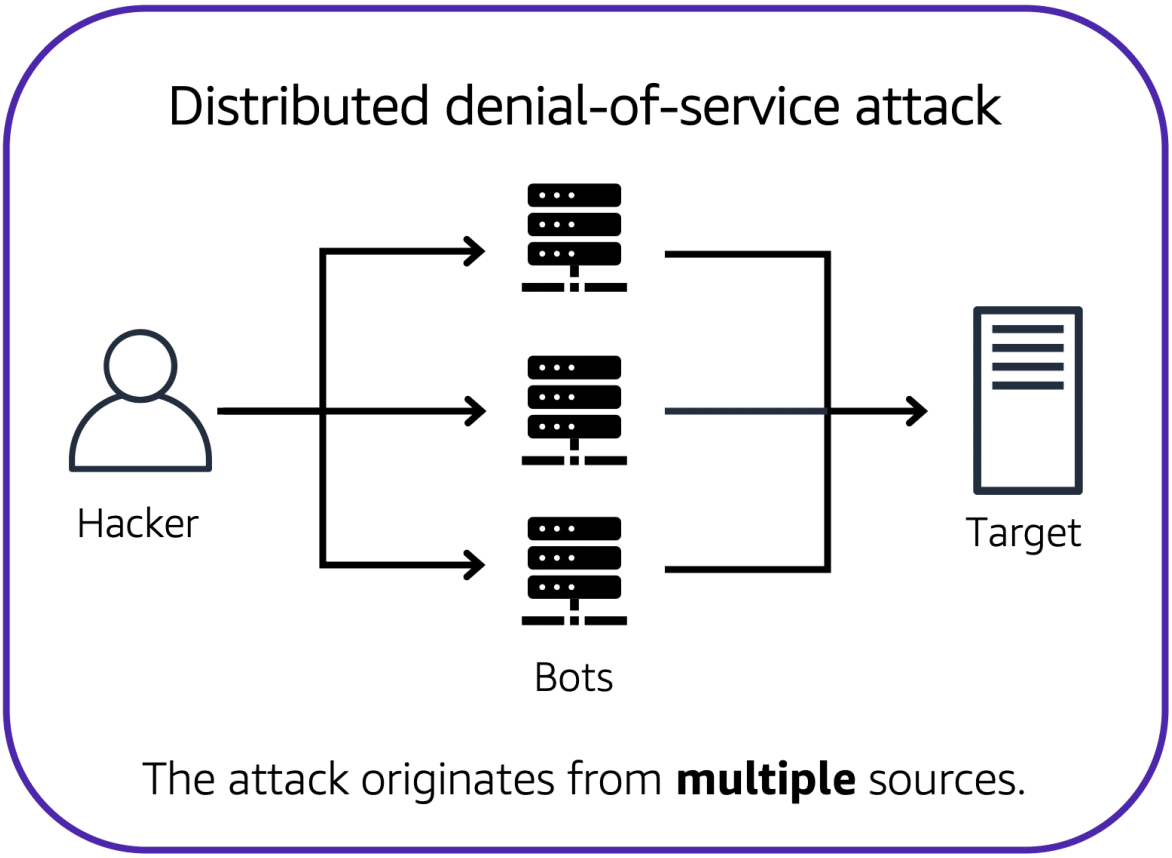
For example, an attacker might flood a website or application with excessive network traffic until the targeted website or application becomes overloaded and is no longer able to respond. If the website or application becomes unavailable, this denies service to users who are trying to make legitimate requests.

Distributed denial-of-service attacks

Now, suppose that the prankster has enlisted the help of friends.

The prankster and their friends repeatedly call the coffee shop with requests to place orders, even though they do not intend to pick them up. These requests are coming in from different phone numbers, and it's impossible for the coffee shop

to block them all. Additionally, the influx of calls has made it increasingly difficult for customers to be able to get their calls through. This is similar to a **distributed denial-of-service attack**.



In a distributed denial-of-service (DDoS) attack, multiple sources are used to start an attack that aims to make a website or application unavailable. This can come from a group of attackers, or even a single attacker. The single attacker can use multiple infected computers (also known as “bots”) to send excessive traffic to a website or application.

To help minimize the effect of DoS and DDoS attacks on your applications, you can use **AWS Shield**(opens in a new tab).

AWS Shield

AWS Shield is a service that protects applications against DDoS attacks. AWS Shield provides two levels of protection: Standard and Advanced.

To learn more about AWS Shield, expand each of the following two categories.

AWS Shield Standard

AWS Shield Standard automatically protects all AWS customers at no cost. It protects your AWS resources from the most common, frequently occurring types of DDoS attacks.

As network traffic comes into your applications, AWS Shield Standard uses a variety of analysis techniques to detect malicious traffic in real time and automatically mitigates it.

AWS Shield Advanced

AWS Shield Advanced is a paid service that provides detailed attack diagnostics and the ability to detect and mitigate sophisticated DDoS attacks.

It also integrates with other services such as Amazon CloudFront, Amazon Route 53, and Elastic Load Balancing. Additionally, you can integrate AWS Shield with AWS WAF by writing custom rules to mitigate complex DDoS attacks.

Additional Security Services

With all the comings and goings on in your coffee shop, you'll want to increase security to your coffee beans, equipment, and even money in the till. For your beans, this could be when they're sitting in your store room, or even when you're transporting them between shops. After all, we don't want unwanted visitors with access to our coffee beans, or even running off with precious equipment.

To start off, let's chat about how you can secure your coffee beans, or your data whether it's at rest, or in transit. For our beans, the simple way to do it would be to lock the door when we'd leave at night. That's the notion of encryption, which is securing a message or data in a way that can only be accessed by authorized parties. Non-authorized parties are therefore less likely to be able to access the message. Or not able to access it at all. Think of it as that key and door example. If you have the key, you can unlock the door. But if you don't, then you cannot unlock that door.

At AWS, this comes in two variations. Encryption at rest and encryption in transit. By at rest, we mean when your data is idle. It's just being stored and not moving. For example, server-side encryption at rest is enabled on all DynamoDB table data. And that helps prevent unauthorized access. DynamoDB's encryption at rest also integrates with AWS KMS, or Key Management Service, for managing the encryption key that is used to encrypt your tables. That's the key for your door, remember? And without it, you won't be able to access your data. So make sure to keep it safe.

Similarly, in-transit means that the data is traveling between, say A and B. Where A is the AWS service, and B could be a client accessing the service. Or even another AWS service itself. For example, let's say we have a Redshift instance running. And we want to connect it with a SQL client. We use secure sockets layer, or SSL connections to encrypt data, and we can use service certificates to validate, and authorize a client. This means that data is protected when passing between Redshift, and our client. And this functionality exists in numerous other AWS services such as SQS, S3, RDS, and many more.

But speaking of other services, the next service we want to highlight is called Amazon Inspector. Inspector helps to improve security, and compliance of your AWS deployed applications by running an automated security assessment against your infrastructure. Specifically, it helps to check on deviations of security best practices, exposure of EC2 instances, vulnerabilities, and so forth. The service consists of three parts a network configuration reachability piece, an Amazon agent, which can be installed an EC2 instances, and a security assessment service that brings them all together. To use it, you configure Inspector options, run the service, out pops a list of potential security issues. The resulting findings are displayed in the Amazon Inspector console, and they are presented with a detailed description of the security issue, and a recommendation on how to fix it. Additionally, you can retrieve findings through an API. So as to go towards the best practice of performing remediation to fix issues.

Another service dimension is our threat detection offering known as Amazon GuardDuty. It analyzes continuous streams of metadata generated from your account, and network activity found on AWS CloudTrail events, Amazon VPC Flow Logs, and DNS logs. It uses integrated threat intelligence such as known malicious IP addresses, anomaly detection, and machine learning to identify threats more accurately. The best part is that it runs independently from your other AWS services. So it won't affect performance or availability of your existing infrastructure, and workloads.

They are so many other security services like Advanced Shield and Security Hub. So please check out the Resources section to learn more. Thanks for following along.

AWS Key Management Service (AWS KMS)

The coffee shop has many items, such as coffee machines, pastries, money in the cash registers, and so on. You can think of these items as data. The coffee shop owners want to ensure that all of these items are secure, whether they're sitting in the storage room or being transported between shop locations.

In the same way, you must ensure that your applications' data is secure while in storage **(encryption at rest)** and while it is transmitted, known as **encryption in transit**.

AWS Key Management Service (AWS KMS)(opens in a new tab) enables you to perform encryption operations through the use of **cryptographic keys**. A cryptographic key is a random string of digits used for locking (encrypting) and unlocking (decrypting) data. You can use AWS KMS to create, manage, and use cryptographic keys. You can also control the use of keys across a wide range of services and in your applications.

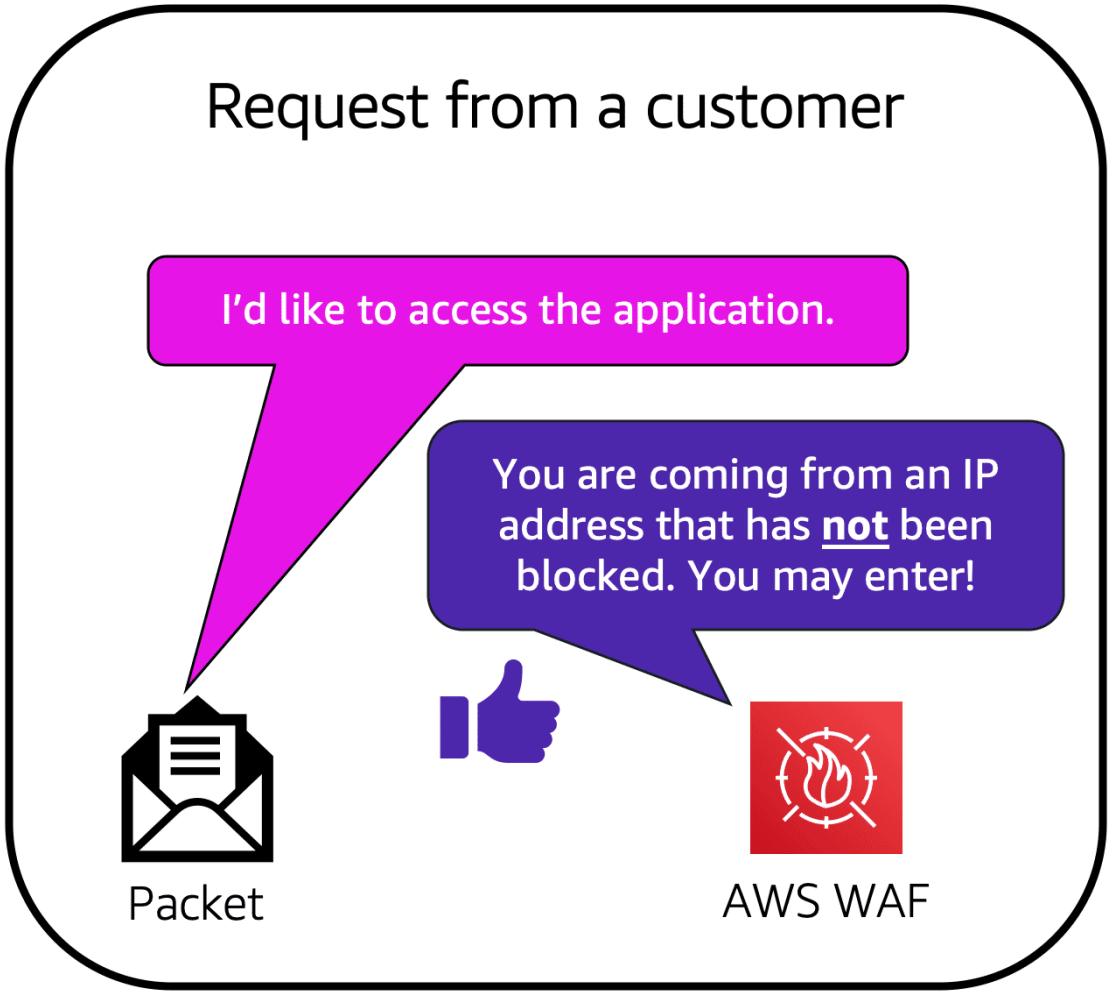
With AWS KMS, you can choose the specific levels of access control that you need for your keys. For example, you can specify which IAM users and roles are able to manage keys. Alternatively, you can temporarily disable keys so that they are no longer in use by anyone. Your keys never leave AWS KMS, and you are always in control of them.

AWS WAF

AWS WAF(opens in a new tab) is a web application firewall that lets you monitor network requests that come into your web applications.

AWS WAF works together with Amazon CloudFront and an Application Load Balancer. Recall the network access control lists that you learned about in an earlier module. AWS WAF works in a similar way to block or allow traffic. However, it does this by using a **web access control list (ACL)**(opens in a new tab) to protect your AWS resources.

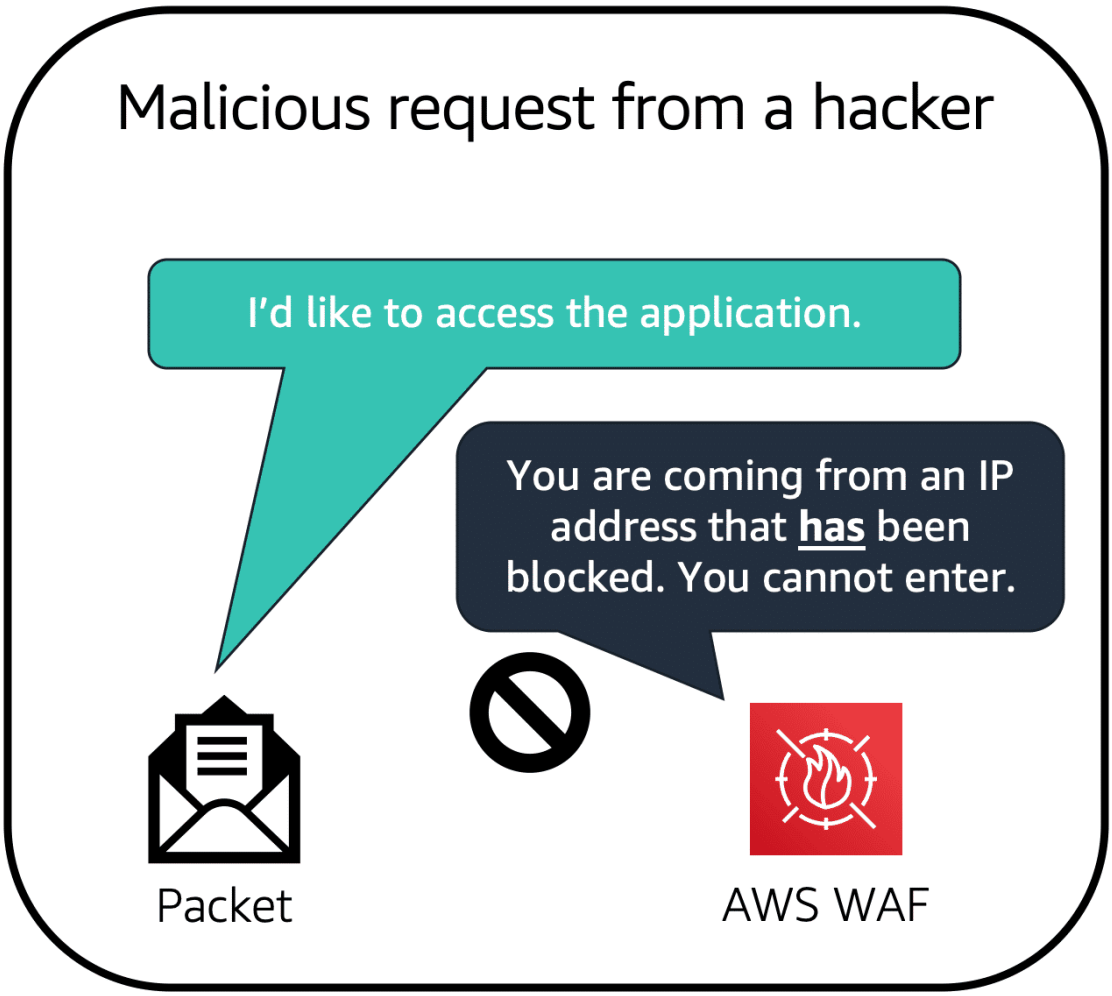
Here's an example of how you can use AWS WAF to allow and block specific requests.



Suppose that your application has been receiving malicious network requests from several IP addresses. You want to prevent these requests from continuing to access your application, but you also want to ensure that legitimate users can

still access it. You configure the web ACL to allow all requests except those from the IP addresses that you have specified.

When a request comes into AWS WAF, it checks against the list of rules that you have configured in the web ACL. If a request does not come from one of the blocked IP addresses, it allows access to the application.



However, if a request comes from one of the blocked IP addresses that you have specified in the web ACL, AWS WAF denies access.

Amazon Inspector

Suppose that the developers at the coffee shop are developing and testing a new ordering application. They want to make sure that they are designing the application in accordance with security best practices. However, they have several other applications to develop, so they cannot spend much time

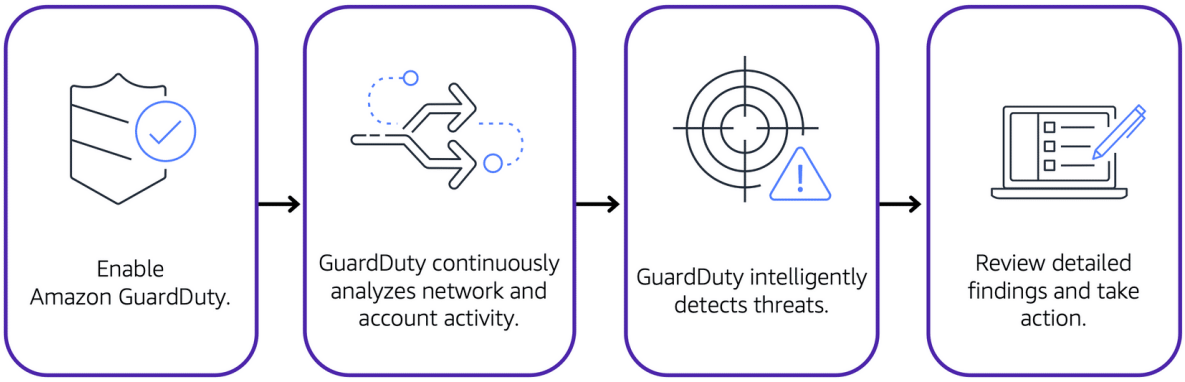
conducting manual assessments. To perform automated security assessments, they decide to use **Amazon Inspector**(opens in a new tab).

Amazon Inspector helps to improve the security and compliance of applications by running automated security assessments. It checks applications for security vulnerabilities and deviations from security best practices, such as open access to Amazon EC2 instances and installations of vulnerable software versions.

After Amazon Inspector has performed an assessment, it provides you with a list of security findings. The list prioritizes by severity level, including a detailed description of each security issue and a recommendation for how to fix it. However, AWS does not guarantee that following the provided recommendations resolves every potential security issue. Under the shared responsibility model, customers are responsible for the security of their applications, processes, and tools that run on AWS services.

Amazon GuardDuty

Amazon GuardDuty(opens in a new tab) is a service that provides intelligent threat detection for your AWS infrastructure and resources. It identifies threats by continuously monitoring the network activity and account behavior within your AWS environment.



After you have enabled GuardDuty for your AWS account, GuardDuty begins monitoring your network and account activity. You do not have to deploy or manage any additional security software. GuardDuty then continuously analyzes data from multiple AWS sources, including VPC Flow Logs and DNS logs.

If GuardDuty detects any threats, you can review detailed findings about them from the AWS Management Console. Findings include recommended steps for remediation. You can also configure AWS Lambda functions to take remediation steps automatically in response to GuardDuty's security findings.

Amazon CloudWatch

Now we're making lots of coffee, serving customers and everything seems to be going spiffy in our coffee shop. But as we run espresso machines more and more, use mugs, constantly open and close the fridge, we want to be able to make sure we're alerted to something which might have gone awry. Maybe an espresso machine needs to be cleaned or repaired. The point is, as a business owner, you need visibility into the state of your systems. Are things running well? Do you see a degraded customer experience? Are you frequently delivering the wrong drink to customers or is everyone getting their orders as expected. You have all sorts of questions about the success of your operations.

And the same idea applies to systems built on AWS. You need a way to monitor the health and operations of your solutions. Luckily, you don't need to build out your own monitoring platform. We have done this for you.

Introducing Amazon CloudWatch. CloudWatch allows you to monitor your AWS infrastructure and the applications you run on AWS in real time. It works by tracking and monitoring metrics. Think of metrics as variables that are tied to your resources. For example, the amount of espressos made by an espresso machine or the CPU utilization of an EC2 instance.

Let's take a customer approach for our coffee shop. Say we have an espresso machine and it needs to be cleaned after it makes 100 espressos. CloudWatch allows us to create a custom metric called Espresso Count. And once it hits 100, we want to alert staff to clean the machine. Simple, right? Well with CloudWatch, you can accomplish this by creating what is called a CloudWatch alarm. You set a threshold for a metric, and when that threshold is reached CloudWatch can generate an alert and trigger an action. This means we can alert on the custom metric, in this case, hitting 100, and then perform an action.

Even better, CloudWatch alarms are integrated with SNS. So we can then send an SMS to the manager to say, clean the machine. You can create all sorts of custom alarms for metrics from all different types of AWS resources.

Now, what if we want to aggregate all those metrics in a single pane of glass? Well, we can use CloudWatch's dashboard feature. Think of a dashboard as a screen, which lists out metrics in near real time. In our case, we can create a CloudWatch dashboard which will show us all our espresso machines and their espresso counts so we can proactively monitor them. These dashboards would auto refresh when they are open so we can see an up-to-date view of our resources.

Finally, what are the benefits of using a service such as CloudWatch? Well, the first one is that you can have access to all your metrics from a central location. This enables you to collect metrics and logs from all your AWS resources applications, and services that run on AWS and on-premises servers, helping you break down silos so that you can easily gain system-wide visibility.

You can also get visibility across your applications, infrastructure, and services, which means you gain insights across your distributed stack so you can correlate and visualize metrics and logs to quickly pinpoint and resolve issues. This in turn means you can reduce mean time to resolution, or MTTR, and improve total cost of ownership, or TCO. So in our coffee shop, if the MTTR of cleaning hours restaurant machines is shorter then we can save on TCO with them. This means freeing up important resources like developers to focus on adding business value.

Lastly, you can drive insights to optimize applications and operational resources. By, for example, aggregating usage across an entire fleet of EC2 instances to derive operational and utilization insights. And now our staff can focus on serving coffee versus constantly cleaning machines before they are due to be cleaned.

And there you have it, folks - you're up to speed on what CloudWatch entails. Now if you excuse me, I have a date with an espresso machine to refill my coffee.

Amazon CloudWatch

Amazon CloudWatch(opens in a new tab) is a web service that enables you to monitor and manage various metrics and configure alarm actions based on data from those metrics.

CloudWatch uses **metrics**(opens in a new tab) to represent the data points for your resources. AWS services send metrics to CloudWatch. CloudWatch then uses these metrics to create graphs automatically that show how performance has changed over time.

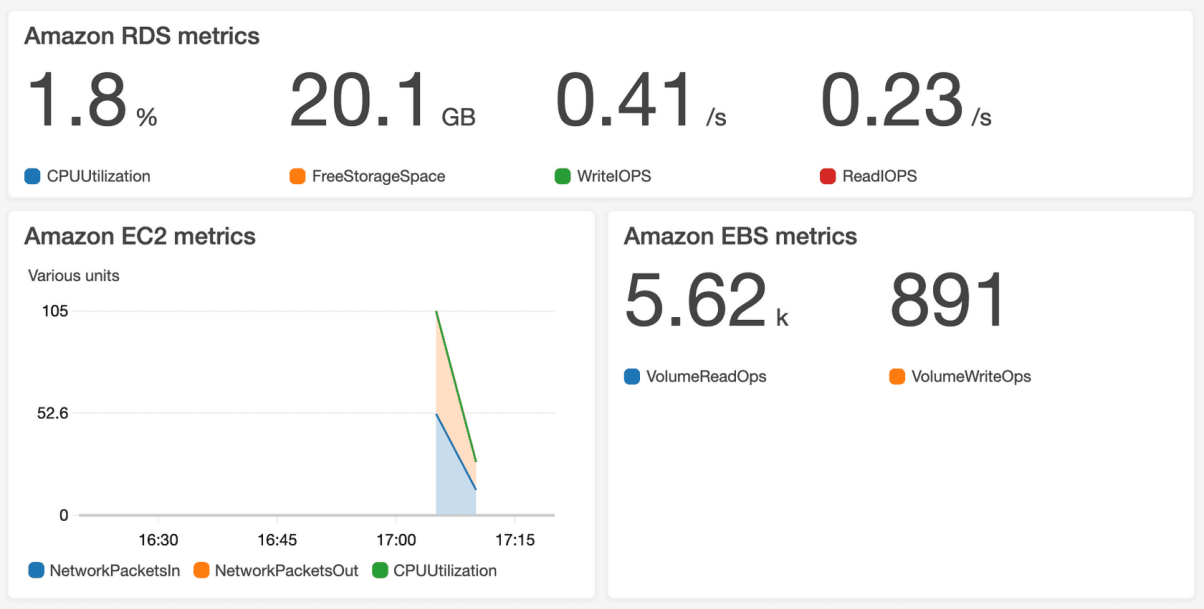
CloudWatch alarms

With CloudWatch, you can create **alarms**(opens in a new tab) that automatically perform actions if the value of your metric has gone above or below a predefined threshold.

For example, suppose that your company's developers use Amazon EC2 instances for application development or testing purposes. If the developers occasionally forget to stop the instances, the instances will continue to run and incur charges.

In this scenario, you could create a CloudWatch alarm that automatically stops an Amazon EC2 instance when the CPU utilization percentage has remained below a certain threshold for a specified period. When configuring the alarm, you can specify to receive a notification whenever this alarm is triggered.

CloudWatch dashboard



The CloudWatch **dashboard**(opens in a new tab) feature enables you to access all the metrics for your resources from a single location. For example, you can use a CloudWatch dashboard to monitor the CPU utilization of an Amazon EC2 instance, the total number of requests made to an Amazon S3 bucket, and more. You can even customize separate dashboards for different business purposes, applications, or resources.

AWS CloudTrail

The Cash Register, one of the world's first self-auditing devices. The principle is simple: trust but verify. You have your entire store being run by a clerk that you trust, but you want to make sure that the cash in the drawer matches the actual sales. So every transaction then is recorded and tabulated. So at the end of the day, you know exactly what should be there.

Being able to audit transactions in IT is a critical element in most compliance structures. But in a physical data center, there are so many places where a human can, even by accident, make changes without any record of that change getting recorded. At AWS, that problem goes away because everything is programmatic.

Introducing AWS CloudTrail, the comprehensive API auditing tool. The engine is simple, every request made to AWS, it doesn't matter if it's to launch an EC2 instance, or add a row to a DynamoDB table, or change a user's permissions. Every request gets logged in the CloudTrail engine. The engine records exactly who made the request, which operator, when did they send the API call? Where were they? What was their IP address? And what was the response? Did something change? And what is the new state? Was the request denied?

From an auditing perspective, well, this is fabulous. So imagine that you're dealing with an auditor who is checking to make sure that nobody from the outside can access your database. That's a good thing. Well, okay, you build a security group that locks out

external traffic. But remember that a root-level administrator still has permissions to change those settings, right?
Well, so how do you prove to the auditor that the security group settings never changed? The answer is CloudTrail. And then CloudTrail can save those logs indefinitely in secure S3 buckets. In addition, with tamper-proof methods like Vault Lock, you then can show absolute provenance of all of these critical security audit logs.

AWS CloudTrail

AWS CloudTrail(opens in a new tab) records API calls for your account. The recorded information includes the identity of the API caller, the time of the API call, the source IP address of the API caller, and more. You can think of CloudTrail as a "trail" of breadcrumbs (or a log of actions) that someone has left behind them.

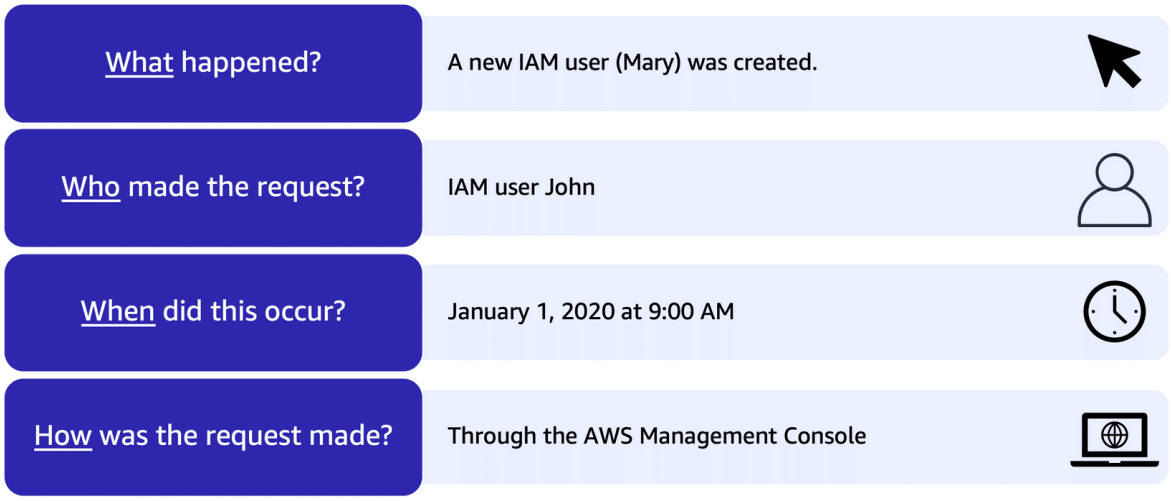
Recall that you can use API calls to provision, manage, and configure your AWS resources. With CloudTrail, you can view a complete history of user activity and API calls for your applications and resources.

Events are typically updated in CloudTrail within 15 minutes after an API call. You can filter events by specifying the time and date that an API call occurred, the user who requested the action, the type of resource that was involved in the API call, and more

Example: AWS CloudTrail event

Suppose that the coffee shop owner is browsing through the AWS Identity and Access Management (IAM) section of the AWS Management Console. They discover that a new IAM user named Mary was created, but they do not know who, when, or which method created the user.

To answer these questions, the owner navigates to AWS CloudTrail.



In the CloudTrail Event History section, the owner applies a filter to display only the events for the "CreateUser" API action in IAM. The owner locates the event for the API call that created an IAM user for Mary. This event record provides complete details about what occurred:

On January 1, 2020 at 9:00 AM, IAM user John created a new IAM user (Mary) through the AWS Management Console.

CloudTrail Insights

Within CloudTrail, you can also enable **CloudTrail Insights**(opens in a new tab). This optional feature allows CloudTrail to automatically detect unusual API activities in your AWS account.

For example, CloudTrail Insights might detect that a higher number of Amazon EC2 instances than usual have recently launched in your account. You can then review the full event details to determine which actions you need to take next.

AWS Trusted Advisor

AWS Trusted Advisor

AWS Trusted Advisor(opens in a new tab) is a web service that inspects your AWS environment and provides real-time recommendations in accordance with AWS best practices.

Trusted Advisor compares its findings to AWS best practices in five categories: cost optimization, performance, security, fault tolerance, and service limits. For the checks in each category, Trusted Advisor offers a list of recommended actions and additional resources to learn more about AWS best practices.

The guidance provided by AWS Trusted Advisor can benefit your company at all stages of deployment. For example, you can use AWS Trusted Advisor to assist you while you are creating new workflows and developing new applications. You can also use it while you are making ongoing improvements to existing applications and resources.

AWS Trusted Advisor dashboard

When you access the Trusted Advisor dashboard on the AWS Management Console, you can review completed checks for cost optimization, performance, security, fault tolerance, and service limits.

For each category:

- •

The green check indicates the number of items for which it detected **no problems**.

- •

The orange triangle represents the number of recommended **investigations**.

- ·

The red circle represents the number of recommended **actions**.

Module 8 Introduction

AWS Free Tier

So you've created a brand spanking new AWS account and want to get started, but you're worried about exhausting your credit limit. Well, don't worry. If you're looking to try out some AWS services, you might be able to try them out under our Free Tier.

Depending on the product you're looking at, the Free Tier offers three different ways to try out a service. Number one, always free. That means a service is available to all AWS customers and the Free Tier does not expire. Number two, 12 months free. That means you have 12 months to try out a service following your initial signup date to AWS, so the date you created your AWS account. And number three, trials. Some services offer a short-term free trial, and then they expire after that period ends.

Simple enough, right? Let's illustrate with a few examples. Let's take AWS Lambda, our serverless compute option. As of March 2020, it allows for one million free invocations per month. That means if you stay under one million invocations, it's always free. This Free Tier never expires. Another example is S3, our object store service. It's free for 12 months for up to five gigs of storage. Thereafter, you'll incur a cost. Great for trying out a static website on S3, I might say. And the last example is Lightsail where you can deploy ready-made application stacks. We offer a one month trial of up to 750 hours of usage. That's a pretty decent amount to deploy and test a WordPress blog. I mean, who knows, maybe you wanna show people your smoothie recipes.

But before I digress, some other services that qualify under the free tier are SageMaker, Comprehend Medical, DynamoDB, SNS, Cognito, and so much more. If you want to see the full list of 60 or so services, please check out our Resources section. Happy exploring of the Free Tier.

AWS Free Tier

The AWS Free Tier(opens in a new tab) enables you to begin using certain services without having to worry about incurring costs for the specified period.

Three types of offers are available:

- Always Free
- 12 Months Free

- Trials

For each free tier offer, make sure to review the specific details about exactly which resource types are included.

Always Free
These offers do not expire and are available to all AWS customers.

For example, AWS Lambda allows 1 million free requests and up to 3.2 million seconds of compute time per month. Amazon DynamoDB allows 25 GB of free storage per month.

12 Months Free
These offers are free for 12 months following your initial sign-up date to AWS.

Examples include specific amounts of Amazon S3 Standard Storage, thresholds for monthly hours of Amazon EC2 compute time, and amounts of Amazon CloudFront data transfer out.

Trials
Short-term free trial offers start from the date you activate a particular service. The length of each trial might vary by number of days or the amount of usage in the service.

For example, Amazon Inspector offers a 90-day free trial. Amazon Lightsail (a service that enables you to run virtual private servers) offers 750 free hours of usage over a 30-day period.

AWS Pricing Concepts

How AWS pricing works

AWS offers a range of cloud computing services with pay-as-you-go pricing.

To learn more about how AWS pricing works, expand each of the following three categories.

Pay for what you use.
For each service, you pay for exactly the amount of resources that you actually use, without requiring long-term contracts or complex licensing.

Pay less when you reserve.

Some services offer reservation options that provide a significant discount compared to On-Demand Instance pricing.

For example, suppose that your company is using Amazon EC2 instances for a workload that needs to run continuously. You might choose to run this workload on Amazon EC2 Instance Savings Plans, because the plan allows you to save up to 72% over the equivalent On-Demand Instance capacity.

Pay less with volume-based discounts when you use more.

Some services offer tiered pricing, so the per-unit cost is incrementally lower with increased usage.

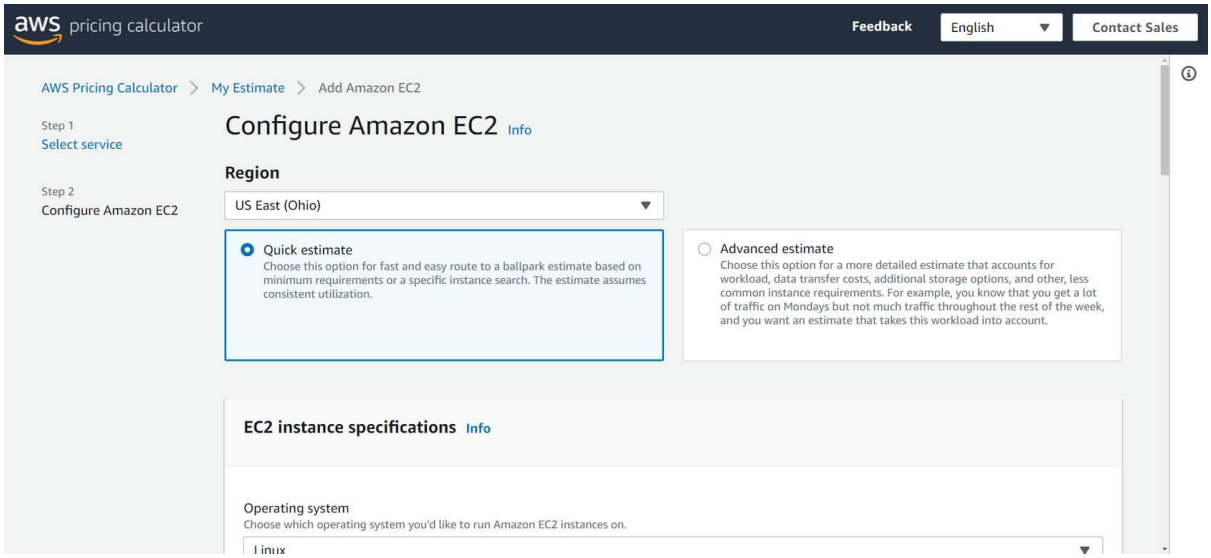
For example, the more Amazon S3 storage space you use, the less you pay for it per GB.

AWS Pricing Calculator

The **AWS Pricing Calculator**(opens in a new tab) lets you explore AWS services and create an estimate for the cost of your use cases on AWS. You can organize your AWS estimates by groups that you define. A group can reflect how your company is organized, such as providing estimates by cost center.

When you have created an estimate, you can save it and generate a link to share it with others.

When you have created an estimate, you can save it and generate a link to share it with others.



Suppose that your company is interested in using Amazon EC2. However, you are not yet sure which AWS Region or instance type would be the most cost-efficient for your use case. In the AWS Pricing Calculator, you can enter details, such as the kind of operating system you need, memory requirements, and input/output (I/O) requirements. By using the AWS Pricing Calculator, you can review an estimated comparison of different EC2 instance types across AWS Regions.

AWS pricing examples

This section presents a few examples of pricing in AWS services.

AWS Lambda

To learn more about AWS Lambda pricing(opens in a new tab), choose each of the following two tabs.

AWS Lambda pricing

For AWS Lambda, you are charged based on the number of requests for your functions and the time that it takes for them to run.

AWS Lambda allows 1 million free requests and up to 3.2 million seconds of compute time per month.

You can save on AWS Lambda costs by signing up for a Compute Savings Plan. A Compute Savings Plan offers lower compute costs in exchange for committing to a consistent amount of usage over a 1-year or 3-year term. This is an example of **paying less when you reserve**.

Pricing example

If you have used AWS Lambda in multiple AWS Regions, you can view the itemized charges by Region on your bill.

In this example, all the AWS Lambda usage occurred in the Northern Virginia Region. The bill lists separate charges for the number of requests for functions and their duration.

Both the number of requests and the total duration of requests in this example are under the thresholds in the AWS Free Tier, so the account owner would not have to pay for any AWS Lambda usage in this month.

Lambda		**$0.00**
US East (N. Virginia)		**$0.00**
AWS Lambda Lambda-GB-Second		$0.00
AWS Lambda - Compute Free Tier - 400,000 GB-Seconds - US East (Northern Virginia)	254.575 seconds	$0.00
AWS Lambda Request		$0.00
AWS Lambda - Requests Free Tier - 1,000,000 Requests - US East (Northern Virginia)	680.000 Requests	$0.00

Amazon EC2

To learn more about Amazon EC2 pricing(opens in a new tab), choose each of the following two tabs.

Amazon EC2 Pricing

With Amazon EC2, you pay for only the compute time that you use while your instances are running.

For some workloads, you can significantly reduce Amazon EC2 costs by using Spot Instances. For example, suppose that you are running a batch processing job that is able to withstand interruptions. Using a Spot Instance would provide you with up to 90% cost savings while still meeting the availability requirements of your workload.

You can find additional cost savings for Amazon EC2 by considering Savings Plans and Reserved Instances.

Pricing Example

The service charges in this example include details for the following items:

- Each Amazon EC2 instance type that has been used
- The amount of Amazon EBS storage space that has been provisioned
- The length of time that Elastic Load Balancing (ELB) has been used

In this example, all the usage amounts are under the thresholds in the AWS Free Tier, so the account owner would not have to pay for any Amazon EC2 usage in this month.

Elastic Compute Cloud		**$0.00**
US East (N. Virginia)		**$0.00**
Amazon Elastic Compute Cloud running Linux/UNIX		$0.00
$0.00 per Linux t2.micro instance-hour (or partial hour) under monthly free tier	106.512 Hrs	$0.00
EBS		$0.00
$0.00 per GB-month of General Purpose (SSD) provisioned storage under monthly free tier	11.294 GB-Mo	$0.00
Elastic Load Balancing - Application		$0.00
$0.00 per Application LoadBalancer-hour (or partial hour) under monthly free tier	268.000 Hrs	$0.00

Amazon S3

To learn more about Amazon S3 pricing(opens in a new tab), choose each of the following two tabs.

Amazon S3 Pricing

For Amazon S3 pricing, consider the following cost components:

- **Storage -** You pay for only the storage that you use. You are charged the rate to store objects in your Amazon S3 buckets based on your objects' sizes, storage classes, and how long you have stored each object during the month.
- **Requests and data retrievals -** You pay for requests made to your Amazon S3 objects and buckets. For example, suppose that you are storing photo files in Amazon S3 buckets and hosting them on a website. Every time a visitor requests the website that includes these photo files, this counts towards requests you must pay for.

- **Data transfer -** There is no cost to transfer data between different Amazon S3 buckets or from Amazon S3 to other services within the same AWS Region. However, you pay for data that you transfer into and out of Amazon S3, with a few exceptions. There is no cost for data transferred into Amazon S3 from the internet or out to Amazon CloudFront. There is also no cost for data transferred out to an Amazon EC2 instance in the same AWS Region as the Amazon S3 bucket.
- **Management and replication -** You pay for the storage management features that you have enabled on your account's Amazon S3 buckets. These features include Amazon S3 inventory, analytics, and object tagging.

Pricing Example

The AWS account in this example has used Amazon S3 in two Regions: Northern Virginia and Ohio. For each Region, itemized charges are based on the following factors:

- The number of requests to add or copy objects into a bucket
- The number of requests to retrieve objects from a bucket
- The amount of storage space used

All the usage for Amazon S3 in this example is under the AWS Free Tier limits, so the account owner would not have to pay for any Amazon S3 usage in this month.

Simple Storage Service		**$0.00**
US East (N. Virginia)		**$0.00**
Amazon Simple Storage Service Requests-Tier1		$0.00
$0.00 per request - PUT, COPY, POST, or LIST requests under the monthly global free tier	185.000 Requests	$0.00
Amazon Simple Storage Service Requests-Tier2		$0.00
$0.00 per request - GET and all other requests under the monthly global free tier	923.000 Requests	$0.00
Amazon Simple Storage Service TimedStorage-ByteHrs		$0.00
$0.000 per GB - storage under the monthly global free tier	0.159 GB-Mo	$0.00
US East (Ohio)		**$0.00**
Amazon Simple Storage Service USE2-Requests-Tier2		$0.00
$0.00 per request - GET and all other requests under the monthly global free tier	4.000 Requests	$0.00
Amazon Simple Storage Service USE2-TimedStorage-ByteHrs		$0.00
$0.000 per GB - storage under the monthly global free tier	0.000001 GB-Mo	$0.00

Billing Dashboard

Hey there, AWS aficionados. Are you like me and curious about billing information of your AWS account? Well, let's walk through a demo so you know where to go.

As you can see, I'm already logged in and now we're gonna search for billing and click the option that shows up. And boom, with that, you already have some useful

information. You can see your month-to-date spend on the right-hand side with the top services being used. Below on the left is the last month, current and forecasted amounts. Below that is the top Free Tier services by usage. You even get the percentages of your month-to-date usage. Nifty, right?

If we scroll up, you can see you have access to other billing tools, such as Cost Explorer, Budgets, along with a few others. You also get access to your bill itself. So let's click Bills. Now, you can see we have the services that we are paying for, and if we expand them, we get to see the costs broken down by region. And for global services, we see them broken down as such. For example, if I click Route 53, you can see it's a global service and is broken down that way.
And there you have it, a simple way to check out what you're spending on each service in AWS. Thanks for joining us.

Use the **AWS Billing & Cost Management dashboard**(opens in a new tab) to pay your AWS bill, monitor your usage, and analyze and control your costs.

- Compare your current month-to-date balance with the previous month, and get a forecast of the next month based on current usage.
- View month-to-date spend by service.
- View Free Tier usage by service.
- Access Cost Explorer and create budgets.
- Purchase and manage Savings Plans.
- Publish AWS Cost and Usage Report

Consolidated Billing

When you own multiple coffee shops around the city, like we talked about before, each one will have its own expenditures and its own profits. At the end of the day for these coffee shops, they're all owned by the same entity. Because of that, the money is flowing to and from one main account, if there was a coffee machine repair person on call to help fix the coffee machines when they break, that repair person will bill the organization, not the individual coffee shop.

Okay, let's take that idea and apply it to AWS. A singular company will more than likely have multiple AWS accounts, and as you already learned, you can manage multiple accounts in AWS by using AWS Organizations. One of the lovely features of AWS Organizations is called consolidated billing. At the end of every month, instead of having to pay an AWS bill for every single account, you can roll those bills up into one bill owned by the owner of the organization. This makes it easier to keep track of your bills. You don't get 100 bills, if you have 100 AWS accounts, just like our coffee shop example, if my repair person visited 11 of my coffee shops, repairing coffee makers, they would just make one bill for the billing cycle. So it's easier for the company to manage.

Same idea with the consolidated billing feature for AWS Organizations. You can still view your AWS bill in an itemized fashion. So you know which accounts spent what, but it all goes into one central location for ease of viewing. There are other benefits of using this feature too. One of them is that the usage for AWS resources is rolled up to the organization level. AWS does offer bulk pricing. Each individual account may only have a small amount of usage, but you can get the bulk discount pricing because of the aggregate across all accounts in the organization.
In addition, if you have a savings plan in place, or if you are using reserved instances for EC2, it can be shared across AWS accounts in the organization. The best part about this is that the feature is free and easy to use. So it simplifies the billing process, lets you share savings across accounts and doesn't cost you any extra money. Nice.

Consolidated billing

In an earlier module, you learned about AWS Organizations, a service that enables you to manage multiple AWS accounts from a central location. AWS Organizations also provides the option for **consolidated billing**(opens in a new tab).

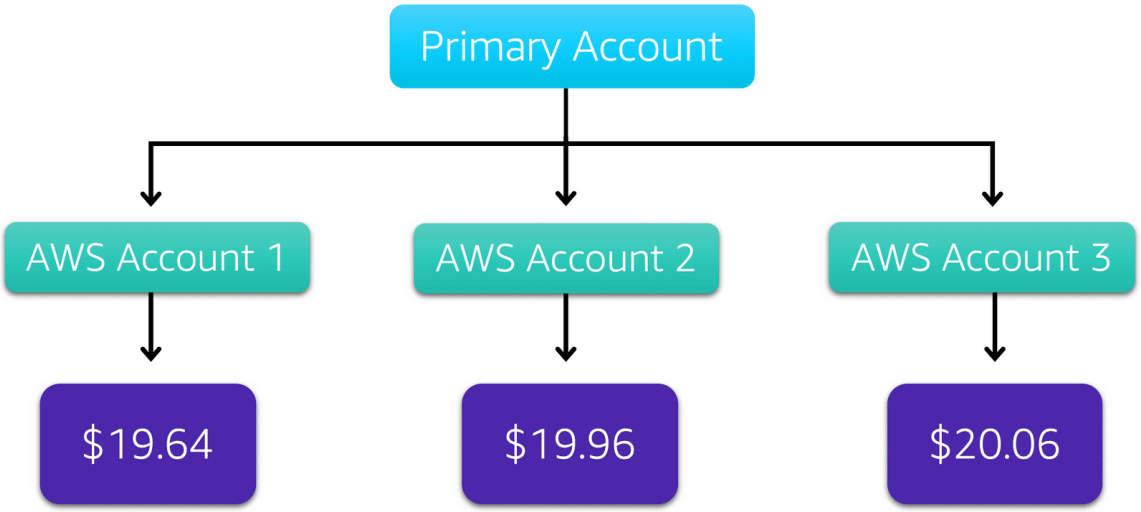
The consolidated billing feature of AWS Organizations enables you to receive a single bill for all AWS accounts in your organization. By consolidating, you can easily track the combined costs of all the linked accounts in your organization. The default maximum number of accounts allowed for an organization is 4, but you can contact AWS Support to increase your quota, if needed.

On your monthly bill, you can review itemized charges incurred by each account. This enables you to have greater transparency into your organization’s accounts while still maintaining the convenience of receiving a single monthly bill.

Another benefit of consolidated billing is the ability to share bulk discount pricing, Savings Plans, and Reserved Instances across the accounts in your organization. For instance, one account might not have enough monthly usage to qualify for discount pricing. However, when multiple accounts are combined, their aggregated usage may result in a benefit that applies across all accounts in the organization.

To review and example of consolidated billing, choose the arrow buttons to display the four steps.

Step 1

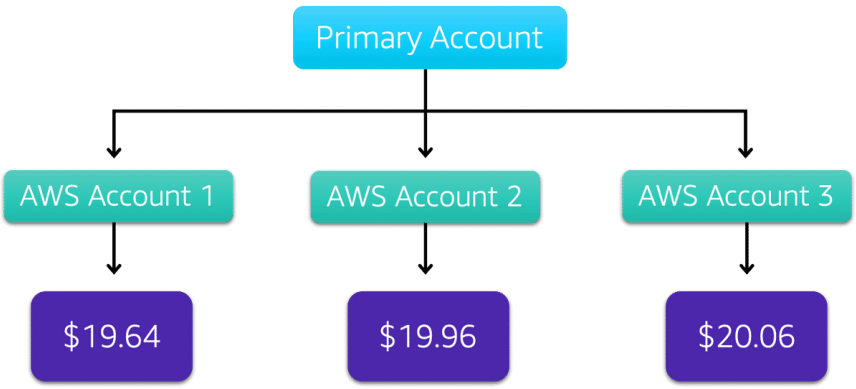


Suppose that you are the business leader who oversees your company's AWS billing.

Your company has three AWS accounts used for separate departments. In this example, Account 1 owes $19.64, Account 2 owes $19.96, and Account 3 owes $20.06. Instead of paying each location's monthly bill separately, you decide to create an organization and add the three accounts.

You manage the organization through the primary account.

Step 2



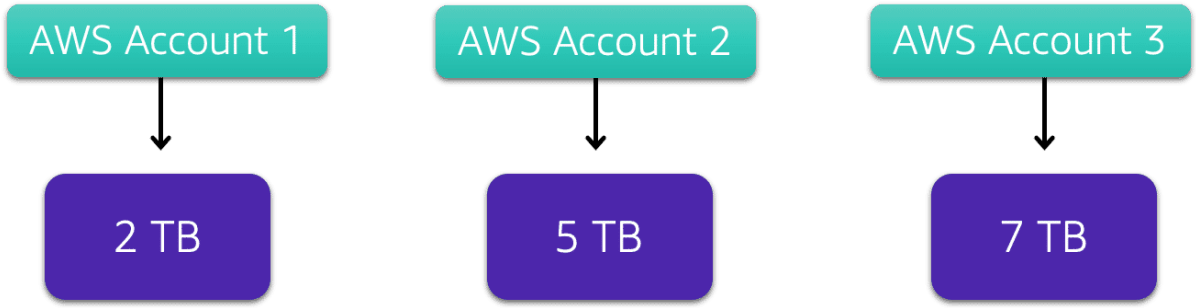
Monthly Consolidated Bill	
Primary Account	$14.14
AWS Account 1	$19.64
AWS Account 2	$19.96
AWS Account 3	$20.06
Total charged to paying account:	**$73.80**

Continuing the example, each month AWS charges your primary payer account for all the linked accounts in a consolidated bill. Through the primary account, you can also get a detailed cost report for each linked account.

The monthly consolidated bill also includes the account usage costs incurred by the primary account. In this case, the primary account incurred $14.14. This cost is not a premium charge for having a primary account.

The consolidated bill shows the costs associated with any actions of the primary account (such as storing files in Amazon S3 or running Amazon EC2 instances). The total charged to the paying account, including the primary account and accounts one through three, is $73.80.

Step 3



Consolidated billing also enables you to share volume pricing discounts across accounts.

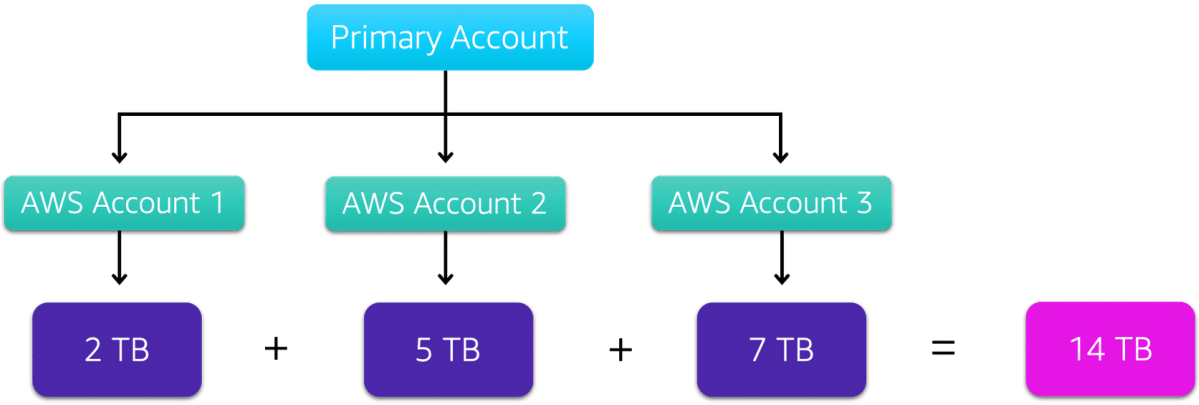
Some AWS services, such as Amazon S3, provide volume pricing discounts that give you lower prices the more that you use the service. In Amazon S3, after customers have transferred 10 TB of data in a month, they pay a lower per-GB transfer price for the next 40 TB of data transferred.

In this example, there are three separate AWS accounts that have transferred different amounts of data in Amazon S3 during the current month:

- Account 1 has transferred 2 TB of data.
- Account 2 has transferred 5 TB of data.
- Account 3 has transferred 7 TB of data.

Because no single account has passed the 10 TB threshold, none of them is eligible for the lower per-GB transfer price for the next 40 TB of data transferred.

Conclusion



Now, suppose that these three separate accounts are brought together as linked accounts within a single AWS organization and are using consolidated billing.

When the Amazon S3 usage for the three linked accounts is combined (2+5+7), this results in a combined data transfer amount of 14 TB. This exceeds the 10-TB threshold.

With consolidated billing, AWS combines the usage from all accounts to determine which volume pricing tiers to apply, giving you a lower overall price whenever possible. AWS then allocates each linked account a portion of the overall volume discount based on the account's usage.

In this example, Account 3 would receive a greater portion of the overall volume discount because at 7 TB, it has transferred more data than Account 1 (at 2 TB) and Account 2 (at 5 TB).

AWS Budgets

AWS Budgets

In **AWS Budgets**(opens in a new tab), you can create budgets to plan your service usage, service costs, and instance reservations.

The information in AWS Budgets updates three times a day. This helps you to accurately determine how close your usage is to your budgeted amounts or to the AWS Free Tier limits.

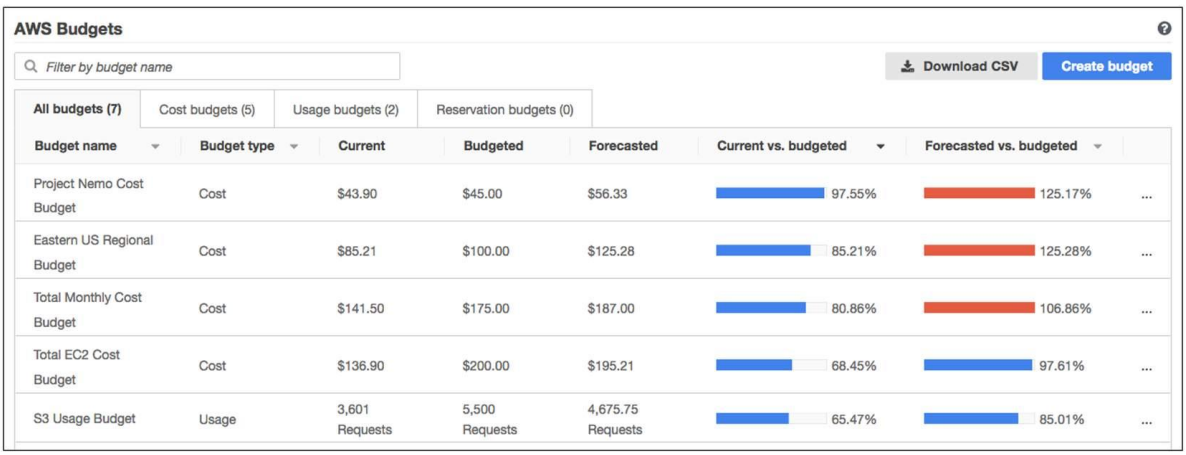
In AWS Budgets, you can also set custom alerts when your usage exceeds (or is forecasted to exceed) the budgeted amount.

Example: AWS Budgets

Suppose that you have set a budget for Amazon EC2. You want to ensure that your company's usage of Amazon EC2 does not exceed $200 for the month.

In AWS Budgets, you could set a custom budget to notify you when your usage has reached half of this amount ($100). This setting would allow you to receive an alert and decide how you would like to proceed with your continued use of Amazon EC2.

To learn more about AWS Budgets, choose each of the numbered markers.



In this sample budget, you can review the following important details:

- The current amount that you have incurred for Amazon EC2 so far this month ($136.90)
- The amount that you are forecasted to spend for the month ($195.21), based on your usage patterns.

You can also review comparisons of your current vs. budgeted usage, and forecasted vs. budgeted usage.

For example, in the top row of this sample budget, the forecasted vs. budgeted bar is at 125.17%. The reason for the increase is that the forecasted amount ($56.33) exceeds the amount that had been budgeted for that item for the month ($45.00).

AWS Cost Explorer

As we have already discussed, AWS has a variable cost model, and you only pay for what you use. You don't have one fixed billed amount at the end of every month. Instead, it fluctuates with the resources you use and how you use them. Because of this cost model, it's really important that you can drill down into your bill and see just how you are spending money.
AWS has a service called the AWS Cost Explorer, and it's a console-based service that allows you to visually see and analyze how you are spending money with AWS. It will show you which services you are spending the most money on, and it gives you 12 months of historical data, so you can track your spending over time. That way, if you see a bump in spending on, say, EC2 from October to December, you can then use that data to go on and figure out why exactly that happened.

Let's take a look at AWS Cost Explorer in my own AWS account. You can see I am logged in to the console, and I'm going to type into the search Cost Explorer. From here, I am brought to the Cost Management dashboard, and I will click on Cost Explorer. You can see this is showing me the last six months of cost associated with my account, and it's currently grouped by service. I can change the timeframe it's showing me to eight months and change the data to be grouped by different attributes, and I'll go ahead and select Region. I can group by other attributes as well.

One important grouping to note is to group by tag. Many resources in AWS are taggable. Tags are essentially user-defined key-value pairs. So you can tag an EC2 instance with a specific project name or a database with the same project name, and then you can come into the AWS Cost Explorer, filter by tag, and see all of the expenses associated with that tag. Cost Explorer also allows you to create custom reports.

So now what I'm going to do is create a report on the daily cost for the month of January of this year. I'm going to group by service. You can see that the cost of the services is generally the same every day, except that I used more EC2 on some days than others. I can then save this report and come back to it when it is needed.

So as you can see, Cost Explorer gives you some powerful defaults for reports, but you can build your own custom ones as well. This will help you identify cost drivers and take

action when necessary to curb spending. Cost optimization is a priority you should be paying close attention to, and you can use the Cost Explorer to help get you going in the right direction.

AWS Cost Explorer

AWS Cost Explorer(opens in a new tab) is a tool that lets you visualize, understand, and manage your AWS costs and usage over time.

AWS Cost Explorer includes a default report of the costs and usage for your top five cost-accruing AWS services. You can apply custom filters and groups to analyze your data. For example, you can view resource usage at the hourly level.

3. **AWS CloudTrail**
4. **AWS Trusted Advisor**
5. **Module 7 Summary**
6. **Module 7 Quiz**

10. Module 8: Pricing and Support
    1. **Module 8 Introduction**
    2. **AWS Free Tier**
    3. **AWS Pricing Concepts**
    4. **Billing Dashboard**
    5. **Consolidated Billing**
    6. **AWS Budgets**
    7. **AWS Cost Explorer**
    8. **AWS Support Plans**
    9. **AWS Marketplace**
    10. **Module 8 Summary**
    11. **Module 8 Quiz**
11. Module 9: Migration and Innovation
    1. **Module 9 Introduction**
    2. **AWS Cloud Adoption Framework (AWS CAF)**
    3. **Migration Strategies**
    4. **AWS Snow Family**
    5. **Innovation with AWS**
    6. **Module 9 Summary**
    7. **Module 9 Quiz**
12. Module 10: The Cloud Journey
    1. **Module 10 Introduction**
    2. **The AWS Well-Architected Framework**
    3. **Benefits of the AWS Cloud**
    4. **Module 10 Summary**
    5. **Module 10 Quiz**
13. Module 11: AWS Certified Cloud Practitioner Basics
    1. **Module 11 Introduction**
    2. **Exam Details**
    3. **Exam Strategies**

14. Final Assessment
    1. **Final Assessment**
15. Contact us
    1. **Contact Us**

Lesson 58 - AWS Budgets

Lesson content

AWS Cost Explorer

To start the video on the AWS Cost Explorer, choose the play button.

Play Video

Play

Loaded: 100.00%

Remaining Time -2:58

1x

Playback Rate 1x

Captions

Picture-in-PictureFullscreen

Mute

To access a transcript of the video, choose the following transcript.

Video transcript

As we have already discussed, AWS has a variable cost model, and you only pay for what you use. You don't have one fixed billed amount at the end of every month. Instead, it fluctuates with the resources you use and how you use them. Because of this cost model, it's really important that you can drill down into your bill and see just how you are spending money.

AWS has a service called the AWS Cost Explorer, and it's a console-based service that allows you to visually see and analyze how you are spending money with AWS. It will show you which services you are spending the most money on, and it gives you 12 months of historical data, so you can track your spending over time. That way, if you see a bump in spending on, say, EC2 from October to December, you can then use that data to go on and figure out why exactly that happened.

Let's take a look at AWS Cost Explorer in my own AWS account. You can see I am logged in to the console, and I'm going to type into the search Cost Explorer. From here, I am brought to the Cost Management dashboard, and I will click on Cost Explorer. You can see this is showing me the last six months of cost associated with my account, and it's currently grouped by service. I can change the timeframe it's showing me to eight months and change the data to be grouped by different attributes, and I'll go ahead and select Region. I can group by other attributes as well.

One important grouping to note is to group by tag. Many resources in AWS are taggable. Tags are essentially user-defined key-value pairs. So you can tag an EC2 instance with a specific project name or a database with the same project name, and then you can come into the AWS Cost Explorer, filter

by tag, and see all of the expenses associated with that tag. Cost Explorer also allows you to create custom reports.

So now what I'm going to do is create a report on the daily cost for the month of January of this year. I'm going to group by service. You can see that the cost of the services is generally the same every day, except that I used more EC2 on some days than others. I can then save this report and come back to it when it is needed.

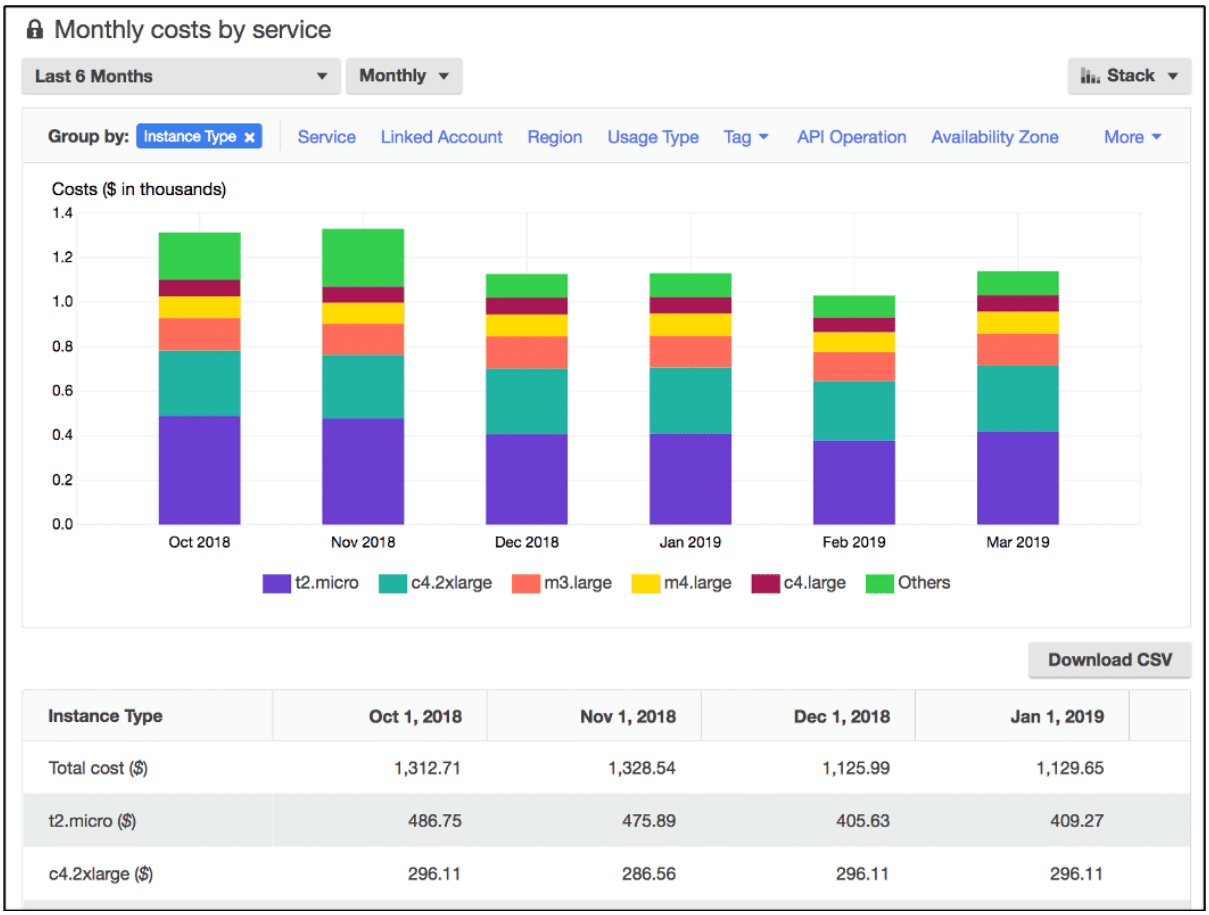
So as you can see, Cost Explorer gives you some powerful defaults for reports, but you can build your own custom ones as well. This will help you identify cost drivers and take action when necessary to curb spending. Cost optimization is a priority you should be paying close attention to, and you can use the Cost Explorer to help get you going in the right direction.

AWS Cost Explorer

AWS Cost Explorer(opens in a new tab) is a tool that lets you visualize, understand, and manage your AWS costs and usage over time.

AWS Cost Explorer includes a default report of the costs and usage for your top five cost-accruing AWS services. You can apply custom filters and groups to analyze your data. For example, you can view resource usage at the hourly level.

Example: AWS Cost Explorer



Instance Type	Oct 1, 2018	Nov 1, 2018	Dec 1, 2018	Jan 1, 2019
Total cost ($)	1,312.71	1,328.54	1,125.99	1,129.65
t2.micro ($)	486.75	475.89	405.63	409.27
c4.2xlarge ($)	296.11	286.56	296.11	296.11

This example of the AWS Cost Explorer dashboard displays monthly costs for Amazon EC2 instances over a 6-month period. The bar for each month separates the costs for different Amazon EC2 instance types, such as t2.micro or m3.large.

By analyzing your AWS costs over time, you can make informed decisions about future costs and how to plan your budgets.

AWS Support Plans

One of the great things about AWS is no matter how big or small your business is, you're never alone. From small startups to large enterprises, private sector and public, all businesses have support options available that are designed to fit your specific needs.

To begin with, every customer automatically gets AWS Basic Support, no cost at all. Any customer can access support functions like 24/7 access to customer service, documentation, whitepapers, support forums, AWS Trusted Advisor, and the AWS Personal Health Dashboard. It's a personalized view of the health of AWS services and any alerts when your resources might be impacted. These functions are free for everyone. But as you begin to move critical workloads into AWS, we offer higher levels of support to match your levels of need.

The next Support plan is our AWS Developer Support, which includes everything in the Basic Support level. Plus, you can now email customer support directly with a 24-hour response time on any questions you have. And responses of less than 12 hours in case your systems are impaired. This is a great plan for businesses that are experimenting with AWS or setting up tests or proofs of concept.

Now, as you begin to take production workloads live, we find customers are more successful advancing to AWS Business Support. Everything in the previous plans is included, plus Trusted Advisor now opens up the entire suite of checks for your account. You are given direct phone access to our support team. They have a 4-hour response time if your production system is impaired, and a 1-hour response time for when you production system is down. Additionally, as part of the Business Support plan, we provide access to infrastructure event management. For an extra fee, we can help you plan for massive events like brand new launches or global advertising blitzes.

Next, for companies migrating production and business-critical workloads to AWS, we recommend AWS Enterprise On-Ramp. It has everything in the previous plans plus a 30-minute response time for business-critical workloads and access to a pool of technical account managers, or TAMs. They provide proactive guidance and coordinate access to programs and AWS experts as needed.

Finally, for companies running mission-critical workloads, we recommend the AWS Enterprise Support. It has everything in the previous plans, plus a 15-minute response time for business and mission-critical workloads and a designated TAM. Your TAM will proactively monitor your environment and assist with optimization. AWS Enterprise Support also includes access to proactive reviews, workshops, and deep dives.

Let's talk a bit more specifically about the job of a TAM. TAMs are part of the concierge support team that comes with both Enterprise Support options. They provide infrastructure event management, Well-Architected reviews, and operations reviews. In Enterprise On-Ramp, your engagement with TAMs is rate limited, whereas rate limits don’t apply in AWS Enterprise Support.

What's a Well-Architected review, you ask? Well, keeping it simple, TAMs work with customers to review architectures using the Well-Architected Framework. Architectures are checked against the six pillars of the Well-Architected Framework: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability. As you can see, the job of a TAM is much more than just handling trouble tickets. AWS Support looks at the customer holistically, not just if they have problems, but how can we help them be successful?

And that's the mission of AWS Support. To learn more about AWS Support, including pricing structures for the plans, go to aws.amazon.com/premiumsupport.

AWS Support

AWS offers four different **Support plans**(opens in a new tab) to help you troubleshoot issues, lower costs, and efficiently use AWS services.

You can choose from the following Support plans to meet your company's needs:

- Basic
- Developer
- Business
- Enterprise On-Ramp
- Enterprise

Basic Support

Basic Support is free for all AWS customers. It includes access to whitepapers, documentation, and support communities. With Basic Support, you can also contact AWS for billing questions and service limit increases.

With Basic Support, you have access to a limited selection of AWS Trusted Advisor checks. Additionally, you can use the **AWS Personal Health Dashboard**, a tool that provides alerts and remediation guidance when AWS is experiencing events that may affect you.

If your company needs support beyond the Basic level, you could consider purchasing Developer, Business, Enterprise On-Ramp, and Enterprise Support.

Developer, Business, Enterprise On-Ramp, and Enterprise Support

The Developer, Business, Enterprise On-Ramp, and Enterprise Support plans include all the benefits of Basic Support, in addition to the ability to open an unrestricted number of technical support cases. These Support plans have pay-by-the-month pricing and require no long-term contracts.

The information in this course highlights only a selection of details for each Support plan. A complete overview of what is included in each Support plan, including pricing for each plan, is available on the AWS Support(opens in a new tab) site.

In general, for pricing, the Developer plan has the lowest cost, the Business and Enterprise On-Ramp plans are in the middle, and the Enterprise plan has the highest cost.

To learn more about AWS support plans, expand each of the following four categories.

Developer Support

Customers in the **Developer Support** plan have access to features such as:

- Best practice guidance
- Client-side diagnostic tools
- Building-block architecture support, which consists of guidance for how to use AWS offerings, features, and services together

For example, suppose that your company is exploring AWS services. You've heard about a few different AWS services. However, you're unsure of how to potentially use them together to build applications that can address your company's needs. In this scenario, the building-block architecture support that is included with the Developer Support plan could help you to identify opportunities for combining specific services and features.

Business Support

Customers with a **Business Support** plan have access to additional features, including:

- Use-case guidance to identify AWS offerings, features, and services that can best support your specific needs
- All AWS Trusted Advisor checks
- Limited support for third-party software, such as common operating systems and application stack components

Suppose that your company has the Business Support plan and wants to install a common third-party operating system onto your Amazon EC2 instances. You could contact AWS Support for assistance with installing, configuring, and troubleshooting the operating system. For advanced topics such as optimizing performance, using custom scripts, or resolving security issues, you may need to contact the third-party software provider directly.

Enterprise On-Ramp Support

In November 2021, AWS opened enrollment into AWS Enterprise On-Ramp Support plan. In addition to all the features included in the Basic, Developer, and Business Support plans, customers with an Enterprise On-Ramp Support plan have access to:

- A pool of Technical Account Managers to provide proactive guidance and coordinate access to programs and AWS experts
- A Cost Optimization workshop (one per year)
- A Concierge support team for billing and account assistance
- Tools to monitor costs and performance through Trusted Advisor and Health API/Dashboard

Enterprise On-Ramp Support plan also provides access to a specific set of proactive support services, which are provided by a pool of Technical Account Managers.

- Consultative review and architecture guidance (one per year)
- Infrastructure Event Management support (one per year)
- Support automation workflows
- 30 minutes or less response time for business-critical issues

Enterprise Support

In addition to all features included in the Basic, Developer, Business, and Enterprise On-Ramp support plans, customers with Enterprise Support have access to:

- A designated Technical Account Manager to provide proactive guidance and coordinate access to programs and AWS experts
- A Concierge support team for billing and account assistance
- Operations Reviews and tools to monitor health
- Training and Game Days to drive innovation
- Tools to monitor costs and performance through Trusted Advisor and Health API/Dashboard

The Enterprise plan also provides full access to proactive services, which are provided by a designated Technical Account Manager:

- Consultative review and architecture guidance
- Infrastructure Event Management support
- Cost Optimization Workshop and tools
- Support automation workflows
- 15 minutes or less response time for business-critical issues

Technical Account Manager (TAM)

The Enterprise On-Ramp and Enterprise Support plans include access to a **Technical Account Manager (TAM)**.

The TAM is your primary point of contact at AWS. If your company subscribes to Enterprise Support or Enterprise On-Ramp, your TAM educates, empowers, and evolves your cloud journey across the full range of AWS services. TAMs provide expert engineering guidance, help you design solutions that efficiently integrate AWS services, assist with cost-effective and resilient architectures, and provide direct access to AWS programs and a broad community of experts.

For example, suppose that you are interested in developing an application that uses several AWS services together. Your TAM could provide insights into how to best use the services together. They achieve this, while aligning with the specific needs that your company is hoping to address through the new application.

AWS Marketplace

Another huge way that AWS supports your enterprise is through the AWS Marketplace. The AWS Marketplace is a curated digital catalog that streamlines your steps to find, deploy and manage third party software running in your AWS architecture, all while providing a range of payment options, and this allows you to accelerate innovation while rapidly and securely deploying a wide range of solutions, while also reducing your total cost of ownership.

To help us understand this more, I wanna talk about what the AWS Marketplace brings to the table, and how it can help you improve your speed and agility with AWS. The first key way that the AWS Marketplace helps customers, is that instead of needing to build, install and maintain the foundational infrastructure needed to run these third party applications in the marketplace, customers have options like one-click deployment that allows them to quickly procure and use products from thousands of software sellers right when you need them.

Now, as you may know, some third party vendors historically have annual contracts for on-premises data centers. You might be wondering, is it the same with an AWS? Well, every vendor can choose what options are available, but almost every vendor in the marketplace will allow you to use any annual licenses you already own and credit them for AWS deployment. But more importantly, as you move forward, most vendors in the marketplace also offer on-demand pay-as-you-go options. These flexible pricing plans give you more options to pay for the software, the way you actually use it without wasted unused licenses weighing down your balance sheets.

Many vendors even offer free trials or Quick Start plans to help you experiment and learn about their offerings. Because they know just like AWS itself, the best way to see if something works for you is to just run it. Even for large enterprises that are used to negotiating custom rates and discount vendors, well the AWS Marketplace can help. The marketplace offers a number of enterprise-focused features, such as custom terms and pricing, where you can manage custom licensing agreements. A private marketplace, where you can create a custom catalog of pre-approved software solutions that meet your specific legal or security standards. Integration into your procurement systems, and a range of cost management tools just to name a few benefits. To learn more about the AWS Marketplace, go to aws.amazon.com/marketplace.

AWS Marketplace

AWS Marketplace(opens in a new tab) is a digital catalog that includes thousands of software listings from independent software vendors. You can use AWS Marketplace to find, test, and buy software that runs on AWS.

For each listing in AWS Marketplace, you can access detailed information on pricing options, available support, and reviews from other AWS customers.

You can also explore software solutions by industry and use case. For example, suppose your company is in the healthcare industry. In AWS Marketplace, you can review use cases that software helps you to address, such as implementing solutions to protect patient records or using machine learning models to analyze a patient's medical history and predict possible health risks.

AWS Marketplace categories

AWS Marketplace offers products in several categories, such as Infrastructure Software, DevOps, Data Products, Professional Services, Business Applications, Machine Learning, Industries, and Internet of Things (IoT).

Within each category, you can narrow your search by browsing through product listings in subcategories. For example, subcategories in the DevOps category include areas such as Application Development, Monitoring, and Testing.

AWS Cloud Adoption Framework (AWS CAF)

Migrating to the cloud is a process. You don't just snap your fingers and have everything magically hosted in AWS. It takes a lot of effort to get applications migrated to AWS, and having a successful cloud migration is something that requires expertise.

Luckily, many people have had successful cloud migrations in the past, and you're not the first one to have solutions in AWS. Because of this, a lot of the knowledge around how to go about hosting on AWS has been captured and shared.

That being said, what position you hold in your organization will impact the things that you need to know or help with for your migration. If you are a developer, your role and viewpoint will be much different than a cloud architect, business analyst or financial analyst. Different types of people bring different perspectives to the table for migration, and you want to harness those different perspectives and make sure everyone is on the same page.

You also want to ensure that you have the right talent to help support your migration, so HR will need to hire at the correct rate to enable your migration. This can be a lot to keep track of, and someone new to the cloud might not think of all the different people who need to be involved.

The AWS professional services team has created something called the AWS Cloud Adoption Framework that can help you manage this process through guidance. The Cloud Adoption Framework exists to provide advice to your company to enable a quick and smooth migration to AWS.

The framework organizes guidance into six areas focused on the different types of people that you'll need to involve for your migration. Just like the different perspectives that we talked about earlier. Each Perspective covers distinct responsibilities covered by different groups. The different Perspectives are Business, People, and Governance Perspectives, which focus on the business capabilities, and then you have the Platform, Security and Operations Perspectives which focus on the technical capabilities. Someone who is a business or finance analysts would fall under the Business Perspective, HR would fall under the People Perspective, and a cloud architect would fall under the Platform Perspective.

Each Perspective is used to uncover gaps in your skills and processes, which are then recorded as inputs. These inputs are then used as the basis for creating what is called an AWS Cloud Adoption Framework Action Plan that you then use to guide your organization's change management as you journey to the cloud. Having an action plan that makes sense for your organization can help keep you on track.

Migrating to the cloud can be complicated, but again, you're not alone in this, there are tons of resources to help you get started, and the Cloud Adoption Framework is a great place to look.

Six core perspectives of the Cloud Adoption Framework

At the highest level, the **AWS Cloud Adoption Framework (AWS CAF)**(opens in a new tab) organizes guidance into six areas of focus, called **Perspectives**. Each Perspective addresses distinct responsibilities. The planning process helps the right people across the organization prepare for the changes ahead.

In general, the **Business**, **People**, and **Governance** Perspectives focus on business capabilities, whereas the **Platform**, **Security**, and **Operations** Perspectives focus on technical capabilities.

To learn more about the AWS CAF, expand each of the following six categories.

Business Perspective

The **Business Perspective** ensures that IT aligns with business needs and that IT investments link to key business results.

Use the Business Perspective to create a strong business case for cloud adoption and prioritize cloud adoption initiatives. Ensure that your business strategies and goals align with your IT strategies and goals.

Common roles in the Business Perspective include:

- Business managers
- Finance managers
- Budget owners
- Strategy stakeholders

People Perspective

The **People Perspective** supports development of an organization-wide change management strategy for successful cloud adoption.

Use the People Perspective to evaluate organizational structures and roles, new skill and process requirements, and identify gaps. This helps prioritize training, staffing, and organizational changes.

Common roles in the People Perspective include:

- Human resources
- Staffing
- People managers

Governance Perspective

The **Governance Perspective** focuses on the skills and processes to align IT strategy with business strategy. This ensures that you maximize the business value and minimize risks.

Use the Governance Perspective to understand how to update the staff skills and processes necessary to ensure business governance in the cloud. Manage and measure cloud investments to evaluate business outcomes.

Common roles in the Governance Perspective include:

- Chief Information Officer (CIO)
- Program managers
- Enterprise architects
- Business analysts
- Portfolio managers

Platform Perspective

The **Platform Perspective** includes principles and patterns for implementing new solutions on the cloud, and migrating on-premises workloads to the cloud.

Use a variety of architectural models to understand and communicate the structure of IT systems and their relationships. Describe the architecture of the target state environment in detail.

Common roles in the Platform Perspective include:

- Chief Technology Officer (CTO)
- IT managers
- Solutions architects

Security Perspective

The **Security Perspective** ensures that the organization meets security objectives for visibility, auditability, control, and agility.

Use the AWS CAF to structure the selection and implementation of security controls that meet the organization's needs.

Common roles in the Security Perspective include:

- Chief Information Security Officer (CISO)
- IT security managers
- IT security analysts

Operations Perspective

The **Operations Perspective** helps you to enable, run, use, operate, and recover IT workloads to the level agreed upon with your business stakeholders.

Define how day-to-day, quarter-to-quarter, and year-to-year business is conducted. Align with and support the operations of the business. The AWS CAF helps these stakeholders define current operating procedures and identify the process changes and training needed to implement successful cloud adoption.

Common roles in the Operations Perspective include:

- IT operations managers
- IT support managers

Migration Strategies

So it is finally time to migrate from your on-premises deployment onto AWS. Like Morgan said in the prior video, it would be great to just snap your fingers and instantly have all the elements from your existing on-prem data center magically appear on AWS, optimized and efficient. But it doesn't work that way.

Every application, or application groups, if they're tightly coupled, will have six possible options when it comes to your enterprise migration. We call these the six R's. Once you've gone through the discovery phase and know exactly what you have in your existing environment, you decide which option among the six R's is the best fit based on time, cost, priority, criticality.

The first option is Rehosting. This is otherwise known as lift and shift. And this is an easy thing for businesses to do because you're not making any changes. At least not at first. Just pick up the applications and move them pretty much as is onto AWS. You may not get all the possible benefits. But some companies found that even without any optimization, they could save up to 30% of their total costs just by rehosting. Also, we find it's easier to optimize applications later once they already live in the cloud. Because your organization has better skills to do so. The hard part, the migration, is already complete.

The second option is called Replatforming. Or lift, tinker, and shift. It's still basically a lift and shift, but instead of a pure one-to-one, you might make a few cloud optimizations. But you're not touching any core code in the process. No new dev efforts are involved here. For example, you could take your existing MySQL database and replatform it onto RDS MySQL, without any code changes at all. Or even consider upgrading to Amazon Aurora. This gives significant benefit to your DBA team as well as improved performance without any code changes.

Now, before we add more migration options two of the six R's don't actually end up on AWS. Number three, Retire. Some parts of your enterprise IT portfolio are just no longer needed. We found as much as 10% to 20% of companies application portfolios include applications that are no longer being used or already have replacements live and functional. Using the AWS migration plan as the opportunity to actually end-of-life these applications can save significant cost and effort for your team. Sometimes you just have to turn off the lights.

Number four, Retain. Some applications are about to be deprecated but maybe not just yet. They still need to run but don't turn them off for another three months or eight months. These apps could be migrated to AWS, but why? You should only migrate what makes sense for your business. And then as time goes on, these applications can be deprecated where they live, and ultimately retired. Okay, let's get back to what is going to move to AWS.

Number five, Repurchase. This is common for companies looking to abandon legacy software vendors and get a fresh start as part of migration. For example, ending a contract with an old CRM vendor and moving to a brand new one. Or perhaps finally ending your licensing with an out of date database vendor in favor of cloud native database offerings. Now, this sounds great, but remember, you'll now be dealing with a new software package and some are easy to implement, some take time. The total upfront expense of the step therefore goes up, but the potential benefits could be substantial.

Number six, Refactoring. Now, you're writing new code. This is driven by a strong business need to add features or performance that might not be possible on prem, but now are within your reach. Dramatic changes to your architecture can be very beneficial to your enterprise but this will come at the highest initial cost in terms of planning and human effort.

Six paths to move to AWS. Once you have the inventory of all your pieces, choosing the right one for each part is critical to the success of your migration.

6 strategies for migration

When migrating applications to the cloud, six of the most common migration strategies(opens in a new tab) that you can implement are:

- Rehosting
- Replatforming
- Refactoring/re-architecting
- Repurchasing
- Retaining
- Retiring

To learn more about migration stretiges, expand each of the following six categories.

Rehosting

Rehosting also known as “lift-and-shift” involves moving applications without changes. In the scenario of a large legacy migration, in which the company is looking to implement its migration and scale quickly to meet a business case, the majority of applications are rehosted

Replatforming

Replatforming, also known as “lift, tinker, and shift,” involves making a few cloud optimizations to realize a tangible benefit. Optimization is achieved without changing the core architecture of the application.
\

Refactoring/re-architecting

Refactoring (also known as **re-architecting**) involves reimagining how an application is architected and developed by using cloud-native features. Refactoring is driven by a strong business need to add features, scale, or performance that would otherwise be difficult to achieve in the application’s existing environment.

Repurchasing

Repurchasing involves moving from a traditional license to a software-as-a-service model.

For example, a business might choose to implement the repurchasing strategy by migrating from a customer relationship management (CRM) system to Salesforce.com.

Retaining

Retaining consists of keeping applications that are critical for the business in the source environment. This might include applications that require major refactoring before they can be migrated, or, work that can be postponed until a later time.

Retiring

Retiring is the process of removing applications that are no longer needed.

AWS Snow Family

Some of our customers need to get data to AWS and most of them would like to do it in an efficient and timely manner. The usual route is to simply copy the required data over the internet or better yet, if they have a Direct Connect line. However, with the limitations of bandwidth, in general, this can take days, weeks, or even months.

For example, a dedicated one gigabyte per second network connection theoretically moves one petabyte of data in about 100 days and in the real world likely longer and at a higher cost, with this customer feedback and as a way to address this gap, we therefore introduce AWS Snow Family of devices.

The first device, which is also our newest offering, is called AWS Snowcone and it's a device that holds up to eight terabytes of data and contains edge computing. Edge computing options are Amazon EC2 instances and AWS IoT Greengrass. To obtain one, you place an order via AWS Management Console we ship it to you, you plug it in and copy your data, and finally, ship it back to us. We'll then copy the data to your AWS

account, usually an Amazon S3 bucket that you own, and boom, Bob is your uncle and your data is available to use. Customers usually use these devices to ship terabytes of information such as analytics data, video libraries, image collections, backups, and even tape replacement data. I mean, who doesn't have a terabyte collection of cat images, that they want to backup to the AWS Cloud?

But what if eight terabytes is not enough? Well, good news, we have a product for that requirement called Snowball Edge. It comes in two versions: a Snowball Edge Compute Optimized option and a Snowball Edge Storage Optimized option. Even better is that they fit into existing server racks and can be clustered for greater computing needs. Ones you plug them into your infrastructure, you can even run AWS Lambda functions, Amazon EC2-compatible AMI's, or even AWS IoT Greengrass to perform simple processing of data right then and there. Customers usually ship these to remote locations, where it's trickier to have a lot of computing power lying around. The use cases include capturing of streams from IoT devices, image compression, video transcoding, and even industrial signaling.

The last device that's up for grabs is also the largest in our fleet, the AWS Snowmobile, and as the name suggests, it's housed in a 45-foot rugged shipping container and pulled along by a freaking truck, I mean this thing is huge. It houses 100 petabytes and is ideal for the largest migrations and even data center shutdowns. We drive the truck to your designated location, plug it in, and it then appears as a network attached storage device with, like I said, capacity of up to 100 petabytes. It is tamper resistant, waterproof, temperature controlled, it even has fire suppression and GPS tracking. I mean, can you believe that it also has 24/7 video surveillance with a dedicated security team and escort security vehicle during transit? That's some serious business.

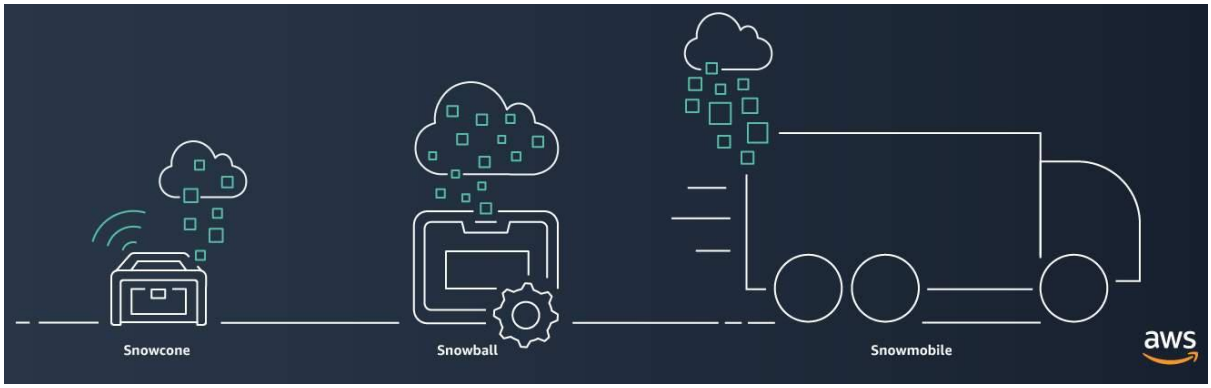
But it should be noted that all Snow Family devices are designed to be secure and tamper-resistant while on-site or in transit. This means the hardware and software is cryptographically signed, and all data stored is automatically encrypted using 256-bit encryption keys, owned and managed by you, the customer. You can even use AWS Key Management Service to generate and manage keys.

As always, it's been a pleasure taking you through yet another AWS service. Hope you've had fun learning about the AWS Snow Family.

AWS Snow Family members

The **AWS Snow Family**(opens in a new tab) is a collection of physical devices that help to physically transport up to exabytes of data into and out of AWS.

AWS Snow Family is composed of **AWS Snowcone**, **AWS Snowball**, and **AWS Snowmobile**.



These devices offer different capacity points, and most include built-in computing capabilities. AWS owns and manages the Snow Family devices and integrates with AWS security, monitoring, storage management, and computing capabilities.

To learn about each category, choose each of the following three tabs.

AWS Snowcone

AWS Snowcone(opens in a new tab) is a small, rugged, and secure edge computing and data transfer device.

It features 2 CPUs, 4 GB of memory, and up to 14 TB of usable storage

AWS Snowball

AWS Snowball(opens in a new tab) offers two types of devices:

- **Snowball Edge Storage Optimized** devices are well suited for large-scale data migrations and recurring transfer workflows, in addition to local computing with higher capacity needs.
    - Storage: 80 TB of hard disk drive (HDD) capacity for block volumes and Amazon S3 compatible object storage, and 1 TB of SATA solid state drive (SSD) for block volumes.
    - Compute: 40 vCPUs, and 80 GiB of memory to support Amazon EC2 sbe1 instances (equivalent to C5).
- **Snowball Edge Compute Optimized** provides powerful computing resources for use cases such as machine learning, full motion video analysis, analytics, and local computing stacks.
    - Storage: 80-TB usable HDD capacity for Amazon S3 compatible object storage or Amazon EBS compatible block volumes and 28 TB of usable NVMe SSD capacity for Amazon EBS compatible block volumes.
    - Compute: 104 vCPUs, 416 GiB of memory, and an optional NVIDIA Tesla V100 GPU. Devices run Amazon EC2 sbe-c and sbe-g instances, which are equivalent to C5, M5a, G3, and P3 instances.

AWS Snowmobile

AWS Snowmobile(opens in a new tab) is an exabyte-scale data transfer service used to move large amounts of data to AWS.

You can transfer up to 100 petabytes of data per Snowmobile, a 45-foot long ruggedized shipping container, pulled by a semi trailer truck.

Innovation with AWS

There's so much more that can be done on AWS that we have time to talk about here. For example, when it comes to migrating onto AWS, we didn't even cover running VMware on AWS. The same VMware based infrastructure that you use on prem can in many cases, just be lifted up and dropped onto AWS via VMware Cloud on AWS. And this is just one of many concepts that make AWS a place where builders go to create and innovate at the pace of their ideas.

When it comes to machine learning and artificial intelligence, AWS has the broadest and deepest set of machine learning and AI services for your business. You can choose from pre-trained AI services for computer vision, language recommendations, and forecasting. Amazon SageMaker: Quickly build, train, and deploy machine learning models at scale.

Or build custom models with support for all the popular open-source frameworks. Our capabilities are built on the most comprehensive cloud platform, optimized for machine learning with high performance compute and no compromises on security and analytics. Tools like Amazon SageMaker and Amazon Augmented AI, or Amazon A2I, provide a machine learning platform that any business can build upon without needing PhD level expertise in-house. Or perhaps, ready-to-go AI solutions like Amazon Lex, the heart of Alexa.

[Alexa] Hello . What can I help you with?

Helps you build interactive chat bots.

Or what about Amazon Textract. Extracting text and data from documents to make them more usable for your enterprise instead of them just being locked away in a repository.

Do you wanna put machine learning literally into the hands of your developers? Why not try AWS DeepRacer? A chance for your developers to experiment with reinforcement learning. One of the newest branches of machine learning algorithms, all while having fun in a racing environment.

AWS offers brand new technologies in things like Internet of Things. Enabling connected devices to communicate all around the world.

Speaking of communication around the world, have you ever wanted to have your own satellite? Too expensive to launch your own? Why not just use AWS Ground Station and only pay for the satellite time you actually need?

I could go on for days talking about all these new technologies. Like literally for days. AWS Training and Certification offers training classes on many of these technologies already, and every month, AWS seems to release something even better for us to talk about.

Innovate with AWS Services

When examining how to use AWS services, it is important to focus on the desired outcomes. You are properly equipped to drive innovation in the cloud if you can clearly articulate the following conditions:

- The current state
- The desired state
- The problems you are trying to solve

Consider some of the paths you might explore in the future as you continue on your cloud journey.

To learn more on ways to innovate with AWS, expand each of the following three categories.

Serverless applications

With AWS, **serverless** refers to applications that don't require you to provision, maintain, or administer servers. You don't need to worry about fault tolerance or availability. AWS handles these capabilities for you.

AWS Lambda is an example of a service that you can use to run serverless applications. If you design your architecture to trigger Lambda functions to run your code, you can bypass the need to manage a fleet of servers.

Building your architecture with serverless applications enables your developers to focus on their core product instead of managing and operating servers.

Artificial intelligence

AWS offers a variety of services powered by **artificial intelligence (AI)**.

For example, you can perform the following tasks:

- Convert speech to text with Amazon Transcribe.
- Discover patterns in text with Amazon Comprehend.
- Identify potentially fraudulent online activities with Amazon Fraud Detector.
- Build voice and text chatbots with Amazon Lex.

Machine learning

Traditional **machine learning (ML)** development is complex, expensive, time consuming, and error prone. AWS offers Amazon SageMaker to remove the difficult work from the process and empower you to build, train, and deploy ML models quickly.

You can use ML to analyze data, solve complex problems, and predict outcomes before they happen.

Amazon Q Developer

Amazon Q Developer analyzes your comments and code as you write them in your integrated development environment (IDE). It goes beyond code completion by using natural language processing to comprehend the comments in your code. By understanding English comments, Amazon Q Developer generates complete functions and code blocks that align with your descriptions. Amazon Q Developer also analyzes the surrounding code, ensuring the generated code matches your style and naming conventions and seamlessly integrates into the existing context.
When scanning for security vulnerabilities, Amazon Q Developer assesses your code against multiple sets of standards and best practices. This includes the following:

- Open Worldwide Application Security Project (OWASP) standards
- Crypto library best practices
- AWS security standards

The security scan feature is continuously updated to help keep applications free from new security vulnerabilities.

The code generation feature of Amazon Q Developer offers code suggestions in real-time in your development environment. It automatically offers code completion and code generation suggestions. It uses natural language processing of English comments in your code and an understanding of surrounding code to suggest whole lines of code, complete functions, and logical blocks of code. The generated code is aligned with your coding style and naming conventions. Amazon Q Developer prioritizes secure coding and responsible artificial intelligence (responsible AI) practices. It is optimized for Amazon APIs and trained extensively on Amazon and open-source code. You have the option to accept the first suggestion, explore more suggestions, or continue writing your own code. It is important to review each code suggestion before accepting it, because you might need to make edits to ensure the suggestion aligns with your intended functionality.

Amazon Q Developer learns from open-source projects and the code it suggests might occasionally resemble code samples from the training data. With the reference log, you can view references to code suggestions that are similar to the training data. When such occurrences happen, Amazon Q Developer notifies you and provides repository and licensing information. Use this information to make decisions about whether to use the code in your project and properly attribute the source code as desired.

The security scanning feature of Amazon Q Developer detects security vulnerabilities in both Amazon Q Developer-generated code and developer-written code. It scans the code to identify potential vulnerabilities and provides suggestions for remediation. This includes scanning for hard-to-find vulnerabilities that might be overlooked. The security scan is compatible with popular IDEs such as VS Code and JetBrains. It supports Python, Java, and JavaScript.

Automated code generation automates repetitive tasks and saves you time. It eliminates the need for you to invest excessive hours in exploring and learning new technologies. Instead, you can rely on high-quality code suggestions that match your coding style. This approach enhances your productivity so you can focus on critical tasks, which encourages innovation and progress in software development. With automated code generation, you can streamline your workflows and achieve significant time savings while ensuring the delivery of code that meets your standards.

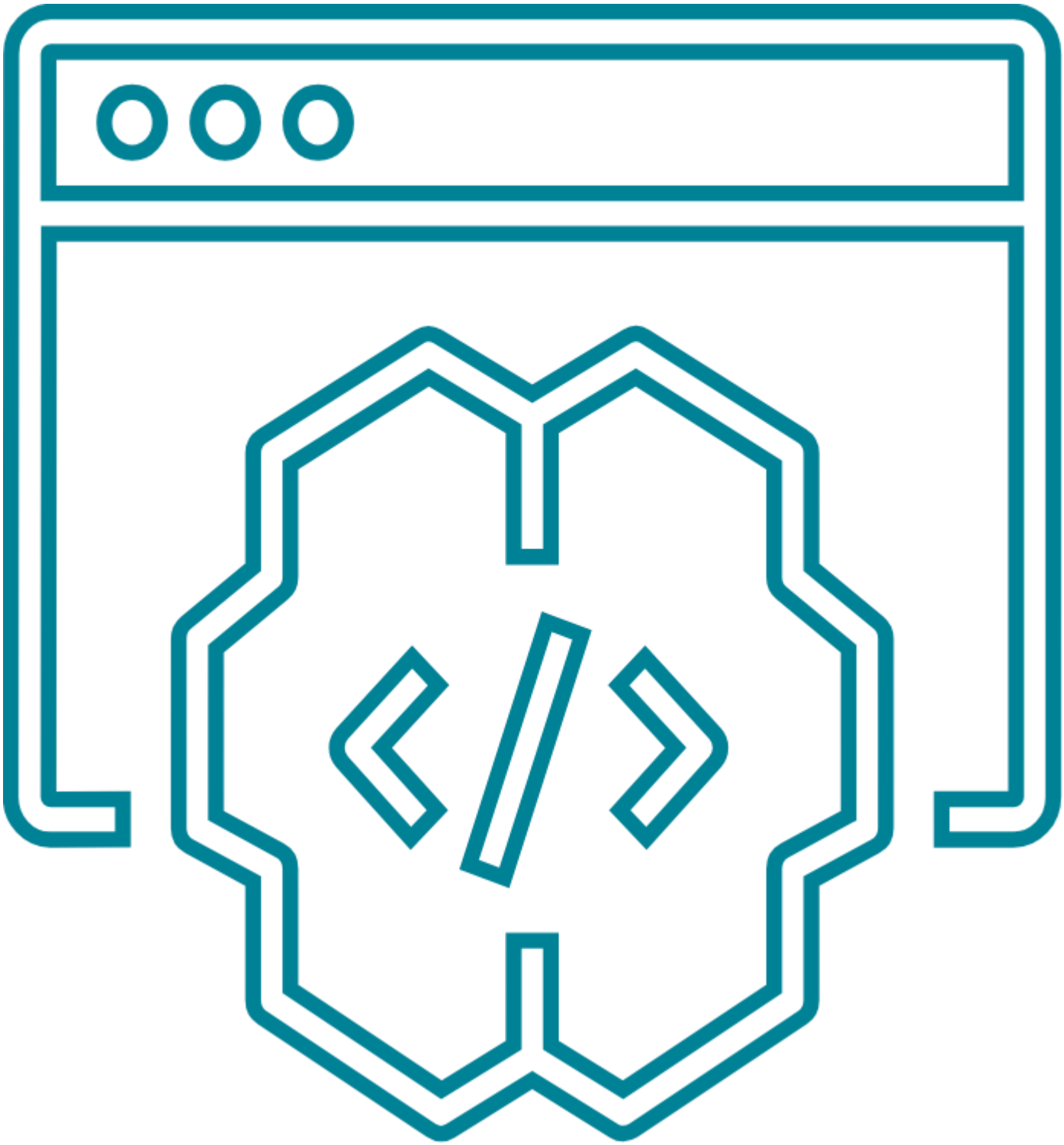
Amazon Q Developer code generation offers many benefits for software development organizations. It accelerates application development for faster delivery of software solutions. By automating repetitive tasks, it optimizes the use of developer time, so developers can focus on more critical aspects of the project. Additionally, code generation helps mitigate security vulnerabilities, safeguarding the integrity of the codebase.

Amazon Q Developer also protects open source intellectual property by providing the open source reference tracker. Amazon Q Developer enhances code quality and reliability, leading to robust and efficient applications. And it supports an efficient response to evolving software threats, keeping the codebase up to date with the latest security practices. Amazon Q Developer has the potential to increase development speed, security, and the quality of software.

What is Amazon Q Developer?

When we show up to the present moment with all of our senses, we invite the world to fill us with joy. The pains of the past are behind us. The future has yet to unfold. But the now is full of beauty simply waiting for our attention.

Amazon Q Developer is a machine learning-powered code generator that provides you with code recommendations in real time.
As you write code, Amazon Q Developer analyzes your code and comments as you write code in your integrated development environment (IDE). Amazon Q Developer then automatically generates suggestions based on your existing code and comments. Additionally, Amazon Q Developer analyzes the surrounding code, ensuring the generated code matches your style, naming conventions, and seamlessly integrates into the existing context.



Installation

Developers can easily access Amazon Q Developer by downloading and installing the AWS Toolkit IDE extension. You can also activate Amazon Q Developer from directly within the AWS Lambda and AWS Cloud9 console code editors.

Installation instructions
Installation instructions very depending on the environment. For more information, see “Setting up Amazon Q Developer” in the Amazon Q Developer User Guide by clicking on the button.
INSTALLATION

Compatibility
Amazon Q Developer seamlessly integrates with popular tools like Visual Studio (VS) Code, JetBrains IDEs (IntelliJ IDEA, PyCharm, etc.), Amazon SageMaker Studio, JupyterLab, AWS Cloud9, and the AWS Lambda console

Support
It supports a wide range of programming languages and development environments, including Python, Java, JavaScript, TypeScript, C#, Go, Rust, PHP, Ruby, Kotlin, C, C++, Shell scripting, SQL, and Scala.

Code Generation
User actions will vary depending on the tools you're integrating with. Take a look at the table below for an example of user actions available in VS Code.

User actions	
Previous and Next recommendation	Left arrow and Right arrow
Accept a recommendation	Tab
Reject a recommendation	ESC
Manually trigger code generation when typing a comment	MacOS [option]-[c], Windows [Alt]-[c]

The AWS Well-Architected Framework

The Well-Architected Framework is designed to enable architects, developers, and users of AWS to build secure, high-performance, resilient, and efficient infrastructure for their applications. It's composed of six pillars to ensure a consistent approach to reviewing and designing of architectures.

The first pillar is Operational Excellence, and it focuses on running and monitoring systems to deliver business value, and with that, continually improving processes and procedures. For example, automating changes with deployment pipelines or responding to events that are triggered.

The Second pillar is Security, and as you know, security is priority number one at AWS. And this pillar exemplifies it by checking integrity of data and, for example, protecting systems by using encryption.

The third pillar is Reliability, and it focuses on recovery planning, such as recovery from an Amazon DynamoDB disruption or an Amazon EC2 node failure. It also covers how you handle change to meet business and customer demand.

The fourth pillar is Performance Efficiency, and it entails using IT and computing resources efficiently. For example, using the right Amazon EC2 type based on workload and memory requirements. It also covers making informed decisions and maintaining efficiency as business needs evolve.

The fifth pillar is Cost Optimization, which looks at optimizing full cost. This is controlling where money is spent. And, for example, checking if you have overestimated your Amazon EC2 server size. You can then lower cost by choosing a more cost-effective size.

And last is the sixth pillar: Sustainability. The Sustainability pillar focuses on minimizing the environmental impacts of running cloud workloads. Reducing energy consumption and increasing efficiency are at the core of this pillar. So, you can design cloud architectures that reduce resource usage. And ultimately, design with the planet and your business in mind.

In the past, you'd need to evaluate these against your AWS infrastructure with the help of a solutions architect. Not that you can't and aren't still encouraged to do that. But we listened to customer feedback and decided to release the framework as a self-service tool, the Well-Architected Tool. You can access it with the AWS Management Console. Create a workload and run it against your AWS account to generate a report showing areas that should be addressed.

It kind of looks like a traffic light system, with green being go, keep up the good work. Orange being, you should probably look into this because there's room for improvement. And red being, okay, you should look at this because something is at risk. These are areas where the tool has detected potential issues, and it presents you with a plan on how to architect using established best practices. It should be noted that you can always override these settings if the questions don't apply to your scenario. It's very customizable. Interesting side note, where I'm from, South Africa, we call traffic lights robots.

If we take a look at the tool itself, you'll see something similar to the following screenshot. As I mentioned, we name our workload and that appears in Section One. Section Two shows you the pillars and drop-downs into the questions for each. Section Three shows the actual questions themselves. Four is the pillar and question stem. Five is a bit of background and the recommendation. Six is the answer you provide with a toggle to designate whether the question is applicable or not. This is important because it affects your overall score. Oh, and I can't forget Section Seven, where you'll be presented with short videos explaining how to answer a particular question.

Anyhow, that's the Well-Architected Framework and tool. Hope you've enjoyed learning how to evaluate your workloads.

The AWS Well-Architected Framework

The AWS Well-Architected Framework(opens in a new tab) helps you understand how to design and operate reliable, secure, efficient, and cost-effective systems in the AWS Cloud. It provides a way for you to consistently measure your architecture against best practices and design principles and identify areas for improvement.

The Well-Architected Framework is based on six pillars:

- Operational excellence
- Security
- Reliability
- Performance efficiency
- Cost optimization
- Sustainability

To learn more, expand each of the following six categories.

Operational excellence

Operational excellence is the ability to run and monitor systems to deliver business value and to continually improve supporting processes and procedures.

Design principles for operational excellence in the cloud include performing operations as code, annotating documentation, anticipating failure, and frequently making small, reversible changes.

Reliability

Reliability is the ability of a system to do the following:

- Recover from infrastructure or service disruptions
- Dynamically acquire computing resources to meet demand
- Mitigate disruptions such as misconfigurations or transient network issues

Reliability includes testing recovery procedures, scaling horizontally to increase aggregate system availability, and automatically recovering from failure.

Performance efficiency

Performance efficiency is the ability to use computing resources efficiently to meet system requirements and to maintain that efficiency as demand changes and technologies evolve.

Evaluating the performance efficiency of your architecture includes experimenting more often, using serverless architectures, and designing systems to be able to go global in minutes.

Cost optimization

Cost optimization is the ability to run systems to deliver business value at the lowest price point.

Cost optimization includes adopting a consumption model, analyzing and attributing expenditure, and using managed services to reduce the cost of ownership.

Sustainability

In December 2021, AWS introduced a sustainability pillar as part of the AWS Well-Architected Framework.

Sustainability is the ability to continually improve sustainability impacts by reducing energy consumption and increasing efficiency across all components of a workload by maximizing the benefits from the provisioned resources and minimizing the total resources required.

To facilitate good design for sustainability:

- Understand your impact
- Establish sustainability goals
- Maximize utilization
- Anticipate and adopt new, more efficient hardware and software offerings
- Use managed services

- Reduce the downstream impact of your cloud workloads

Benefits of the AWS Cloud

Alright, so now you've learned all there is to know about AWS and you're ready to go build your empire, right? Okay, so even if that isn't true you should still now have a good understanding of a lot of the different services AWS has to offer and how you can use them together like building blocks to build solutions for your business that scale, are flexible and are reliable. You also should by now understand some of the terminology AWS uses. Knowing this terminology is important for moving forward with your cloud journey. As you read blog posts or AWS documentation, you now should know a lot of the acronyms and phrases that you'll see. What you should also keep in mind as you move forward is the six main benefits of using the AWS Cloud. You'll start to evaluate what applications and solutions will be built in the Cloud. And knowing these benefits can really help you make those decisions in an informed way. Let's start with the first benefit.

This is a super important thing to remember. When you are building traditional on-premises data centers, you need to invest a large amount of money upfront just to get started. You need to invest cash into the actual data center physical space, all the hardware, the staff for racking and stacking that hardware and the overhead of keeping the data center running. For a medium to large business, this can be hundreds of thousands or millions of dollars. Then no matter what utilization is of that data center, you have a fixed cost and can set up your budget around that fixed cost. Billing with AWS is fundamentally different than the traditional billing model we just talked about. Your bill with AWS will be variable month to month as you consume more or less resources. Now the great thing about this is if you are just getting started there's no need to come up with a million dollars to invest into your AWS environment. Instead, you can start small, get billed for only what you use, and as you use more resources over time your bill will fluctuate with that growth. Now, if your bill gets out of your allotted budget, you can find ways to save money by turning off unused instances, deleting old resources that aren't being used, optimizing your applications, using tools like AWS Trusted Advisor to help you find opportunities to save money. Just because your bill was high one month, it doesn't mean you can't get it lowered the next month. And that's just not true with traditional on-premises data center billing. That's the first advantage. The next is

Benefit from massive economies of scale. This one is straight forward. AWS is building out massive amounts of capacity all around the world. And in order to build all those data centers, AWS is buying huge amounts of hardware. AWS is also an expert at building efficient data centers. We can buy the hardware at a lower price because of the massive volume, and then we install it and run it efficiently. Because of these factors, you can achieve a lower variable cost than you could running a data center on your own. Next up is

Stop guessing capacity. When you are building out a traditional data center, you need to estimate how much capacity you will need to run your business. Let's say you estimate you will have a user base of 10 million people over the next three years. You purchase enough hardware to support that growth over time and then turns out you only have

about 500,000 users, much lower than you anticipated. Well, guess what? You are still stuck with that cost and hardware for the 10 million users. You overestimated your capacity. The other side of the coin here is that you estimate your capacity too low, and you now need to scramble to build out your data center, to handle the higher capacity before your customers have a degraded experience or before you lose customers altogether. That takes time and you most likely won't keep up, or you will have to put in a heroic effort to achieve it. Guessing your capacity upfront can be problematic if you either over or underestimate. With AWS, you don't need to guess your capacity. Instead, provision the resources you need for the now and then use the scaling mechanisms for each resource to scale up or down based on the real life capacity it needs from day to day as scaling can take minutes with AWS instead of weeks or months with on-premises resources. The next is my personal favorite benefit.

Increase speed and agility. With AWS, it's easy to try new things. You can spin up test environments and run experiments on new ways to approach solving a problem. And then if that approach didn't work, you can just delete those resources and stop incurring cost. Traditional data centers don't offer the same flexibility. And therefore, if you want to run an experiment that will require new servers being bought and installed, the cost of the experiment failing is high because you already bought the server. The flexibility of AWS helps drive innovation as well as the ease of provisioning resources. Getting a new database stood up on-premises might take weeks, in AWS, it takes minutes. That ability to quickly get the resources you need helps you get to market quicker. All right, next up.

Stop spending money running and maintaining data centers. If you aren't a data center company, why spend so much money and time running data centers? Let AWS take the undifferentiated heavy lifting off your hands and instead focus on what makes your business valuable. What makes your offering better than your competitors? AWS lets you focus on finding the answer to that and delighting your customers with new innovative solutions, instead of focusing on running hardware in a data center. And last but not least, we have.

Go global in minutes. Let's say you're a company based in the US and you want to expand your operations to Germany. You have data centers in the US serving this region but you'll need data centers in Germany to serve that region. Traditionally, you would need to have staff in Germany running and operating a data center for you. With AWS, you can just replicate your architecture to a region in Germany. And if you're set up correctly, you can do this in an automated fashion using tools like AWS CloudFormation. The time it takes for you to expand into a secondary area of the world traditionally can be months or years. With AWS, it just takes minutes.

That's it. Those are the six main benefits of using AWS. And from all of us at AWS, congratulations on completing the AWS Cloud Practitioner Essentials course.

If you want to dive even deeper, please check out our other course offerings for more information on AWS products and services.

Advantages of cloud computing

Operating in the AWS Cloud offers many benefits over computing in on-premises or hybrid environments.

In this section, you will learn about six advantages of cloud computing:

- Trade upfront expense for variable expense.
- Benefit from massive economies of scale.
- Stop guessing capacity.
- Increase speed and agility.
- Stop spending money running and maintaining data centers.
- Go global in minutes.

To learn more about the advantages of cloud computing, expand each of the following six categories.

Trade upfront expense for variable expense.

Upfront expenses include data centers, physical servers, and other resources that you would need to invest in before using computing resources.

Instead of investing heavily in data centers and servers before you know how you're going to use them, you can pay only when you consume computing resources.

Benefit from massive economies of scale.

By using cloud computing, you can achieve a lower variable cost than you can get on your own.
Because usage from hundreds of thousands of customers aggregates in the cloud, providers such as AWS can achieve higher economies of scale. Economies of scale translate into lower pay-as-you-go prices.

Stop guessing capacity.

With cloud computing, you don't have to predict how much infrastructure capacity you will need before deploying an application.

For example, you can launch Amazon Elastic Compute Cloud (Amazon EC2) instances when needed and pay only for the compute time you use. Instead of paying for resources that are unused or dealing with limited capacity, you can access only the capacity that you need, and scale in or out in response to demand.

Stop spending money running and maintaining data centers.

Cloud computing in data centers often requires you to spend more money and time managing infrastructure and servers.

A benefit of cloud computing is the ability to focus less on these tasks and more on your applications and customers.

Go global in minutes.

The AWS Cloud global footprint enables you to quickly deploy applications to customers around the world, while providing them with low latency.

Exam Details

Exam domains

The AWS Certified Cloud Practitioner exam includes four domains:

1. 1

   Cloud Concepts

2. 2

   Security and Compliance

3. 3

   Technology

4. 4
   Billing and Pricing

The areas covered describe each domain in the Exam Guide(opens in a new tab) for the AWS Certified Cloud Practitioner certification. For a description of each domain, review the AWS Certified Cloud Practitioner website(opens in a new tab). (opens in a new tab)You are encouraged to read the information in the Exam Guide as part of your preparation for the exam.

Each domain in the exam is weighted. The weight represents the percentage of questions in the exam that correspond to that particular domain. These are approximations, so the questions on your exam may not match these percentages exactly. The exam does not indicate the domain associated with a question. In fact, some questions can potentially fall under multiple domains.

Domain	Percentage of exam
Domain 1: Cloud Concepts	24%
Domain 2: Security and Compliance	30%
Domain 3: Technology	34%
Domain 4: Billing and Pricing	12%
Total	100%

You are encouraged to use these benchmarks to help you determine how to allocate your time studying for the exam.

Recommended experience

Candidates for the AWS Certified Cloud Practitioner exam should have a basic understanding of IT services and their uses in the AWS Cloud platform.

We recommend that you have at least six months of experience with the AWS Cloud in any role, including project managers, IT managers, sales managers, decision makers, and marketers. These roles are in addition to those working in finance, procurement, and legal departments.

Exam details

The AWS Certified Cloud Practitioner exam consists of **65 questions** to be completed in **90 minutes**. The minimum passing score is **700** (the maximum score is 1,000).

Two types of questions are included on the exam: multiple choice and multiple response.

- A **multiple-choice** question has one correct response and three incorrect responses, or distractors.
- A **multiple-response** question has two or more correct responses out of five or more options.

On the exam, there is no penalty for guessing. Any questions that you do not answer are scored as incorrect. If you are not sure of what the correct answer is, it's always best for you to guess instead of leaving any questions unanswered.

The exam enables you to flag any questions that you'd like to review before submitting the exam. This helps you to use your time during the exam efficiently, knowing that you can always go back and review any questions that you were initially unsure of.

Whitepapers and resources

As part of your preparation for the AWS Certified Cloud Practitioner exam, we recommend that you review the following whitepapers and resources:

- Overview of Amazon Web Services(opens in a new tab)
- How AWS Pricing Works(opens in a new tab)
- Compare AWS Support Plans

Exam Strategies

This section explores several strategies that can help you to increase the probability of passing the exam.

To learn more about exam strategies, expand each of the following three categories.

Predict the answer before reviewing the response options.

Next, try to predict the correct answer before looking at any of the response options.

This strategy helps you to draw directly from your knowledge and skills without distraction from incorrect response options. If your prediction turns out to be one of the response options, this can be helpful for knowing whether you're on the right track. However, make sure that you review all the other response options for that question.

Eliminate incorrect response options.

Before selecting your response to a question, eliminate any options that you believe to be incorrect.

This strategy helps you to focus on the correct option (or options, for multiple-response questions) and ensure that you have fulfilled all the requirements of the question.